

ESERCIZI DE IL NUOVO JAVA

(www.nuovojava.it)

[Versione 6.0 12/2024]



EDITORE ULRICO HOEPLI MILANO

Indice degli esercizi

Esercizi de “Il nuovo Java”	VI
Introduzione	VI
Esercizi del capitolo 1	8
Introduzione a Java	8
Esercizi del capitolo 2	15
Componenti fondamentali di un programma Java	15
Esercizi del capitolo 3	25
Stile di codifica, tipi di dati ed array	25
Esercizi del capitolo 4	35
Operatori e Gestione del flusso di esecuzione	35
Esercizi del capitolo 5	46
Incapsulamento e visibilità	46
Esercizi del capitolo 6	59
Ereditarietà ed interfacce	59
Esercizi del capitolo 7	69
Polimorfismo	69
Esercizi del capitolo 8	78
Eccezioni ed asserzioni	78
Esercizi riassuntivi Parte II	86
Un esempio guidato alla programmazione ad oggetti	86
Esercizi del capitolo 9	104
Tipi Enumerazioni e Tipi Record	104
Esercizi del capitolo 10	112
Tipi Generici e Collezioni	112
Esercizi del capitolo 11 e dell’appendice E	123
Tipi Annotazioni, Reflection e la variabile d’ambiente CLASSPATH	123
Esercizi del capitolo 12	133
Gestione dei thread	133
Esercizi del capitolo 13 e dell’appendice F	140
Espressioni Lambda e package java.lang	140
Esercizi del capitolo 14 e dell’appendice H	160
Stream API e Framework Collections	160
Esercizi del capitolo 15 e dell’appendice I	168
Input-Output e Date-Time API	168
Esercizi del capitolo 16	185
Moduli	185
Esercizi del capitolo 17 e dell’appendice G	192
Java Database Connectivity e package java.util	192
Esercizi del capitolo 18 e dell’appendice M	198
Java e il mondo XML e JAXB	198
Esercizi del capitolo 19	199
Introduzione alle Interfacce grafiche (GUI)	199

Esercizi del capitolo 20	200
Introduzione a JavaFX	200

Esercizi de “Il nuovo Java”

Introduzione

Questo documento rappresenta, insieme al documento delle appendici ed al materiale extra online, il naturale completamento del libro “Il nuovo Java”.

Tutti gli esercizi sono stati spostati in questo documento per non togliere spazio alla teoria e non far lievitare il costo del libro.

Per ogni capitolo del libro (e per ogni appendice, dove applicabile) sono stati organizzati esercizi specifici per validare (ed ampliare) quanto appreso durante lo studio.

Gli esercizi sono fondamentali, imprescindibili. Infatti la teoria solitamente risulta chiara, ma applicare i concetti appresi è tutt’altro che facile. La programmazione non comprende solo l’implementazione del codice, bensì molte altre componenti ne fanno parte e influiscono sul risultato finale.

Gli esercizi che troverete in questo documento quindi si sforzano di insistere anche su argomenti che altri libri trattano superficialmente, o non trattano per niente, come l’analisi, la progettazione, l’architettura object oriented. Inoltre esistono esercizi per tutti gli argomenti, anche per quelli che possono sembrare banali al lettore più esperto, ma che possono risultare fondamentali per il neofita. In questa edizione, troverete oltre 500 esercizi, che saranno continuamente aggiornati in quantità e qualità.

Questo documento quindi sarà probabilmente aggiornato in futuro. Questa filosofia di rilascio incrementale, ci permetterà di eseguire ulteriori verifiche sulla qualità e la correttezza del contenuto. L’autore si riserva di notificare qualsiasi novità o informazione, tramite i suoi canali social ed in particolare sul canale ufficiale Telegram dedicato alle news su “Il nuovo Java”, all’indirizzo <https://t.me/nuovojava>.

PS: non è un canale interattivo, riceverete solo notifiche da parte dell’autore.

Gli esercizi sono stati progettati per soddisfare tutte le tipologie di lettori. Sono stati introdotti anche centinaia di esercizi a risposta singola o multipla, sulla falsariga dei test delle certificazioni OCA (Oracle Certification Associate) e OCP (Oracle Certified Programmer). Tali esercizi supportano la preparazione alla certificazione, e sono stati impostati in modo tale da far acquisire sicurezza al programmatore. Spesso consistono essenzialmente nel leggere del codice e comprenderne il significato nei dettagli, e rappresentano anche per i più esperti un banco di prova notevole. Siamo sicuri che il lettore apprezzerà lo sforzo compiuto per realizzare un così gran numero di esercizi eterogenei e tutti originali.

Diversamente da molti altri testi, abbiamo comunque fornito delle soluzioni per tutti gli esercizi (con poche eccezioni). Ovviamente quando si tratta di codificare una soluzione, esistono centinaia di valide alternative, quindi non bisogna considerare le soluzioni proposte come quelle oggettivamente migliori. È possibile scaricare tutti i listati delle soluzioni e degli esercizi (e tante altre risorse) all’indirizzo: <https://www.nuovojava.it>.

Le soluzioni sono in un documento separato, perché riteniamo più semplice così consultare la soluzione dopo aver svolto un certo esercizio.

Ricordiamo ancora una volta che, soprattutto per chi è alle prime armi, è importante iniziare a scrivere tutto il codice a mano, senza copia-incolla o particolari aiuti da parte dello strumento di sviluppo che si sta utilizzando. È anche molto importante commentare il proprio codice riga per riga, soprattutto all'inizio. Questo vi permetterà di fissare bene le definizioni, e ad avere maggiore sicurezza nello scrivere codice.

Sarà sufficiente scrivere il codice sorgente su un editor di testo come Blocco Note di Windows (come descritto nel modulo 1) e compilare da riga di comando (si raccomanda la lettura dell'appendice A).

Sconsigliamo di svolgere gli esercizi di questi primissimi capitoli (diciamo i primi 4) utilizzando un IDE complesso come Eclipse o Netbeans... si potrebbe finire a studiare l'IDE piuttosto che Java. È invece consigliato (se non si vuole avere troppo a che fare con Blocco Note e la riga di comando) l'utilizzo di EJE (<https://sourceforge.net/projects/eje>) che offre delle semplici utility pensate proprio per chi inizia a programmare. Per esempio consente di compilare e mandare in esecuzione i nostri file con la pressione di due semplici pulsanti.

EJE è di semplice installazione (basta decomprimere il file in una cartella qualsiasi) ed utilizzo.

Infine suggeriamo, dopo aver svolto un esercizio, di consultare la relativa soluzione prima di passare al successivo (spesso c'è propedeuticità tra un esercizio e il successivo), o almeno di leggere le soluzioni.

Per qualsiasi tipo di segnalazione, è possibile scrivere direttamente all'autore all'indirizzo: claudio@claudiodesio.com. È anche possibile contattarlo tramite i più importanti social network e sul sito personale dove è presente un blog ed è possibile iscriversi alla sua newsletter per essere sempre aggiornati su novità, corsi, iniziative, articoli e tanto altro:

Internet (blog, corsi, newsletter, etc.):

<https://www.claudiodesio.com>

Facebook:

<https://www.facebook.com/claudiodesiocesari>

Twitter:

<https://twitter.com/cdesio>

LinkedIn:

<https://www.linkedin.com/in/claudiodesio>

YouTube:

<https://www.youtube.com/claudiodesiocesari>

Instagram:

<https://www.instagram.com/nuovojava>

<https://www.instagram.com/javaforaliens>

<https://www.instagram.com/cdesio>

Buon lavoro!

Claudio De Sio Cesari

Esercizi del capitolo 1

Introduzione a Java

I seguenti esercizi sono stati pensati per chi inizia da zero. Questi hanno l'unico scopo di far prendere un minimo di confidenza con l'ambiente di programmazione Java e la scrittura del codice. Ricordiamo ancora una volta che, soprattutto se si sta iniziando, è importante iniziare a scrivere tutto il codice a mano, senza copia incolla, o particolari aiuti da parte del tool di sviluppo che si sta utilizzando.

Sarà sufficiente scrivere il codice sorgente su un editor di testo come il blocco note di Windows (come descritto nel capitolo 1) e compilare da riga di comando (si raccomanda la lettura dell'appendice A).

Sconsigliamo di svolgere gli esercizi dei primi 4 capitoli utilizzando un IDE complesso come Eclipse o Netbeans, a meno che non siate già programmatori esperti. Infatti bisognerebbe studiare anche l'IDE, e perdere di vista l'obiettivo principale, quello di studiare Java.

È invece consigliato (se non si vuole avere troppo a che fare con il blocco note e la riga di comando) l'utilizzo di EJE, che offre delle semplici utility pensate proprio per chi inizia a programmare. Per esempio permette di compilare e mandare in esecuzione i nostri file con la pressione di due semplici pulsanti.

EJE è disponibile per il download gratuito all'indirizzo <https://sourceforge.net/projects/eje>. Non c'è bisogno di installazione, basta decomprimere il file ZIP scaricato in una cartella qualsiasi. Per eseguirlo bisogna fare un doppio clic sul file `eje.bat`. Potete leggere il file `LEGGIMI.txt`, per lanciarlo su piattaforme diverse da Windows, o per avere una panoramica delle sue caratteristiche.

Infine, dopo aver svolto un esercizio, suggeriamo di consultare la soluzione relativa prima di passare al successivo (spesso c'è propedeuticità tra un esercizio e il successivo). Questo consiglio vale per tutti gli esercizi di tutti i capitoli e appendici.

Esercizio 1.a)

Digitare, salvare, compilare ed eseguire il programma `HelloWorld`. Il lettore dovrà eseguire questo esercizio due volte: la prima volta utilizzando il Notepad e la prompt dei comandi (cfr. paragrafo 1.5), e la seconda utilizzando EJE.

EJE permette di inserire parti di codice pre-formattate tramite il menu `Inserisci` (o tramite shortcut). Nel video a questo [link](#), viene mostrato come creare questo programma in pochi secondi con le shortcut. Il video è in inglese ma è possibile attivare i sottotitoli in italiano.

Esercizio 1.b) *Concetti base di informatica, Vero o Falso:*

1. Un computer è composto da hardware e software.

2. Il sistema operativo fa parte dell'hardware di un computer. Infatti un computer non può funzionare senza sistema operativo.
3. Blocco Note di Windows è un software.
4. Il cavo dell'alimentatore di un computer è un hardware.
5. Il linguaggio macchina è il linguaggio che il processore di un computer riesce ad interpretare.
6. Il linguaggio macchina è unico e standard.
7. Il linguaggio macchina ha un vocabolario formato solo da due simboli 0 e 1.
8. Sia il compilatore che l'interprete hanno il compito di tradurre le istruzioni scritte con un certo linguaggio di programmazione in istruzioni in linguaggio macchina.
9. In generale, un programma scritto in un linguaggio interpretato ha un tempo di esecuzione (runtime) più veloce rispetto ad un programma scritto con un linguaggio compilato.
10. Un programma eseguibile è composto dai suoi file sorgente.

Esercizio 1.c) Caratteristiche di Java, Vero o Falso:

1. Java è il nome di una tecnologia e contemporaneamente il nome di un linguaggio di programmazione.
2. Java è un linguaggio interpretato ma non compilato.
3. Java è un linguaggio veloce ma non robusto.
4. Java è un linguaggio difficile da imparare perché in ogni caso obbliga ad imparare l'Object Orientation.
5. La Java Virtual Machine è un software che supervisiona il software scritto in Java.
6. La JVM gestisce la memoria automaticamente mediante la Garbage Collection.
7. L'indipendenza dalla piattaforma è una caratteristica poco importante.
8. Java è un sistema chiuso.
9. La Garbage Collection garantisce l'indipendenza dalla piattaforma.
10. Java è un linguaggio gratuito che raccoglie le caratteristiche migliori di altri linguaggi, e ne esclude quelle ritenute peggiori e più pericolose.

Esercizio 1.d) Codice Java, Vero o Falso:

1. La seguente dichiarazione del metodo `main` è corretta:
`public static main(String argomenti[]) {...}`
2. La seguente dichiarazione del metodo `main` è corretta:
`public static void Main(String args[]){...}`
3. La seguente dichiarazione del metodo `main` è corretta:
`public static void main(String argomenti[]) {...}`
4. La seguente dichiarazione del metodo `main` è corretta:
`public static void main(String Argomenti[]) {...}`
5. La seguente dichiarazione di classe è corretta:
`public class {...}`
6. La seguente dichiarazione di classe è corretta:
`public Class Auto {...}`
7. La seguente dichiarazione di classe è corretta:
`public class Auto {...}`
8. È possibile dichiarare un metodo al di fuori del blocco di codice che definisce una classe.
9. Il blocco di codice che definisce un metodo è delimitato da due parentesi tonde.
10. Il blocco di codice che definisce un metodo è delimitato da due parentesi quadre.

Esercizio 1.e) Ambiente e processo di sviluppo, Vero o Falso:

1. La JVM è un software che simula un hardware.
2. Il bytecode è contenuto in un file con suffisso .class.
3. Lo sviluppo Java consiste nello scrivere il programma, salvarlo, mandarlo in esecuzione ed infine compilarlo.
4. Lo sviluppo Java consiste nello scrivere il programma, salvarlo, compilarlo ed infine mandarlo in esecuzione.
5. Il nome del file che contiene una classe Java deve coincidere con il nome della classe, anche se non si tiene conto delle lettere maiuscole e minuscole.
6. Una volta compilato un programma scritto in Java è possibile eseguirlo su di un qualsiasi sistema operativo che abbia una JVM.
7. Per eseguire una qualsiasi applicazione Java basta avere un browser.
8. Il compilatore del JDK viene invocato tramite il comando `javac` e la JVM viene invocata tramite il comando `java`.
9. Per mandare in esecuzione un file che si chiama `Pippo.class`, dobbiamo eseguire il seguente comando dalla prompt: `java Pippo.java`.
10. Per mandare in esecuzione un file che si chiama `Pippo.class`, dobbiamo eseguire il seguente comando dalla prompt: `java Pippo.class`.

Esercizio 1.f)

Eliminare il modificatore `static` del metodo `main` dalla classe `HelloWorld`. Utilizzando la prompt dei comandi, compilare ed eseguire il programma, e interpretare il messaggio di errore dell'esecuzione.

Saper interpretare i messaggi di errore è assolutamente fondamentale. Nell'approfondimento 1.3, trovate la descrizione degli errori che si incontrano più frequentemente (sia in fase di compilazione che in fase di esecuzione del programma) quando si inizia a studiare Java.

Esercizio 1.g)

Eliminare la prima parentesi graffa aperta incontrata dalla classe `HelloWorld`. Utilizzando la prompt dei comandi, compilare il programma e interpretare il messaggio di errore della compilazione.

Esercizio 1.h)

Eliminare l'ultima parentesi chiusa (ultimo simbolo del programma) dalla classe `HelloWorld`. Utilizzando la prompt dei comandi, compilare il programma e interpretare il messaggio di errore.

Esercizio 1.i)

Eliminare il simbolo di `;` dalla classe `HelloWorld`. Utilizzando la prompt dei comandi compilare il programma e interpretare il messaggio di errore.

Esercizio 1.j)

Raddoppiare la parentesi graffa finale della classe `HelloWorld`. Utilizzando la prompt dei comandi compilare il programma e interpretare il messaggio di errore.

Esercizio 1.k)

Aggiungere all'interno del metodo `main`, un blocco di parentesi graffe:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        System.out.println(args);  
    }  
}
```

Compilare, eseguire e trarre delle conclusioni.

Esercizio 1.l)

Raddoppiare il simbolo di `;` nel programma `HelloWorld` compilare ed eseguire, cosa succede?

Esercizio 1.m)

Scrivere il programma `HelloWorld` scrivendo ogni parola e ogni simbolo al rigo successivo. C'è qualche problema?

Esercizio 1.n)

Provare a far stampare una stringa a piacere al programma `HelloWorld` al posto della stringa `Hello World!`.

Esercizio 1.o)

Provare a far stampare un numero al programma `HelloWorld` al posto della stringa `Hello World!`.

Esercizio 1.p)

Provare a far stampare la somma di due numeri al programma `HelloWorld` al posto della stringa `Hello World!`, dopo aver letto la soluzione dell'esercizio precedente 1.o.

Esercizio 1.q)

Compilare ed eseguire il seguente programma:

```
public class HelloWorld {  
    public static void main(String args[]) {  
  
    }  
}
```

cosa viene stampato?

Esercizio 1.r)

Compilare ed eseguire il seguente programma:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        System.out.println("");  
    }  
}
```

cosa viene stampato?

Esercizio 1.s)

Compilare ed eseguire il seguente programma:

```
public class HelloWorld {  
    public static void main(String args[]) {  
        System.out.println(args);  
    }  
}
```

cosa viene stampato?

Esercizio 1.t)

Scrivere il programma `Elenco` che stampa un elenco della spesa, dove ogni articolo da comprare risiede sul proprio rigo.

Esercizio 1.u)

Scrivere il programma `ElencoCompatto` che stampa un elenco della spesa, dove ogni articolo è separato da un altro con una virgola.



Esercizio 1.v)

Creare un nuovo file di nome `SayJava` che stampa la stringa “JAVA” come nel seguente esempio:

```
-----  
|  JAVA  |  
-----
```

Esercizio 1.w)

Scrivere un programma definito dalla classe `Frecce` che stampi il seguente output:

```
<-----<<<  
  
>>>----->
```

Esercizio 1.x)

Scrivere un programma definito dalla classe `StampaContatti` che stampi l'elenco di contatti di una rubrica. Ogni contatto dovrà essere stampato su tre righe: nella prima ci sarà il nome del contatto, nella seconda l'indirizzo, e nella terza il numero di telefono. Ogni contatto deve essere separato dal successivo da una riga vuota.

**Esercizio 1.y)**

Creare una classe chiamata `StampaClasseRigaVuota` che stampi il seguente output:

```
public class RigaVuota {  
    public static void main(String args[]) {  
        System.out.println();  
    }  
}
```

**Esercizio 1.z)**

Creare un nuovo file di nome `SayMyName` che stampa il vostro nome come nel seguente esempio:

```
***** *      *      *      *      *      *      *  
*      *      *      *      *      *      *      *  
*      *      *      *      *      *      *      *  
*      *      *      *      *      *      *      *  
***** ***** *      *      *      *      *      *
```

Ovviamente se il vostro nome è troppo lungo può bastare anche solo un soprannome, o le prime lettere. Questo esercizio è importante perché ci costringerà a compilare ed eseguire continuamente l'applicazione per verificare che l'output sia quello desiderato. Questo ci permetterà di fare esperienza, con l'ambiente di sviluppo.

Si noti che per stampare ogni carattere sono state utilizzate 5 colonne e 5 righe (ad eccezione del carattere I). Ogni carattere è separato da un altro di 3 colonne.

Se si vuole eseguire quest'applicazione con EJE, si consiglia di eseguire quest'applicazione dopo aver settato il font *Consolas* su Windows e Linux (*Monaco* su MAC). In alternativa potete eseguire l'applicazione da riga di comando.

Esercizio 1.aa)

Creare la classe `SommaTreInteri` come descritto nel paragrafo 1.5.3.

Esercizi del capitolo 2

Componenti fondamentali di un programma Java

Ricordiamo che molti esercizi sono propedeutici ai successivi, quindi non conviene saltarne qualcuno o quantomeno vi consigliamo di consultare le soluzioni prima di andare avanti.



Esercizio 2.a)

Viene fornita (copiare, salvare e compilare) la seguente classe:

```
public class NumeroIntero {  
    public int numeroIntero;  
    public void stampaNumero() {  
        System.out.println(numeroIntero);  
    }  
}
```

Questa classe definisce il concetto di numero intero come oggetto. In essa vengono dichiarati una variabile intera ed un metodo che stamperà la variabile stessa.

Scrivere, compilare ed eseguire una classe che:

1. istanzierà almeno due oggetti dalla classe `NumeroIntero` (contenente un metodo `main`);
2. cambierà il valore delle relative variabili e testerà la veridicità delle avvenute assegnazioni, sfruttando il metodo `stampaNumero`;
3. aggiungerà un costruttore alla classe `NumeroIntero` che inizializzi la variabile d'istanza.

Due domande ancora:

4. a che tipologia di variabili appartiene la variabile `numeroIntero` definita nella classe `NumeroIntero`?
5. Se istanziamo un oggetto della classe `NumeroIntero`, senza assegnare un nuovo valore alla variabile `numeroIntero`, quanto varrà quest'ultima?

Esercizio 2.b) Concetti sui componenti fondamentali, Vero o Falso:

1. Una variabile d'istanza deve essere per forza inizializzata dal programmatore.
2. Una variabile locale condivide il ciclo di vita con l'oggetto in cui è definita.
3. Un parametro ha un ciclo di vita coincidente con il metodo in cui è dichiarato: nasce quando il metodo viene invocato, muore quando termina il metodo.
4. Una variabile d'istanza appartiene alla classe in cui è dichiarata.
5. Un metodo è sinonimo di azione, operazione.
6. Sia le variabili sia i metodi sono utilizzabili di solito mediante l'operatore `dot`, applicato ad un'istanza della classe dove sono stati dichiarati.
7. Un costruttore è un metodo che non restituisce mai niente, infatti ha come tipo di ritorno `void`.
8. Un costruttore viene detto di default, se non ha parametri.
9. Un costruttore è un metodo e quindi può essere utilizzato mediante l'operatore `dot`, applicato ad un'istanza della classe dove è stato dichiarato.

10. Un package è fisicamente una cartella che contiene classi, le quali hanno dichiarato esplicitamente di far parte del package stesso nei rispettivi file sorgente.

Esercizio 2.c) Sintassi dei componenti fondamentali. Vero o Falso:

1. Nella dichiarazione di un metodo (non costruttore), il nome è sempre seguito dalle parentesi che circondano i parametri opzionali, ed è sempre preceduto da un tipo di ritorno.
2. Il seguente metodo è dichiarato in maniera corretta:

```
public void metodo() {  
    return 5;  
}
```
3. Il seguente metodo è dichiarato in maniera corretta:

```
public int metodo() {  
    System.out.println("Ciao");  
}
```
4. La seguente variabile è dichiarata in maniera corretta:

```
public int a = 0;
```
5. La seguente variabile `x` è utilizzata in maniera corretta (fare riferimento alla classe `Punto` definita in questo capitolo):

```
Punto p1 = new Punto();  
Punto.x = 10;
```
6. La seguente variabile `x` è utilizzata in maniera corretta (fare riferimento alla classe `Punto` definita in questo capitolo):

```
Punto p1 = new Punto();  
Punto.p1.x = 10;
```
7. La seguente variabile `x` è utilizzata in maniera corretta (fare riferimento alla classe `Punto` definita in questo capitolo):

```
Punto p1 = new Punto();  
x = 10;
```
8. Il seguente costruttore è utilizzato in maniera corretta (fare riferimento alla classe `Punto` definita in questo capitolo):

```
Punto p1 = new Punto();  
p1.Punto();
```
9. Il seguente costruttore è dichiarato in maniera corretta:

```
public class Computer {  
    public void Computer() {  
    }  
}
```
10. Il seguente costruttore è dichiarato in maniera corretta:

```
public class Computer {  
    public computer(int a) {  
  
    }  
}
```




Esercizio 2.d)

Creare una classe `Quadrato`, che dichiari una variabile d'istanza intera `lato`. Quindi creare un metodo pubblico che si chiami `perimetro` e che ritorni il perimetro del quadrato, e un metodo pubblico `area` che ritorni l'area del quadrato.

Ricordiamo per chi lo ha dimenticato, che il perimetro è la somma dei lati del quadrato, mentre l'area si calcola moltiplicando il lato per sé stesso. Infine ricordiamo che il simbolo per eseguire una moltiplicazione in Java è il `*`.

Esercizio 2.e)

Creare una classe `TestQuadrato` che contenga un metodo `main` che istanzi un oggetto di tipo `Quadrato`, con lato di valore 5. Quindi stampare il perimetro e l'area dell'oggetto appena creato.

Esercizio 2.f)

Dopo aver svolto l'esercizio precedente, dovrete aver impostato la variabile `lato` a mano con un'istruzione come la seguente:

```
nomeOggetto.lato = 5;
```

Per poter evitare questa istruzione si crei un costruttore nella classe `Quadrato` dell'esercizio 2.d, che prenda in input il valore della variabile `lato`. Dopo aver finito, si compili la classe `Quadrato`. La classe `TestQuadrato` invece, non compilerà più per via dell'istruzione specificata sopra e per il mancato utilizzo del nuovo costruttore. Si modifichi il codice della classe `TestQuadrato` in modo tale che compili e sia eseguita correttamente.

Esercizio 2.g)

Nella classe `Quadrato` creata nell'esercizio 2.d, sostituire il valore 4 usato per calcolare il perimetro, con una costante d'istanza `NUMERO_LATI`.

Si noti che per la costante è stato usato un nome costituito da sole lettere maiuscole separate con un simbolo di sottolineatura (underscore). Questa è una convenzione che si usa per le costanti, come spiegato nel paragrafo 3.1.

Questo non dovrebbe influire sulla classe `TestQuadrato`.

Esercizio 2.h)

Creare una classe `Rettangolo`, equivalente alla classe `Quadrato` creata nell'esercizio 2.d e perfezionata negli esercizi successivi. Prima di codificare la classe decidere che specifiche deve avere questa classe (variabili e metodi).

Mai “buttarsi sul codice” direttamente. È un errore classico che soprattutto all'inizio può portare a perdersi. Bisogna prima definire nella propria mente, o

ancora meglio su un foglio di carta, le specifiche. Il consiglio è quello di avere ben chiare le varie definizioni (variabili d'istanza, variabili locali, metodi, costruttori etc.).

Esercizio 2.i)

Creare una classe `TestRettangolo` che contenga un metodo `main` e che testi la classe `Rettangolo`, equivalentemente a come abbiamo fatto nell'esercizio 2.f. Questa volta si istanzino almeno due rettangoli diversi.



Esercizio 2.j)

Aggiungere alle classi `Quadrato` e `Rettangolo` create negli esercizi precedenti, un metodo chiamato `stampaDettagli`, che stampa i dettagli della figura geometrica in questione, compresi perimetro e area. Creare anche una nuova versione delle classi `TestQuadrato` e `TestRettangolo` che richiama direttamente i metodi `stampaDettagli` sugli oggetti istanziati.



Esercizio 2.k)

Partendo dalla soluzione dell'esercizio precedente, creare un ulteriore metodo nelle classi `Quadrato` e `Rettangolo`, chiamato `dammiDettagli`, che restituisce la stessa stringa che veniva stampata nel metodo `stampaDettagli`. Dopo averlo creato, fare in modo che il metodo `stampaDettagli`, sfrutti il metodo `dammiDettagli` in modo tale da non duplicare il codice. Creare una classe chiamata `TestQuadrilateri` che stampa i dettagli di un quadrato e di un rettangolo.

Esercizio 2.l)

Si astragga con una classe il concetto di `Nazione`, creando almeno un costruttore e delle variabili d'istanza, ma nessun metodo.



Esercizio 2.m)

Dopo aver creato la classe `Nazione` dell'esercizio 2.l, riuscite a creare uno o più metodi? Se ci riuscite, definiteli all'interno della classe. Se non ci riuscite, sapete spiegare il perché?

Esercizio 2.n)

Data la seguente classe:

```
public class Esercizio2N {  
    public String string;  
    public int intero;  
    final public String integer = "inizializzazione";  
}
```

Quale delle seguenti affermazioni è vera (scegliere una sola affermazione)?

1. Il codice può essere compilato correttamente.
2. Il codice non può essere compilato correttamente perché non è possibile dichiarare una variabile con nome `string`.
3. Il codice non può essere compilato correttamente perché non è possibile dichiarare una variabile con nome `intero`.
4. Il codice non può essere compilato correttamente perché non è possibile dichiarare una variabile di tipo `String` chiamandola `integer`.
5. Il codice non può essere compilato correttamente perché la variabile con nome `integer` dichiara i modificatori in ordine inverso (dovrebbe essere prima dichiarata `public` e poi `final`).

Esercizio 2.o)

Data la seguente classe:

```
public class Esercizio2O {  
    public String toString() {  
        return "Esercizio2O"  
    }  
  
    public void main() {  
  
    }  
  
    public static void metodo() {  
  
    }  
  
    static public void main(String argomenti[]) {  
  
    }  
}
```

C'è un solo errore in questa classe che ne impedirà la compilazione, quale?

Se non si è in grado di rispondere alla domanda, scrivere e compilare la classe a mano, e interpretare l'errore. Poi risolverlo e verificare la risoluzione compilando.

Esercizio 2.p)

Data la seguente classe:

```
public class Esercizio2P {  
    public String stringa;  
  
    public static void main(String argz[]) {
```

```
        public int intero = 0;
    }
}
```

C'è un solo errore in questa classe che ne impedirà la compilazione, quale?

Se non si è in grado di rispondere alla domanda, scrivere e compilare la classe a mano, e interpretare l'errore. Poi risolverlo e verificare la risoluzione compilando.

Esercizio 2.q)

Data la seguente classe:

```
public class Esercizio2Q {

    public static void main(String args) {
        System.out.println("Quelo")
    }
}
```

Ci sono ben tre errori che in questa classe ne impediranno la compilazione, quali sono?

Se non si è in grado di rispondere alla domanda, scrivere e compilare la classe a mano, e interpretare l'errore. Poi risolvere gli errori e verificare la risoluzione compilando.

Esercizio 2.r)

Data la seguente classe:

```
public class Esercizio2R {

    public int var1;
    public int var2;

    System.out.println("Esercizio 2.r");

    public Esercizio2R() {

    }

    public Esercizio2R(int a , int b) {
        var1 = b;
        var2 = a;
    }

    public static void main(String args[]) {
        Esercizio2R esercizio2R = new Esercizio2R (4,7);
        System.out.println(esercizio2R.var1);
        System.out.println(esercizio2R.var2);
    }
}
```

Una volta eseguito, cosa stamperà questo programma?

1. Questo programma non può essere eseguito.
2. Questo programma non compila.
3. Stamperà 74.

4. Stamperà 47.

Esercizio 2.s)

Date le seguenti classi:

```
public class Corso {
    public String nome;

    public Corso() {

    }

    public Corso(String n) {
        nome = n;
    }
}

public class Esercizio2s {

    public static void main(String args[]) {
        Corso corso1 = new Corso();
        corso1.nome = "Java";
        Corso corso2 = new Corso("Java");
        System.out.println(corso1.nome);
        System.out.println(corso2.nome);
    }
}
```

Quale sequenza di istruzioni tra le seguenti serve per eseguire il programma?

1. javac Corso.java, javac Esercizio2S.java, java Corso.
2. javac Corso.java, javac Esercizio2S.java, java Corso.class.
3. javac Corso.java, javac Esercizio2S.java, java Esercizio2S.
4. javac Corso.java, javac Esercizio2S.java, java Esercizio2S Corso.

Esercizio 2.t)

Date le seguenti classi:

```
public class Corso {
    public String nome;

    public Corso() {

    }

    public Corso(String n) {
        nome = n;
    }
}

public class Esercizio2t {

    public static void main(String args[]) {
        Corso corso1 = new Corso();
        corso1.nome = "Java";
        Corso corso2 = new Corso("Java");
        System.out.println(corso1.nome);
        System.out.println(corso2.nome);
    }
}
```

```
}  
}
```

Una volta eseguito, cosa stamperà questo programma?

1. Questo programma non può essere eseguito.
2. Questo programma non compila.
3. Stamperà:

```
Java  
Java
```

4. Stamperà:

```
Java
```

Esercizio 2.u)

Data la seguente classe:

```
public class Esercizio2u {  
    int c = 3;  
    public static void main(String args[]) {  
        int a = 1;  
        int b = 2, c, d = 4;  
        System.out.println(a+b+c+d);  
    }  
}
```

Una volta eseguito, cosa stamperà questo programma?

1. Questo programma non compila.
2. Stamperà "10".
3. Stamperà: "7".
4. Stamperà: "0".

Esercizio 2.v)

Si crei una classe di nome `Esercizio2V` che consenta di ottenere la somma 2, 3, 5 e 10 numeri interi.

Esercizio 2.w)

Consideriamo la soluzione dell'esercizio 1.x, dove avevamo creato una classe che simulava la stampa dei dettagli di alcuni contatti di una rubrica:

```
public class StampaContatti {  
    public static void main(String args[]) {  
        System.out.println("Elenco Contatti");  
        System.out.println();  
        System.out.println("Claudio De Sio Cesari");  
        System.out.println("Via Java 13");  
        System.out.println("131313131313");  
        System.out.println();  
        System.out.println("Stevie Wonder");  
        System.out.println("Via Musica 10");  
        System.out.println("1010101010");  
        System.out.println();  
        System.out.println("Gennaro Capuozzo");  
        System.out.println("Piazza Quattro Giornate 1");  
        System.out.println("1111111111");  
    }  
}
```

```
}
```

Astrarre, salvare, e compilare una classe `Contatto` che contiene le necessarie variabili e uno o più costruttori.



Esercizio 2.x)

Consideriamo la soluzione dell'esercizio 2.w, creiamo una nuova versione della classe `StampaContatti` dell'esercizio 1.x (il cui listato è riportato anche nella traccia dell'esercizio 2.w), questa volta sfruttando la classe `Contatto` creata nell'esercizio 2.w. l'output del programma deve essere lo stesso del programma `StampaContatti` dell'esercizio 1.x.

Consiglio: utilizzare un metodo simile a `stampaDettagli` che abbiamo definito nella soluzione dell'esercizio 2.k.



Esercizio 2.y)

Considerando la soluzione dell'esercizio 2.x, creare una classe `Rubrica`, che contenga i contatti creati nell'esercizio 2.x. Essa deve definire un costruttore senza parametri che istanzi le proprie variabili di istanza. Creare una nuova versione della classe `StampaContatti`, che abbia sempre lo stesso output. Tale versione non deve istanziare gli oggetti `Contatto`, ma prenderli dalla classe `Rubrica`.



Esercizio 2.z)

Creare una classe che astragga il concetto di “città”, chiamandola `Citta` (Java non supporta le lettere accentate per i nomi). Dopodiché dichiarare una classe `Nazione` che dichiari una variabile d'istanza `capitale` di tipo `Citta`. Infine creare una classe `Esercizio2Z` che istanzi una `Nazione` con una capitale, e stampi una frase che verifichi l'effettiva associazione tra la nazione e la capitale.

Si consiglia di creare dei costruttori per tali classi.

Esercizio 2.aa)

Creare una classe `Libro`, che definisce le variabili che si ritiene opportuno, un costruttore che ci permetta di inizializzarle, ed un metodo `stampaDettagli`, sul modello del metodo definito nella classe `Contatto` dell'esercizio 2.y. Creare anche una classe `TestLibro` che testa il funzionamento

della classe `Libro`, che istanzia due libri e chiami il metodo `stampaDettagli` per ognuno di loro. Controllare se la stampa ottenuta è quella desiderata.

Esercizio 2.bb)

Creare una classe `CalcolaPerimetroQuadrato`, che calcola il perimetro di un quadrato che abbia come lunghezza del lato il valore 17, e ne stampi il risultato. Usare questa volta solo una classe con metodo `main`, che dichiari delle variabili in maniera leggibile.

È possibile anche settare il valore del lato tramite un oggetto `Scanner` come fatto negli esempi del capitolo 1.

Esercizio 2.cc)

Consideriamo la classe `Aritmetica` definita nel paragrafo 2.2.2.3:

```
public class Aritmetica {  
    public int somma(int a, int b) {  
        return (a + b);  
    }  
}
```

Creare una versione “maggiormente leggibile” di questa classe, così come spiegato nel paragrafo 2.2.2.3.

Esercizio 2.dd)

Consideriamo la classe `Aritmetica` definita nel paragrafo 2.2.2.3:

```
public class Aritmetica {  
    public int somma(int a, int b) {  
        return (a + b);  
    }  
}
```

Creare una versione che rappresenta un compromesso tra la versione leggibile e la versione sintetica di questa classe, così come spiegato nel paragrafo 2.2.2.3.

Esercizi del capitolo 3

Stile di codifica, tipi di dati ed array

Di seguito una serie di esercizi per fare pratica con quanto appreso nel capitolo 3. Ricordiamo che molti esercizi sono propedeutici ai successivi, quindi non conviene saltarne qualcuno o quantomeno vi consigliamo di consultare le soluzioni prima di andare avanti.



Esercizio 3.a)

Scrivere un semplice programma che svolga le seguenti operazioni aritmetiche correttamente, scegliendo accuratamente i tipi di dati da utilizzare per immagazzinare i risultati di esse.

1. Una divisione (usare il simbolo `/`) tra due interi `a = 5`, e `b = 3`. Immagazzinare il risultato in una variabile `r1`, scegliendone il tipo di dato opportunamente.
2. Una moltiplicazione (usare il simbolo `*`) tra un `char c = 'a'`, ed uno `short s = 5000`. Immagazzinare il risultato in una variabile `r2`, scegliendone il tipo di dato opportunamente.
3. Una somma (usare il simbolo `+`) tra un `int i = 6` ed un `float f = 3.14F`. Immagazzinare il risultato in una variabile `r3`, scegliendone il tipo di dato opportunamente.
4. Una sottrazione (usare il simbolo `-`) tra `r1`, `r2` e `r3`. Immagazzinare il risultato in una variabile `r4`, scegliendone il tipo di dato opportunamente.

Verificare la correttezza delle operazioni stampandone i risultati parziali e il risultato finale. Tenere presente la promozione automatica nelle espressioni, e utilizzare il cast opportunamente. È richiesta la creazione di una sola classe con un `main` che svolga le operazioni.



Esercizio 3.b)

Scrivere un programma con i seguenti requisiti:

1. Implementare una classe `Persona` che dichiari le variabili `nome`, `cognome`, `eta` (età). Si dichiari inoltre un metodo `dettagli` che restituisca in una stringa le informazioni sulla persona in questione.

Ricordarsi di utilizzare le convenzioni e le regole descritte in questo capitolo.

2. Implementare una classe `Principale` che, nel metodo `main`, istanzi due oggetti chiamati `persona1` e `persona2` della classe `Persona`, inizializzando per ognuno di essi i relativi campi utilizzando l'operatore `dot`.
3. Dichiarare un terzo reference (`persona3`) che punti ad uno degli oggetti già istanziati. Controllare che effettivamente `persona3` punti all'oggetto voluto, stampando i campi di `persona3` sempre mediante l'operatore `dot`.

4. Commentare adeguatamente le classi realizzate e sfruttare lo strumento javadoc per produrre la relativa documentazione.

Nella documentazione standard di Java sono usate tutte le regole e le convenzioni descritte in questo capitolo. Basta osservare che `String` inizia con lettera maiuscola, essendo una classe. Si può concludere che anche `System` è una classe.

Esercizio 3.c) Array, Vero o Falso:

1. Un array è un oggetto e quindi può essere dichiarato, istanziato ed inizializzato.
2. Un array bidimensionale è un array i cui elementi sono altri array.
3. Il metodo `length` restituisce il numero degli elementi di un array.
4. Un array non è ridimensionabile.
5. Un array è eterogeneo di default.
6. Un array di interi può contenere come elementi `byte`, ovvero le seguenti righe di codice non producono errori in compilazione:

```
int arr [] = new int[2];
byte a = 1, b = 2;
arr [0] = a;
arr [1] = b;
```
7. Un array di interi può contenere come elementi `char`, ovvero le seguenti righe di codice non producono errori in compilazione:

```
char a = 'a', b = 'b';
int arr [] = {a,b};
```
8. Un array di stringhe può contenere come elementi `char`, ovvero le seguenti righe di codice non producono errori in compilazione:

```
String arr [] = {'a' , 'b'};
```
9. Un array di stringhe è un array bidimensionale, perché le stringhe non sono altro che array di caratteri. Per esempio:

```
String arr [] = {"a" , "b"};
```

è un array bidimensionale.
10. Se abbiamo il seguente array bidimensionale:

```
int arr [][]= {
    {1, 2, 3},
    {1,2},
    {1,2,3,4,5}
};
```

risulterà che:

```
arr.length = 3;
arr[0].length = 3;
arr[1].length = 2;
arr[2].length = 5;
arr[0][0] = 1;
arr[0][1] = 2;
arr[0][2] = 3;
arr[1][0] = 1;
arr[1][1] = 2;
arr[1][2] = 3;
arr[2][0] = 1;
arr[2][1] = 2;
arr[2][2] = 3;
```

```
arr[2][3] = 4;  
arr[2][4] = 5;
```

Esercizio 3.d)

Creare una classe `StampaMioNome` con un metodo `main`, che stampa il vostro nome usando un array di caratteri.



Esercizio 3.e)

Creare una classe `Risultato` che dichiara una sola variabile d'istanza di tipo `float` di nome `risultato`. Aggiungere eventuali metodi e costruttori utili. Creare una classe `CambiaRisultato` che dichiara un metodo `public` di nome `cambiaRisultato` che prende in input un oggetto di tipo `Risultato` e ne modifica la variabile interna `risultato` sommandola con un altro valore. Creare una classe con un metodo `main` di nome `TestRisultato` che stampi la variabile `risultato` di un oggetto di tipo `Risultato`, prima e dopo che questo oggetto sia passato in input al metodo `cambiaRisultato` di un oggetto di tipo `CambiaRisultato`.



Esercizio 3.f)

Dopo aver svolto l'esercizio precedente, aggiungere alla classe `CambiaRisultato` un metodo che si chiama sempre `cambiaRisultato` che però prende in input una variabile `float` e ne modifica il risultato.

Creare poi una classe `TestRisultatoFloat` equivalente, che esegua le stesse operazioni della classe `TestRisultato` realizzata nell'esercizio precedente.

Esercizio 3.g)

Creare una classe `TestArgs` con un metodo `main` che stampa la variabile `args[0]`. Quindi testarla passando vari input da riga di comando (cfr. paragrafo 3.6.7).

Esercizio 3.h)

La seguente classe dichiara vari identificatori di tipo stringa:

```
public class Esercizio3H {  
    public String Break,  
        String,  
        character,  
        bit,
```

```
continues,  
exports,  
Class,  
imports,  
_AAA_,  
_@_,  
_;  
}
```

Sono tutti validi? Una volta individuati quelli non validi, usare opportunamente i commenti per escluderli dalla compilazione.

Esercizio 3.i)

Data la seguente classe:

```
package com.claudiodesio.manuale.esercizi;  
  
public class Esercizio3I {  
    public final String nomeLinguaggio = "Java 14";  
    public int intero;  
    public void stampaStringa() {  
        System.out.println(nomeLinguaggio);  
    }  
}
```

C'è qualche convenzione non rispettata per i nomi? Se sì, quale e come modificare per risolvere? Se non si rispettano le convenzioni dei nomi, il codice non compila?

Esercizio 3.j)

Considerando l'esercizio 1.y, creare una classe chiamata `StampaClasseRigaVuota` che stampi il seguente output:

```
public class RigaVuota {  
    public static void main(String args[]) {  
        System.out.println("");  
    }  
}
```



Esercizio 3.k)

Data la seguente classe:

```
public class Esercizio3K {  
    public static void main(String args[]) {  
        char d = (char)100;  
        float f = (float)d*1_000_000_000;  
        System.out.println((long)f);  
    }  
}
```

Quale dei seguenti output produrrà una volta eseguita?

1. Nessuno, avremo un errore in compilazione per un errore di sintassi alla seconda riga del metodo `main`.
2. Un numero imprecisato.
3. 1000000000000
4. 100_000_000_000
5. 100.000.000.000
6. 100,000,000,000
7. 660000000000
8. 0.0
9. Un carattere Unicode non prevedibile.
10. Il codice compila ma al runtime avremo una eccezione alla prima riga del metodo `main` perché non è possibile fare un cast di un `int` in un `char`.
11. Il codice compila ma al runtime avremo una eccezione alla seconda riga del metodo `main` perché non è possibile fare un cast di un `int` in un `float`.
12. Il codice compila ma al runtime avremo una eccezione alla terza riga del metodo `main` perché non è possibile fare un cast di un `float` in un `long`.
13. Il codice non compila per altre ragioni.
14. Il codice va in eccezione al runtime per altre ragioni.

Esercizio 3.l)

Data la seguente classe:

```
public class Esercizio3L {  
    public static void main(String args[]) {  
        bit i1 = 8;  
        short i2 = -1024;  
        integer i3 = 638;  
        long i5 = 888_666_777;  
        float i6 = 0;  
        double i7 = 0x11B;  
    }  
}
```

Le variabili all'interno del metodo `main` sono tutte dichiarate correttamente? E i valori assegnati sono tutti entro l'intervallo di rappresentazione dei rispettivi tipi?



Esercizio 3.m)

Data la seguente classe:

```
public class Esercizio3M {  
    public static void main(String args[]) {  
        boolean b = true;  
        char c = 'I';  
        System.out.println(b);  
        System.out.println(c+1);  
    }  
}
```

Tenendo presente che la lettera `I`, è codificata tramite il numero decimale 73, quale dei seguenti output produrrà una volta eseguita?

1. Nessuno, avremo un errore in compilazione.
2. `true` alla prima riga e `74` alla seconda.
3. `true` alla prima riga e `L` alla seconda.
4. `true` alla prima riga e `J` alla seconda.
5. `0` alla prima riga e `74` alla seconda.
6. `0` alla prima riga e `J` alla seconda.
7. `0` alla prima riga e `L` alla seconda.

Esercizio 3.n)

Data la seguente classe:

```
public class Esercizio3N {  
    public static void main(String args[]) {  
        String s = "Jav";  
        char c = "a";  
        System.out.println(s+c+1);  
    }  
}
```

quale dei seguenti output produrrà una volta eseguita?

1. Nessuno, avremo un errore in compilazione.
2. Java
3. Java1
4. Javb

Esercizio 3.o)

Data la classe:

```
package parcheggio;  
  
public class Auto {  
    public String tipo;  
  
    public Auto(String t) {  
        tipo = t;  
    }  
}
```

Affinché la seguente classe compili:

```
package laboratori;  
//Inserisci il codice qui  
  
public class Autista {  
    public void guida(Auto auto) {  
        System.out.println("Sto guidando l'auto " + auto.tipo);  
    }  
}
```

Bisogna inserire una riga di codice. Quale (o quali, potrebbero essere valide anche più di una) tra le seguenti righe permetterebbe alla classe `Autista` di essere compilata?

1. `import parcheggio.Auto;`
2. `import parcheggio.*;`
3. `import parcheggio. laboratori.*;`

4. `import parcheggio.Auto.*;`
5. `import parcheggio.*.Auto;`
6. `import laboratori.parcheggio.Auto;`

Esercizio 3.p)

Considerando le classi `Auto` e `Autista` dell'esercizio precedente, che pezzo di codice bisogna aggiungere alla seguente classe:

```
public class Esercizio3P {  
    //Inserisci il codice qui  
  
    public static void main(String args[]) {  
        Auto auto = new Auto("Toyota Yaris");  
        Autista autista = new Autista();  
        autista.guida(auto);  
    }  
}
```

Scegliere una delle seguenti opzioni:

1. `import parcheggio.*;`
2. `import laboratori.*;`
3. `import parcheggio.Auto;` e `import laboratori.Autista;`
4. Non bisogna aggiungere codice. Il codice è già compilabile.

Esercizio 3.q)

La seguente affermazione è corretta?

Quando passiamo un reference di un oggetto in input ad un metodo, siamo sicuri che terminata l'esecuzione del metodo, il nostro reference continuerà a puntare sempre allo stesso oggetto a cui puntava prima dell'esecuzione del metodo. Questo non significa che la struttura interna dell'oggetto, non possa essere modificata all'interno del metodo. Infatti il parametro locale del metodo, assumerà lo stesso indirizzamento del reference passato, e potrà quindi lavorare sullo stesso oggetto.

Esercizio 3.r)

Scrivere un programma che prenda in input un argomento (variabile `args` del metodo `main`) e lo immagazzini come terzo elemento di un array di stringhe locale chiamato per l'appunto `array`.

Questo programma funzionerà solo se viene passato almeno un argomento. Eventuali parametri passati da riga di comando oltre al primo, saranno ignorati dal programma.

Esercizio 3.s)

Data la classe:

```
public class Auto {  
    public String tipo;  
  
    public Auto(String t) {  
        tipo = t;  
    }  
}
```

Creare una classe `Nave` che astragga il concetto di nave che carica automobili. Questa classe deve definire un metodo `caricaAuto` a cui sarà passato un oggetto `Auto`. Ogni oggetto `auto` verrà immagazzinato in una variabile d'istanza array chiamata `autoArray`, nel primo posto disponibile dell'array. Creare anche una classe di test che è possibile chiamare `Esercizio3S`.



Esercizio 3.t)

Creare un programma `Esercizio3T` che deve essere eseguito passando un parametro da riga di comando rappresentante un numero intero, nel seguente modo:

```
java Esercizio3T 9
```

Con EJE è possibile passare parametri con la scorciatoia di tastiera Maiuscolo - F9, ovvero con il menu esegui con argomenti (execute with args).

Si può sostituire il numero intero 9 con un qualsiasi altro numero intero.

Il programma dovrà:

- utilizzare `args[0]`, che contiene un numero intero, per creare un array di interi della stessa dimensione dell'argomento;
- stampare una frase che confermerà la creazione dell'array, stampandone la dimensione.

Visto che i parametri vengono immagazzinati negli elementi dell'array di stringhe `args`, bisogna convertire il parametro passato in input da stringa ad intero. Cercare nella libreria il metodo `parseInt` della classe `Integer`. Leggere la documentazione, capire come funziona ed utilizzarlo nel programma.

Nel caso ci si trovi in difficoltà si esegua una ricerca su Google per cercare aiuto, il web è pieno di soluzioni.

Esercizio 3.u)

Creare un programma che:

- crei un array di tipi carattere, con tutte le lettere dell'alfabeto;
- utilizzi un metodo (statico) della classe `java.util.Arrays` per stampare il suo contenuto sotto forma di stringa. Cercare nella documentazione ufficiale il metodo adatto allo scopo.

Nel caso ci si trovi in difficoltà si esegua una ricerca su Google per cercare aiuto, il web è pieno di soluzioni.

Esercizio 3.v)

Creare una classe con metodo main, che quando viene lanciata, stampi un numero casuale di tipo intero.

Suggerimento: esiste un metodo di una classe nella libreria Java che fa esattamente questo. Per trovarlo, si parta dal tradurre in inglese la parola “casuale” e si cerchi nella documentazione.

**Esercizio 3.w)**

Data la seguente classe:

```
public class Esercizio3W {  
    public static void main(String args[]) {  
        var var = "var";  
        var a = " ";  
        var b = " = ";  
        var c = 8;  
        var d = ";";  
        var e = c + d;  
        System.out.println(var + a + "i" + b + c +d);  
    }  
}
```

È possibile compilare questa applicazione?

1. No, avremo un errore in compilazione per un errore di sintassi alla prima riga del metodo main.
2. No, avremo un errore in compilazione per un errore di sintassi alla seconda riga del metodo main.
3. No, avremo un errore in compilazione per un errore di sintassi alla terza riga del metodo main.
4. No, avremo un errore in compilazione per un errore di sintassi alla quarta riga del metodo main.
5. No, avremo un errore in compilazione per un errore di sintassi alla quinta riga del metodo main.
6. No, avremo un errore in compilazione per un errore di sintassi alla sesta riga del metodo main.
7. No, avremo un errore in compilazione per un errore di sintassi alla settima riga del metodo main.
8. Sì.

Esercizio 3.x)

Nella classe `Rubrica` sostituire le tre variabili d'istanza `contatto1`, `contatto2` e `contatto3` con un array e modificare la classe `StampaContatti` di conseguenza. Riscrivere la soluzione dell'Esercizio 2.y, utilizzando la parola `var`, ovunque sia possibile usarla.

Esercizio 3.y)

Quali dei seguenti snippet può essere compilato senza errori:

1. `public class var {}`
2. `private class var {}`
3. `public var MiaClasse {}`
4. `var var[] = new int[8];`
5. `public var var = 1;`

**Esercizio 3.z)**

Creare una classe `Pagella` che astragga il concetto di pagella scolastica. Essa deve avere le seguenti informazioni:

- nome, cognome e classe dell'alunno;
- una tabella che associ per ogni materia, il voto e il giudizio.

Deve inoltre dichiarare un metodo che stampi in maniera leggibile i dati della pagella.

Creare anche una classe `Esercizio3Z` che stampi una o più pagelle.

Esercizio 3.aa)

Creare una classe `CalcolaAreaTriangolo`, che calcola l'area di un triangolo che abbia come base il valore 3.8, come altezza il valore 5.4, e ne stampi il risultato. Usare solo una classe con metodo `main`, che dichiari delle variabili in maniera leggibile.

È possibile anche settare i valori delle variabili tramite un oggetto `Scanner`.

Esercizi del capitolo 4

Operatori e Gestione del flusso di esecuzione

Dopo aver studiato il capitolo 4 dovremmo già essere in grado di poter scrivere programmi usando il linguaggio Java. Quello che manca ancora sono le nozioni di Object Orientation che si studieranno per bene a partire dal prossimo capitolo. Intanto prendiamo dimestichezza con il flusso di esecuzione.



Esercizio 4.a)

Scrivere un semplice programma, costituito da un'unica classe, che sfruttando esclusivamente un ciclo infinito, l'operatore modulo, due costrutti `if`, un `break` ed un `continue`, stampi solo i primi cinque numeri pari.

Esercizio 4.b)

Scrivere un'applicazione che stampi i 26 caratteri dell'alfabeto (inglese) con un ciclo.



Esercizio 4.c)

Scrivere una semplice classe che stampi a video la tavola pitagorica.

Suggerimento 1: non sono necessari array.

Suggerimento 2: il metodo `System.out.println` stampa l'argomento che gli viene passato e poi sposta il cursore alla riga successiva; infatti `println` sta per "print line". Esiste anche il metodo `System.out.print` che invece stampa solamente il parametro passato.

Suggerimento 3: sfruttare un doppio ciclo innestato.

Esercizio 4.d) Operatori e flusso di esecuzione, Vero o Falso:

1. Gli operatori unari di pre-incremento e post-incremento applicati ad una variabile danno lo stesso risultato, ovvero se abbiamo:

```
int i = 5;  
sia  
i++;
```

- sia
++i;
aggiornano il valore di i a 6;
2. `d += 1` è equivalente a `d++`, dove `d` è una variabile `double`.
 3. Se abbiamo:

```
int i = 5;  
int j = ++i;  
int k = j++;  
int h = k--;
```

`boolean flag = ((i != j) && ( (j <= k) || (i <= h) ));`
`flag` avrà valore `false`.
 4. L'istruzione:

```
System.out.println(1 + 2 + "3");
```

stamperà 33.
 5. Il costrutto `switch` può in ogni caso sostituire il costrutto `if`.
 6. L'operatore ternario può in ogni caso sostituire il costrutto `if`.
 7. Il costrutto `for` può in ogni caso sostituire il costrutto `while`.
 8. Il costrutto `do` può in ogni caso sostituire il costrutto `while`.
 9. Il costrutto `switch` può in ogni caso sostituire il costrutto `while`.
 10. I comandi `break` e `continue` possono essere utilizzati nei costrutti `switch`, `for`, `while` e `do` ma non nel costrutto `if`.

Esercizio 4.e)

Modificare la classe `TestArgs` creata nell'esercizio 3.g, in modo tale da evitare eccezioni al runtime, con un costrutto imparato in questo capitolo.

Esercizio 4.f)

È buona norma aggiungere la clausola `default` ad un costrutto `switch`. Sapreste spiegare perché?

Esercizio 4.g)

È buona norma aggiungere la clausola `else` ad un costrutto `if`. Sapreste spiegare perché?

Esercizio 4.h)

Creare una classe con un metodo `main` che, utilizzando un ciclo `for`, selezioni i primi 10 numeri divisibili per 3, e li stampi dopo averli concatenati con una stringa in modo tale che l'output del programma sia:

```
Numero multiplo di 3 = 3  
Numero multiplo di 3 = 6  
Numero multiplo di 3 = 9  
Numero multiplo di 3 = 12  
Numero multiplo di 3 = 15
```

```
Numero multiplo di 3 = 18
Numero multiplo di 3 = 21
Numero multiplo di 3 = 24
Numero multiplo di 3 = 27
Numero multiplo di 3 = 30
```

Usare un ciclo `for` ordinario.

Esercizio 4.i)

Ripetere l'esercizio 4.h usando un ciclo `while` invece del ciclo `for`.

Esercizio 4.j)

Ripetere l'esercizio 4.h usando un ciclo `do-while` invece del ciclo `for`.

Esercizio 4.k)

Creare una classe chiamata `PariODispari` che definisca un metodo chiamato `dammiPariODispari`, che restituisca in maniera casuale la stringa "Pari" o "Dispari". Creare anche una classe di test che invocando tale metodo stampi il risultato.

Suggerimento: vedere al soluzione dell'esercizio 3.v.

Esercizio 4.l)

Sfruttando la classe `PariODispari` creata nell'esercizio precedente, creare una classe chiamata `TestaOCroce` che definisca un metodo chiamato `dammiTestaOCroce` che utilizzando un'espressione `switch`, restituisca la stringa "Testa" o "Croce". Creare anche una classe di test che invocando tale metodo stampi il risultato.



Esercizio 4.m)

Si consideri il seguente codice:

```
import java.util.Scanner;

public class ProgrammaInterattivo {
    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        String stringa = "";
        System.out.println("Digita qualcosa e batti enter, oppure scrivi "
            + "\"fine\" per terminare il programma");
        while(!(stringa = scanner.next()).equals("fine")) {
```

```
        System.out.println("Hai digitato " + stringa.toUpperCase() + "!");
    }
    System.out.println("Fine programma!");
}
}
```

Questa classe legge l'input da tastiera mediante la classe `Scanner` del package `java.util`. Il metodo `next` usato nel costrutto `while` (con una sintassi complessa che comprende anche l'assegnazione alla variabile `stringa`) è un metodo bloccante (ovvero che blocca l'esecuzione del codice in attesa di input da parte dell'utente) che legge l'input da tastiera sino a quando si preme il tasto `enter` (invio). Il programma termina quando si digita la parola "fine".

Modificare il programma precedente in modo tale che diventi un moderatore di parole, ovvero che censuri alcune parole digitate.

Esercizio 4.n)

Quali dei seguenti operatori può essere utilizzato con variabili booleane?

1. +
2. %
3. ++
4. /=
5. &
6. =
7. !!
8. >>>

Esercizio 4.o)

Qual è l'output del seguente programma?

```
public class Esercizio40 {
    public static void main(String args[]) {
        int i = 99;
        if (i++ >= 100) {
            System.out.println(i+=10);
        } else {
            System.out.println(--i==99?i++:++i);
        }
    }
}
```

Scegliere tra le seguenti opzioni:

1. 10
2. 101
3. 99
4. 110



Esercizio 4.p)

Qual è l'output del seguente programma?

```
public class Esercizio4P {  
    public static void main(String args[]) {  
        int i = 22;  
        int j = i++%3;  
        i = j!=0?j:i;  
        switch (i) {  
            case 1:  
                System.out.println(8<<2);  
            case 0:  
                System.out.println(8>>2);  
                break;  
            case 2:  
                System.out.println(i!=j);  
                break;  
            case 3:  
                System.out.println(++j);  
                break;  
            default:  
                System.out.println(i++);  
                break;  
        }  
    }  
}
```

Scegliere tra le seguenti opzioni:

1. 24
2. 6 e al rigo successivo 10
3. 10
4. true
5. false
6. 22
7. 21
8. 32 e al rigo successivo 2

Esercizio 4.q)

Si scriva un programma che chieda all'utente di inserire il numero di giorni passati dall'ultima vacanza fatta. Una volta inserito questo numero, il programma dovrà rispondere quanti minuti sono passati dall'ultima vacanza.

**Esercizio 4.r)**

Data la seguente classe:

```
public class Esercizio4R {  
  
    private static int matrice[][] = {  
        {1, 7, 3, 9, 5, 3},  
        {6, 2, 3},  
    }  
}
```

```
        {7, 5, 1, 4, 0},  
        {1, 0, 2, 9, 6, 3, 7, 8, 4}  
    };  
  
    public static void main(String args[]) {  
  
    }  
}
```

implementare il metodo `main` in modo che legga un numero (tra 0 e 9) come parametro `args[0]`, e trovi la posizione (riga e colonna) della prima occorrenza del numero specificato all'interno dell'array bidimensionale denominato `matrice`.



Esercizio 4.s)

La soluzione dell'esercizio precedente fallisce nel momento in cui:

- non si specifica un argomento da riga di comando;
- si specifica da riga di comando un argomento intero che non rientra nel range 0-9;
- si specifica da riga di comando un argomento che non è un numero intero;

Aggiungere, alla soluzione dell'esercizio 4.r, il codice che gestisce i tre casi (specificando un messaggio con le istruzioni da seguire per il corretto utilizzo).

Il terzo caso può essere gestito con quanto studiato finora, ma più avanti studieremo dei metodi per semplificare il nostro codice. In particolare nel capitolo 8 dove si parla di gestione delle eccezioni, e nell'appendice G in cui parleremo delle regular expression, vedremo che esistono delle soluzioni piuttosto semplici per gestire il terzo caso.

Esercizio 4.t)

Dichiarare una classe `CalcolatriceSemplificata` che dati due numeri definisca i metodi per:

- Sommarli.
- Sottrarre il secondo dal primo.
- Moltiplicarli.
- Dividerli.
- Restituire il resto della divisione.
- Restituire il numero più grande (il massimo).
- Restituire il numero più piccolo (il minimo).
- Restituire la media dei due numeri.

Creare una classe che testa il funzionamento di tutti i metodi.



Esercizio 4.u)

Dichiarare una classe che utilizza la classe `Scanner`, che permette all'utente di interagire con la classe `CalcolatriceSemplificata`: l'utente deve poter scrivere il primo operando, selezionare da una lista l'operazione da eseguire e specificare il secondo operando. Il programma deve restituire il giusto risultato.

Attenzione che i metodi bloccanti di `Scanner` leggono le pressioni dei tasti. Questo significa che a volte troveremo caratteri non riconosciuti in output, in corrispondenza della pressione di tasti come Maiuscolo, o Canc.

Esercizio 4.v)

Dichiarare una classe `StranaCalcolatrice` che dati un numero imprecisato di numeri definisca i metodi per:

1. Sommarli.
2. Moltiplicarli.
3. Restituire il numero più grande (il massimo).
4. Restituire il numero più piccolo (il minimo).

**Esercizio 4.w)**

Dichiarare una classe che utilizza la classe `Scanner`, che permette all'utente di interagire con la classe `StranaCalcolatrice`. Al lettore è lasciata la decisione di come l'utente dovrà interagire con il programma. Il suggerimento è quello di scrivere un algoritmo su un foglio di carta prima di iniziare a scrivere il codice.

Attenzione che i metodi bloccanti di `Scanner` leggono le pressioni dei tasti. Questo significa che a volte troveremo caratteri non riconosciuti in output, in corrispondenza della pressione di tasti come Maiuscolo, o Canc.

**Esercizio 4.x)**

Utilizzando la classe `TestaOCroce` definita nella soluzione dell'esercizio 4.1, creare una classe chiamata `GiocoTestaOCroce`, che simuli il lancio di una moneta, e che permetta all'utente di

indovinare se uscirà “Testa” o “Croce”. Il programma dovrà stampare un messaggio finale che stabilisca che se l’utente ha vinto (ovvero ha indovinato) oppure no.



Esercizio 4.y)

Modificare la classe `GiocoTestaOCroce` creato nell’esercizio precedente, in modo tale che il programma inizialmente permetta di specificare il numero di lanci da fare. Il programma dovrà contare il numero di volte che l’utente ha indovinato il risultato del lancio e il numero di volte che non ha indovinato, e dovrà decidere se l’utente ha vinto la partita oppure no.



Esercizio 4.z)

Riprendendo la classe `Rubrica` creata nell’esercizio 3.x, creare un metodo chiamato `cercaContattiPerNome(String nome)` che prende come input una stringa che può rappresentare un nome o una parte di esso, e deve restituire un array di oggetti `Contatto` che contengano la stringa nel proprio nome. Creare anche una classe di test chiamata `RicercaContatti` che permette all’utente di specificare la stringa da passare come criterio di ricerca al metodo `cercaContattiPerNome` e che stampi i risultati della ricerca.

Esercizio 4.aa) JShell, Vero o Falso:

1. JShell è un programma che si trova nella directory bin del JDK, come i comandi java e javac, e lo possiamo richiamare direttamente da riga di comando, perché abbiamo settato correttamente la variabile PATH alla cartella bin a cui appartiene.
2. JShell è un IDE.
3. In una sessione JShell non è possibile dichiarare package.
4. Il simbolo di terminazione di uno statement ; si può omettere solo se scriviamo un’istruzione Java su di un’unica riga.
5. Dichiarare due volte la stessa variabile è possibile, perché JShell segue regole diverse rispetto al compilatore. L’ultima variabile dichiarata sovrascriverà la precedente.
6. Se dichiariamo una variabile di tipo `String` senza inizializzarla, essa sarà inizializzata automaticamente a `null`.
7. Non è possibile dichiarare un’interfaccia in una sessione JShell.
8. Il modificatore `abstract` è sempre ignorato all’interno di una sessione JShell.
9. Non è possibile dichiarare un metodo `main` in una sessione JShell.
10. È possibile dichiarare annotazioni ed enumerazioni in una sessione JShell.

Esercizio 4.bb) JShell comandi, Vero o Falso:

1. Per terminare una sessione JShell, bisogna digitare il comando `goodbye`.
2. Tutti i comandi di JShell devono avere come prefisso il simbolo `\`.
3. I comandi `help` e `?` sono equivalenti.
4. Il comando `history` mostra tutti gli snippet e tutti i comandi lanciati dall'utente nella sessione corrente. Accanto agli snippet è presente uno snippet id che permette di richiamare lo snippet corrispondente.
5. Il comando `list` mostra tutti i comandi digitati dall'utente in questa sessione.
6. Se lanciamo il comando `/types -all` elencherà tutte le variabili dichiarate nella sessione corrente.
7. Il comando `reload` provocherà che tutte le istruzioni eseguite nella sessione corrente siano eseguite nuovamente.
8. Il comando `drop` può cancellare un certo snippet specificando il suo snippet id.
9. Se specifichiamo il comando `/set start JAVASE`, importeremo in questa ed in tutte le altre sessioni future di JShell, tutte i package di Java Standard Edition.
10. Con il comando `/!` richiamiamo l'ultimo snippet editato, sia esso valido o non valido.

Esercizio 4.cc)

Consideriamo le seguenti righe editate all'interno di una sessione JShell:

```
jshell> public int a;  
a ==> 0  
  
jshell> private String a  
a ==> null  
  
jshell> /reset  
| Resetting state.  
  
jshell> /list
```

Quale sarà l'output del comando `list` finale?

Esercizio 4.dd)

Considerando tutte le istruzioni del precedente esercizio, quale sarà l'output del seguente comando?

```
jshell> /history
```

Esercizio 4.ee)

Se volessimo aprire il file `HelloWorld.java` (realizzato nel primo capitolo) all'interno di JShell che si trova nella cartella corrente, che comando dobbiamo lanciare?

1. `/save HelloWorld.java`
2. `/retain HelloWorld.java`
3. `/reload HelloWorld.java`

4. `/open HelloWorld.java`
5. `/start HelloWorld.java`
6. `/env HelloWorld.java`
7. `/!`

Esercizio 4.ff)

Una volta aperto il file `HelloWorld.java` all'interno di una sessione JShell, come possiamo far stampare la stringa `Hello World!`?

Esercizio 4.gg)

Che comando dobbiamo lanciare se vogliamo copiare il file `HelloWorld.java` aperto nell'esercizio 4.ee in una cartella `C:/cartella`?

1. `/save HelloWorld.java`
2. `/retain HelloWorld.java`
3. `/save HelloWorld2.java`
4. `/save C:/cartella/HelloWorld.java`
5. `/save -start start C:/cartella/HelloWorld.java`
6. `/env C:/cartella/HelloWorld.java`
7. `/! C:/cartella/HelloWorld.java`

Esercizio 4.hh) JShell strumenti ausiliari, Vero o Falso:

1. In una sessione JShell è possibile dichiarare una variabile senza specificare un reference .
2. In una sessione JShell è possibile dichiarare un reference senza specificare il tipo del reference .
3. In una sessione JShell è possibile scrivere un valore, poi con il tasto TAB far dedurre automaticamente il tipo della variabile a JShell, per poi scrivere il nome del reference.
4. Mentre si dichiara un reference ad un tipo non importato, è possibile farci suggerire da JShell una lista di possibili `import` da usare per il tipo, premendo contemporaneamente i tasti SHIFT e TAB, rilasciarli e dopo premere il tasto v.
5. Il comando `/edit` aprirà il programma Notepad++.
6. In una sessione JShell la pressione contemporanea dei tasti CTRL – BARRA SPAZIATRICE provoca l'auto-completamento del codice che si è iniziato a scrivere, o nel caso di più opzioni disponibili, la scelta tra esse.
7. In una sessione JShell la pressione contemporanea dei tasti CTRL – E provoca lo spostamento del cursore al termine della riga.
8. In una sessione JShell la pressione contemporanea dei tasti ALT – D provoca la cancellazione della parola alla destra del cursore.
9. Il comando `/set feedback silent` provoca che JShell eviti di stampare i messaggi dell'analisi del codice scritto.
10. Scrivendo `System` e premendo due volte il tasto TAB, JShell ci mostrerà la documentazione della classe `System`.

**Esercizio 4.ii)**

Sfruttando la proprietà dei tipi `char` di essere compatibili con il tipo intero, scrivere una classe con il metodo `main`, che sfruttando un ciclo `for`, inizializzi la seguente variabile:

```
char alfabeto [] = new char[26];
```

I caratteri dell'alfabeto sono 26 e non 21 perché contiamo anche i caratteri `j`, `k`, `w`, `x` e `y`.

Esercizio 4.jj)

Considerato la soluzione dell'esercizio 4.jj, aggiungere il ciclo più semplice che stampa tutti gli elementi dell'array `alfabeto`.

Esercizi del capitolo 5

Incapsulamento e visibilità

È ora di entrare pian piano nella giusta mentalità object oriented. Alcune delle soluzioni degli esercizi che troverete di seguito, evidenzieranno le tipiche difficoltà che si incontrano quando si programma, e come evitarle.

Dopo ogni esercizio conviene consultare anche la soluzione, perché spesso un esercizio è propedeutico al successivo.

Esercizio 5.a) *Object Orientation in generale (teoria), Vero o Falso:*

1. L'Object Orientation esiste solo da pochi anni.
2. Java è un linguaggio object oriented non puro, SmallTalk è un linguaggio object oriented puro.
3. Tutti i linguaggi orientati agli oggetti supportano allo stesso modo i paradigmi object oriented.
4. Si può dire che un linguaggio è object oriented se supporta incapsulamento, ereditarietà e polimorfismo; infatti altri paradigmi come l'astrazione e il riuso appartengono anche alla filosofia funzionale.
5. Applicare l'astrazione significa concentrarsi solo sulle caratteristiche importanti dell'entità da astrarre.
6. La realtà che ci circonda è fonte d'ispirazione per la filosofia object oriented.
7. L'incapsulamento ci aiuta ad interagire con gli oggetti, l'astrazione ci aiuta ad interagire con le classi.
8. Il riuso è favorito dall'implementazione degli altri paradigmi object oriented.
9. L'incapsulamento divide gli oggetti in due parti separate: l'interfaccia pubblica e l'implementazione interna.
10. Per l'utilizzo dell'oggetto basta conoscere l'implementazione interna, non è necessario conoscere l'interfaccia pubblica.

Esercizio 5.b)

Incapsulare e completare le seguenti classi:

```
public class Pilota {
    public String nome;

    public Pilota(String nome) {
        // impostare il nome
    }
}

public class Auto {
    public String scuderia;
    public Pilota pilota;

    public Auto(String scuderia, Pilota pilota) {
```

```
        // impostare scuderia e pilota
    }
    public String dammiDettagli() {
        // restituire una stringa descrittiva dell'oggetto
    }
}
```

Tenere presente che le classi Auto e Pilota devono poi essere utilizzate dalle seguenti classi:

```
public class TestGara {
    public static void main(String args[]) {
        Gara imola = new Gara("GP di Imola");
        imola.corriGara();
        String risultato = imola.getRisultato();
        System.out.println(risultato);
    }
}

public class Gara {
    private String nome;
    private String risultato;
    private Auto griglia [];

    public Gara(String nome) {
        setNome(nome);
        setRisultato("Corsa non terminata");
        creaGrigliaDiPartenza();
    }

    public void creaGrigliaDiPartenza() {
        Pilota uno = new Pilota("Pippo");
        Pilota due = new Pilota("Pluto");
        Pilota tre = new Pilota("Topolino");
        Pilota quattro = new Pilota("Paperino");
        Auto autoNumeroUno = new Auto("Ferrari", uno);
        Auto autoNumeroDue = new Auto("Renault", due);
        Auto autoNumeroTre = new Auto("BMW", tre);
        Auto autoNumeroQuattro = new Auto("Mercedes", quattro);
        griglia = new Auto[4];
        griglia[0] = autoNumeroUno;
        griglia[1] = autoNumeroDue;
        griglia[2] = autoNumeroTre;
        griglia[3] = autoNumeroQuattro;
    }

    public void corriGara() {
        int numeroVincente = (int) (Math.random() * 4);
        Auto vincitore = griglia[numeroVincente];
        String risultato = vincitore.dammiDettagli();
        setRisultato(risultato);
    }

    public void setRisultato(String vincitore) {
        this.risultato = "Il vincitore di " + this.getNome() + ": "
            + vincitore;
    }

    public String getRisultato() {
        return risultato;
    }

    public void setNome(String nome) {
```

```
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}
```

La classe `TestGara` contiene il metodo `main` e quindi determina il flusso di esecuzione dell'applicazione. È molto leggibile: si istanzia un oggetto `gara` e lo si chiama “GP di Imola”, si fa correre la corsa, si richiede il risultato e lo si stampa a video.

La classe `Gara` invece contiene pochi e semplici metodi e tre variabili d'istanza: `nome` (il nome della gara), `risultato` (una stringa che contiene il nome del vincitore della gara se è stata corsa) e `griglia` (un array di oggetti `Auto` che partecipano alla gara).

Il costruttore prende in input una stringa con il nome della gara che viene opportunamente impostato. Inoltre il valore della stringa `risultato` è impostata a “Corsa non terminata”. Infine è chiamato il metodo `creaGrigliaDiPartenza`.

Il metodo `creaGrigliaDiPartenza` istanzia quattro oggetti `Pilota` assegnando loro dei nomi. Poi istanzia quattro oggetti `Auto` assegnando loro i nomi delle scuderie ed i relativi piloti. Infine istanzia ed inizializza l'array `griglia` con le auto appena create. Una gara, dopo essere stata istanziata, è pronta per essere corsa.

Il metodo `corriGara` contiene codice che va analizzato con più attenzione. Nella prima riga, infatti, viene chiamato il metodo `random` della classe `Math` (appartenente al package `java.lang` che viene importato automaticamente). La classe `Math` astrae il concetto di *matematica* e sarà descritta più avanti in questo libro. Essa contiene metodi che astraggono classiche funzioni matematiche, come la radice quadrata o il logaritmo. Tra questi metodi utilizziamo il metodo `random` che restituisce un numero generato in maniera casuale di tipo `double`, compreso tra 0 ed 0,9999999... (ovvero il numero `double` immediatamente più piccolo di 1). Nell'esercizio abbiamo moltiplicato per 4 questo numero, ottenendo un numero `double` casuale compreso tra 0 e 3,99999999... Questo è poi convertito ad intero, quindi vengono troncate tutte le cifre decimali. Abbiamo quindi ottenuto che la variabile `numeroVincente` immagazzini al runtime un numero generato casualmente, compreso tra 0 e 3, ovvero i possibili indici dell'array `griglia`.

Il metodo `corriGara` genera quindi un numero casuale tra 0 e 3. Lo utilizza per individuare l'oggetto `Auto` dell'array `griglia` che vince la gara, per poi impostare il risultato tramite il metodo `dammiDettagli` dell'oggetto `Auto` (che scriverà il lettore).

Tutti gli altri metodi della classe sono di tipo `accessor` e `mutator`.

Esercizio 5.c) *Modificatori d'accesso, static, e package, Vero o Falso:*

1. Una classe dichiarata `private` non può essere utilizzata fuori dal package in cui è dichiarata.
2. La seguente dichiarazione di classe è scorretta:
`public static class Classe {...}`
3. La seguente dichiarazione di classe è scorretta:
`protected class Classe {...}`
4. La seguente dichiarazione di metodo è scorretta:
`public void static metodo () {...}`
5. Un metodo statico può utilizzare solo variabili statiche, locali e parametri e non bisogna per forza istanziare un oggetto dalla classe in cui è definito per utilizzarlo.
6. Se un metodo è dichiarato `static`, non può essere chiamato al di fuori del proprio package.
7. Una classe `static` non è accessibile fuori dal package in cui è dichiarata.

8. Un metodo `protected` viene ereditato in ogni sottoclasse qualsiasi sia il suo package.
9. Una variabile `static` viene condivisa da tutte le istanze della classe a cui appartiene.
10. Se non anteponiamo modificatori ad un metodo, il metodo è accessibile solo all'interno dello stesso package.

Esercizio 5.d) *Object Orientation in Java (pratica), Vero o Falso:*

1. Un metodo statico deve essere per forza pubblico.
2. L'implementazione dell'incapsulamento implica l'utilizzo delle parole chiave `set` e `get`.
3. Per utilizzare le variabili incapsulate di una superclasse in una sottoclasse bisogna dichiararle almeno `protected`.
4. I metodi dichiarati privati non vengono ereditati nelle sottoclassi.
5. Un inizializzatore d'istanza viene invocato prima di ogni costruttore.
6. Una variabile privata risulta disponibile (tecnicamente come se fosse pubblica) tramite l'operatore `dot`, a tutte le istanze della classe in cui è dichiarata.
7. La parola chiave `this` permette di referenziare i membri di un oggetto che sarà creato solo al runtime all'interno dell'oggetto stesso.
8. La parola chiave `this` è sempre facoltativa.
9. La parola chiave `this` permette di chiamare un costruttore, da un metodo della stessa classe con la sintassi `this()`. È necessario però che questa sia la prima istruzione del metodo.
10. Il singleton pattern permette di creare una classe che può essere istanziata una sola volta.

Esercizio 5.e)

Astrarre il concetto di `Moneta` con una classe (completa di commenti). Assumiamo che tutte le monete avranno come valuta l'EURO, ed avranno la variabile d'istanza incapsulata `valore`. Ha senso creare una moneta senza specificarne il valore? Creare un vincolo affinché le monete possano essere istanziate obbligatoriamente con un valore.

Conviene stampare una frase in ogni metodo importante per potere verificare l'avvenuta esecuzione del nostro codice. Per esempio quando si istanzia una moneta. Questo vale anche per i prossimi esercizi.

Esercizio 5.f)

Considerando la classe `Moneta` creata nell'esercizio precedente, è corretto creare la variabile `valore` incapsulata con i metodi `setValore` e `getValore`? Modificare la classe in modo tale da astrarre al meglio la classe.

Esercizio 5.g)

Creare una classe `TestMonete` con un metodo `main` che istanzia una moneta da 20 centesimi e una moneta da 1 centesimo ed eseguire l'applicazione. C'è qualcosa che non va su quanto viene stampato? Se è così, cambiare il codice in modo tale che le stampe siano senza errori grammaticali.

Esercizio 5.h)

Nella classe `TestMonete` istanziare anche una moneta da 1 euro. Probabilmente ci sarà un altro baco nella stampa, risolverlo. Aggiungere inoltre un metodo `getDescrizione` nella classe `Moneta`, che ritorna una stringa descrittiva della moneta corrente.

**Esercizio 5.i)**

Creare una classe (completa di commenti) `PortaMonete` che astrae il concetto di portamonete. Questo deve poter contenere al massimo 10 monete (la classe `Moneta` dovrebbe essere già stata creata nell'esercizio precedente). Per ora creare solo un costruttore che consenta di impostare il contenuto delle monete.

Anche in questo caso conviene stampare la descrizione delle azioni che si stanno invocando.

Esercizio 5.j)

Nella classe `TestMonete` istanziare anche un oggetto `PortaMonete` con 8 monete e uno con 11 monete, verificare che tutto funzioni correttamente. Aggiornare eventualmente anche il commento della classe.

Esercizio 5.k)

Creare un metodo `aggiungi` all'interno della classe che permetta di aggiungere una moneta al portamonete. Prevedere gli appositi controlli di consistenza.

Esercizio 5.l)

Creare un metodo `stato` che stampi l'attuale contenuto del portamonete.

Esercizio 5.m)

Creare un metodo `preleva` all'interno della classe che consenta di prelevare (e quindi rimuovere) una moneta dal portamonete. Prevedere gli appositi controlli di consistenza.

Esercizio 5.n)

Modificare la classe `TestMonete` in modo tale da testare nella maniera più completa le classi create.



Esercizio 5.o)

Studiare attentamente l'approfondimento 5.7. Creare un semplice activity diagram, che possa rappresentare il flusso delle attività del processo presentato (esempi di attività saranno per esempio “analisi dei casi d'uso”, e “astrazioni chiave”).



Esercizio 5.p)

Problem statement: creare un programma di autenticazione, che richiede all'utente username e password, e gli garantisce l'accesso (basterà stampare un messaggio di benvenuto con il nome dell'utente), se le credenziali inserite sono corrette. Il sistema deve supportare l'autenticazione con almeno tre coppie username e password, con cui ci si può loggare.

Eseguire l'analisi dei casi d'uso ed individuare i vari casi d'uso, seguendo il processo descritto nell'approfondimento 5.7 nei paragrafi 5.7.2 e 5.7.2.1.

Esistono vari strumenti software che supportano i diagrammi UML (https://en.wikipedia.org/wiki/List_of_Unified_Modeling_Language_tools). **Ognuno di essi ha la sua filosofia e la sua complessità. Dovreste sceglierne uno prima o poi (magari dopo aver fatto qualche confronto), ma per ora va benissimo anche usare un foglio di carta, una matita, e una gomma da cancellare. La scelta di un tool, capire come funziona etc. richiede del tempo.**



Esercizio 5.q)

Individuare gli scenari dei casi d'uso del precedente esercizio, seguendo i consigli del paragrafo 5.7.2.2 dell'approfondimento 5.7.



Esercizio 5.r)

Continuiamo l'esercizio precedente seguendo il processo descritto nell'approfondimento 5.7. Essendo una semplice applicazione desktop, sembra superfluo creare un diagramma di deployment. Ma se pensiamo a come poter un giorno riutilizzare una parte di quest'applicazione (d'altronde stiamo parlando di un'applicazione che permette di autenticarsi in un sistema, e che potremmo integrare anche nel caso di studio introdotto nell'approfondimento 5.7 per loggarsi nell'applicazione *Logos*), allora potrebbe essere molto utile realizzare una *visione dall'alto* che ne specifichi le dipendenze tra i vari componenti del software. Quindi proviamo a creare un component diagram (o

volendo un deployment diagram), creando componenti che abbiano nomi indicativi, e che poi provvederemo a realizzare con le classi. Deve essere giusto un banale diagramma (high level deployment diagram), che esalti la possibilità che una certa parte del software sia riutilizzabile.

È possibile prendere spunto dal diagramma definito nel paragrafo 5.7.3 nell'approfondimento 5.7.



Esercizio 5.s)

Continuiamo l'esercizio precedente seguendo il processo descritto nel paragrafo 5.7.4 dell'approfondimento 5.7. Passiamo quindi ad individuare le classi candidate, e di conseguenza le key abstraction.



Esercizio 5.t)

Continuiamo l'esercizio precedente verificando la fattibilità degli scenari individuati, creando sequence diagram basati sulle interazioni delle classi individuate, come descritto nel paragrafo 5.7.5 dell'approfondimento 5.7.



Esercizio 5.u)

In base ai passi fatti negli esercizi 5.p, 5.q, 5.r, 5.s, e 5.t, implementare una soluzione funzionante, senza utilizzare l'incapsulamento. Tale soluzione potrebbe impattare sulla nostra analisi, nel senso che potrebbe convalidare alcune scelte fatte o smentendole. Il nostro obiettivo rimane quello di creare del codice funzionante, quindi se qualche passo dell'analisi, non sembra fattibile con la programmazione, dobbiamo dare precedenza alla implementazione.

Esercizio 5.v)

Incapsulare la classe `Utente` dell'esercizio 5.u, e modificare di conseguenza la classe `Autenticazione` affinché tutto continui a funzionare.



Esercizio 5.w)

Disegnare un class diagram contenente le due classi `Utente` e `Autenticazione` modificate nell'esercizio precedente che mostrano le loro variabili e i loro metodi. È possibile consultare l'approfondimento 4.7 contenente uno schema di riferimento per la sintassi UML. In particolare per i membri statici (che vanno sottolineati) e la notazione di *aggregazione* che sussiste tra la classe contenitore `Autenticazione` e la classe contenuta `Utente`. Usare anche la notazione di molteplicità. Inoltre includere le due classi nella notazione di package.



Esercizio 5.x)

Abbiamo già notato alla fine dell'esercizio 5.u, che la soluzione di codice implementata non ci soddisfaceva. Infatti durante la nostra analisi avevamo individuato un flusso di esecuzione basato su classi e metodi differenti, come mostrato nei sequence diagram delle soluzioni dell'esercizio 5.t, che riportiamo per comodità.

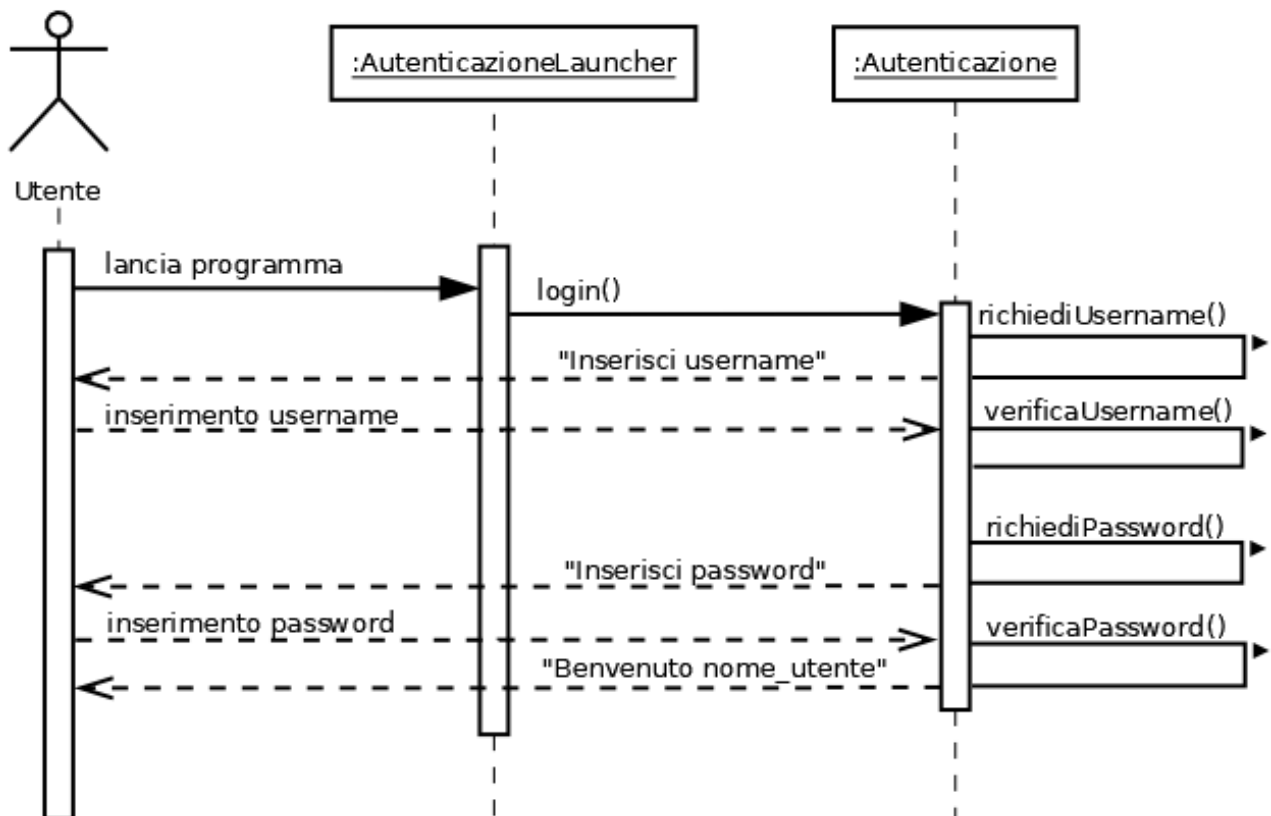


Figura 5.t.1 - Sequence diagram che rappresenta lo scenario principale.

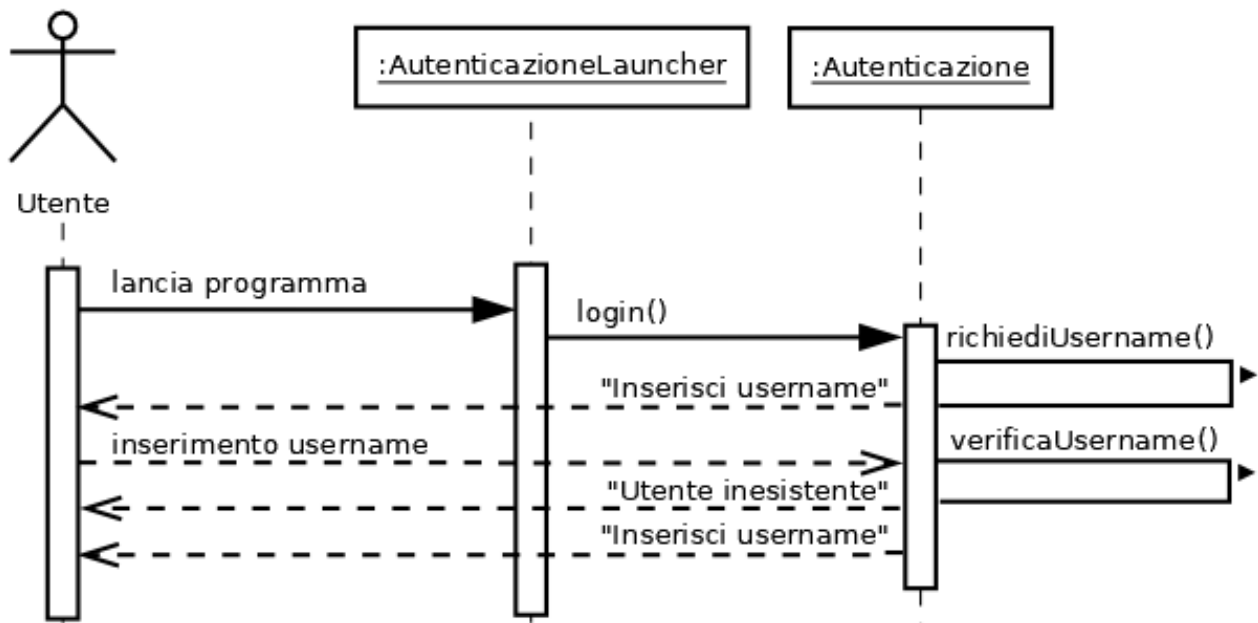


Figura 5.t.2 - Sequence diagram che rappresenta il secondo scenario.

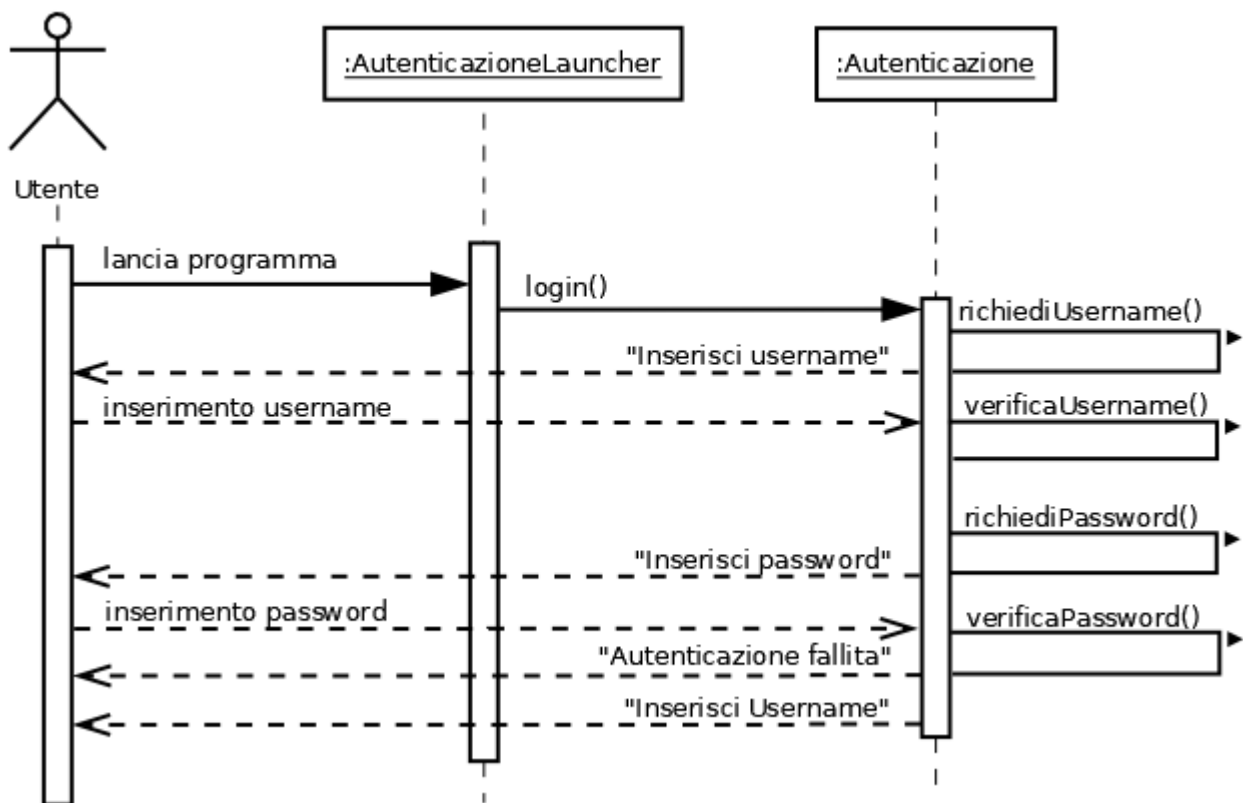


Figura 5.t.3 - Sequence diagram che rappresenta il terzo scenario.

Creare un class diagram che rappresenti correttamente le classi necessarie a far funzionare gli scenari descritti con i sequence diagram, supportandosi con la syntax reference dell'approfondimento 4.7. Inoltre trasformare la classe `Autenticazione` in un singleton, come descritto nel paragrafo 5.4.6.

È possibile anche definire nuovi metodi o variabili se opportuno. Se non si riescono a definire i particolari della classe (per esempio tipi di ritorno, nomi o tipi di parametri e così via, semplicemente non definirli).

Ovviamente la classe `Utente` deve rimanere incapsulata.



Esercizio 5.y)

Una volta implementa la soluzione dell'esercizio precedente, dovremmo avere una classe `Autenticazione` con diversi metodi e responsabilità. Ha un unico metodo pubblico che gestisce il flusso (`login`) e diversi metodi privati, alcuni verificano la correttezza dei dati inseriti dall'utente (utilizzando la variabile d'istanza `utenti`) ed altri per stampare messaggi all'utente.

Un oggetto che si chiama `Autenticazione` è giusto che abbia la responsabilità di:

1. Gestire il flusso di esecuzione dell'applicazione.
2. Verificare la correttezza dei dati inseriti dall'utente.
3. Contenere la lista degli utenti.
4. Stampare messaggi.

Le responsabilità identificano l'astrazione della classe, e questa classe è sicuramente troppo carica di responsabilità. Facciamo quindi evolvere il nostro class diagram: cerchiamo di astrarre la classe `Autenticazione`, nella maniera più corretta e trovare altre astrazioni (classi) che possano implementare le responsabilità più specifiche. Partiamo dal trovare una classe che contenga i dati su cui vogliamo lavorare. Come la chiamiamo? Che variabili e metodi deve contenere? Siccome dobbiamo pensare in maniera object oriented, cerchiamo di renderla il più possibile riutilizzabile, ma anche coerente con il contesto in cui la stiamo definendo.

Esercizio 5.z)

Continuando l'esercizio precedente, modificare il class diagram individuando una classe che abbia le responsabilità di stampare i messaggi a video.

Esercizio 5.aa)

Continuando l'esercizio precedente, la classe `Autenticazione` è ora astratta correttamente? Confermare o modificare ancora il diagramma.

Esercizio 5.bb)

In base alla conclusione dell'esercizio precedente, implementare la soluzione di codice più vicina alla soluzione progettata. Rispetto alla soluzione a cui eravamo arrivati nell'esercizio 5.u, dovremmo avere la stessa funzionalità, ma un codice più semplice con cui interagire, astratto meglio, e più riutilizzabile.

Esercizio 5.cc)

Data la seguente classe:

```
public class BluRay {
    int capacitaMassimaInGigaByte= 25
    byte[] contenuto;

    BluRay() {
    }

    void setContenuto (byte[] contenuto) {
        this.contenuto = contenuto;
    }

    byte[] getContenuto () {
        return contenuto;
    }
}
```

Aggiungere i modificatori più appropriati per ogni membro.

Esercizio 5.dd)

Quali delle seguenti dichiarazioni di `import` statici sono valide?

1. `import static java.lang.*;`
2. `import static java.lang.Math;`
3. `import static java.lang.Math.*;`
4. `import static java.lang.Math.PI;`
5. `import static java.lang.Math.random();`
6. `import static java.lang.Math.random;`

Esercizio 5.ee)

Qual è l'output del seguente programma?

```
public class InitTest {
    {
        System.out.println("Initializer");
    }
    static {
        System.out.println("Static Initializer");
    }
    public InitTest () {
        System.out.println("Constructor");
    }
    public void method() {
        System.out.println("Method");
    }
    public static void staticMethod() {
        System.out.println("Static Method");
        new InitTest().method();
    }
    public static void main(String args[]) {
        InitTest.staticMethod();
    }
}
```


Esercizio 5.ff)

Creare una classe `Libro` incapsulata, in modo tale che astragga il concetto di un libro che può essere venduto in una libreria. Tra i campi definiti dalla classe `Libro`, deve esserci il campo `genere` (inteso come genere letterario). Fare in modo che un libro possa essere associato solo ad un genere letterario compreso tra un insieme di generi letterari predefiniti, per esempio l'insieme costituito dai generi romanzo, thriller, saggio, manuale. Creare una classe che testi che gli oggetti `Libro` funzionino nella maniera corretta.

**Esercizio 5.gg)**

Tenendo conto dell'esercizio precedente, creare inoltre la classe `Scaffale`. Ogni scaffale deve essere dedicato ad un certo genere letterario compreso nell'elenco dei generi prescelti nel precedente esercizio, e quindi deve contenere solo libri dello stesso genere. Nella classe `Scaffale` deve esserci un metodo chiamato `aggiungiLibro(Libro libro)`, che permette di aggiungere allo scaffale un libro con il genere corretto. Creare anche la classe `Libreria`. Una libreria deve contenere un solo scaffale per ogni genere, e quindi bisogna impedire che possano essere aggiunti due scaffali con lo stesso genere. Inoltre la classe `Libreria` deve implementare il pattern Singleton. Infine creare una classe `TestLibreria`, che crea un oggetto di tipo `Libreria` aggiungendogli degli oggetti di tipo `Scaffale` ai quali siano stati aggiunti oggetti di tipo `Libro`. Verificare che alla libreria non possano essere aggiunti due scaffali con lo stesso genere, e che ad uno scaffale non possano essere aggiunti libri con generi diversi da quello definito dallo scaffale.

Esercizio 5.hh)

Quali delle seguenti affermazioni sono vere riguardo i diagrammi dei casi d'uso UML?

1. Nel diagramma dei casi d'uso UML non è possibile utilizzare il simbolo di classe.
2. Un caso d'uso è rappresentato con un rettangolo in UML.
3. Il diagramma dei casi d'uso è un diagramma statico, infatti non contiene informazioni riguardo la linea temporale.
4. Gli attori del diagramma dei casi d'uso rappresentano gli utenti del sistema, e quindi persone fisiche.
5. Un caso d'uso può essere collegato a più attori.

Esercizio 5.ii)

Quali delle seguenti affermazioni sono vere?

1. Uno scenario di un caso d'uso non contiene condizioni.
2. È possibile descrivere gli scenari con un activity diagram oltre che con del testo semplice.
3. Bisognerebbe descrivere per ogni caso d'uso tutti gli scenari.

4. La descrizione degli scenari ci permette di scoprire eventuali flussi, prima di ancora di scrivere il codice.
5. Un deployment diagram mostra l'interazione tra l'utente e il sistema.
6. Le classi candidate sono le classi fondamentali che sicuramente saranno implementate nel nostro programma.

Esercizio 5.jj)

Quali delle seguenti affermazioni sono vere riguardo il sequence diagram?

1. Un activity diagram ci permette di simulare uno scenario, ma usando oggetti.
2. L'activity line (linea di attività) di un oggetto è rappresentata da una linea tratteggiata.
3. I messaggi definiscono i metodi.
4. Nell'activity diagram, è possibile rappresentare un attore.
5. Il nome degli oggetti in un sequence diagram sono rappresentati con la sintassi `nomeOggetto:NomeClasse`, ma sia il nome dell'oggetto, che il nome della classe possono essere omessi.

Esercizi del capitolo 6

Ereditarietà ed interfacce

Per questo capitolo eviteremo di far scrivere troppo codice al lettore. È molto importante invece concentrarsi piuttosto sulle definizioni. Se non si conoscono bene tutte i concetti della teoria, si finirà con lo scrivere codice incoerente dal punto di vista della filosofia ad oggetti.

Come sempre si consiglia leggere la soluzione dopo ogni esercizio.

Esercizio 6.a) *Object Orientation in Java (teoria), Vero o Falso:*

1. L'implementazione dell'ereditarietà implica scrivere sempre qualche riga in meno.
2. La seguente dichiarazione di classe è scorretta:

```
public final class Classe extends AltraClasse {...}
```
3. L'ereditarietà è utile solo se si utilizza la specializzazione. Infatti, specializzando ereditiamo nella sottoclasse (o sottoclassi) membri della superclasse che non bisogna riscrivere. Con la generalizzazione invece creiamo una classe in più, e quindi scriviamo più codice.
4. La parola chiave `super` permette di chiamare metodi e costruttori di superclassi. La parola chiave `this` consente di chiamare metodi e costruttori della stessa classe in cui ci si trova.
5. L'ereditarietà multipla non esiste in Java perché non esiste nella realtà.
6. Un'interfaccia funzionale è un'interfaccia che dichiara un unico metodo di default.
7. Una sottoclasse è più “grande” di una superclasse (nel senso che solitamente aggiunge caratteristiche e funzionalità nuove rispetto alla superclasse).
8. Supponiamo di sviluppare un'applicazione per gestire un torneo di calcio. Esiste ereditarietà derivata da specializzazione tra le classi `Squadra` e `Giocatore`.
9. Supponiamo di sviluppare un'applicazione per gestire un torneo di calcio. Esiste ereditarietà derivata da generalizzazione tra le classi `Squadra` e `Giocatore`.
10. In generale, se avessimo due classi `Padre` e `Figlio`, non esisterebbe ereditarietà tra queste due classi.

Esercizio 6.b)

Data la seguente classe:

```
public class Persona {  
    private String nome;  
  
    public void setName(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}
```

Commentare la seguente classe `Impiegato` evidenziando dove sono utilizzati i paradigmi object oriented: incapsulamento, ereditarietà e riuso.

```
public class Impiegato extends Persona {  
  
    private int matricola;  
  
    public void setDati(String nome, int matricola) {  
        setNome(nome);  
        setMatricola(matricola);  
    }  
  
    public void setMatricola(int matricola) {  
        this.matricola = matricola;  
    }  
  
    public int getMatricola() {  
        return matricola;  
    }  
  
    public String dammiDettagli() {  
        return getNome() + ", matricola: " + getMatricola();  
    }  
}
```

Esercizio 6.c) *Classi astratte ed interfacce, Vero o Falso:*

1. La seguente dichiarazione di classe è scorretta:
`public abstract final class Classe {...}`
2. La seguente dichiarazione di classe è scorretta:
`public abstract class Classe;`
3. La seguente dichiarazione di interfaccia è scorretta:
`public final interface Classe {...}`
4. Una classe astratta contiene per forza metodi astratti.
5. Un'interfaccia può essere estesa da un'altra interfaccia.
6. Una classe può estendere una sola classe ma implementare più interfacce.
7. Il pregio delle classi astratte e delle interfacce è che obbligano le sottoclassi ad implementare i metodi astratti ereditati. Quindi rappresentano un ottimo strumento per la progettazione object oriented.
8. Un'interfaccia può dichiarare più costruttori.
9. Un'interfaccia non può dichiarare variabili ma costanti statiche e pubbliche.
10. Una classe astratta può implementare un'interfaccia.

Esercizio 6.d)

Descrivere tutte le relazioni di ereditarietà tra le seguenti classi:

- Professore
- Studente
- Persona
- Cattedra
- Corso
- Aula

- Lezione

Esercizio 6.e)

Se volessimo creare la gerarchia definita nell'esercizio precedente, tra `Studente`, `Persona` e `Professore` quale potrebbe essere una classe astratta?

Esercizio 6.f)

Creare l'interfaccia (con commenti) `Musicale` che dichiara un metodo `suona`. Come dichiarereste questo metodo: statico, di default o astratto?

Esercizio 6.g)

Creare due sottointerfacce (con commenti) di `Musicale`: `StrumentoMusicale` e `Suoneria`. Come dichiarereste il metodo `suona` nelle due sottointerfacce: statico, di default o astratto?

Esercizio 6.h)

Supponiamo di creare una classe `Smartphone` che implementa entrambe le interfacce definite nell'esercizio precedente. Cosa c'è di sbagliato?

Esercizio 6.i) Interfacce dopo Java 8, Vero o Falso:

1. I metodi statici non si ereditano.
2. La seguente dichiarazione di interfaccia è scorretta:

```
public static interface Interface;
```
3. La seguente dichiarazione di interfaccia è scorretta:

```
public interface class {...}
```
4. I metodi astratti di un'interfaccia non vengono ereditati da un'altra interfaccia.
5. I metodi astratti di un'interfaccia non possono essere implementati da un'altra interfaccia.
6. I metodi statici di un'interfaccia non possono essere implementati da un'altra interfaccia.
7. Un'interfaccia `A`, definisce un metodo di default `m()`. L'interfaccia `B` estende l'interfaccia `A` ridefinendo il metodo `m()` con un'implementazione di default. Una classe astratta `C` implementa l'interfaccia `A` ma ridefinendo il metodo `m()`. Una classe concreta (non astratta) `D` estende la classe `C` e implementa l'interfaccia `B`, senza ridefinire il metodo `m()`. La classe `D` eredita il metodo `m()` definito nella classe `C`.
8. Un'interfaccia `E`, definisce un metodo astratto `m()`. L'interfaccia `F` estende l'interfaccia `E` ridefinendo il metodo `m()` con un'implementazione di default. Una classe astratta `G` implementa l'interfaccia `E` ma non ridefinisce il metodo `m()`. Una classe concreta (non astratta) `H` estende la classe astratta `G` e implementa l'interfaccia `F`, senza ridefinire il metodo `m()`. La classe `H` non può essere compilata correttamente.

9. Un'interfaccia può estendere più interfacce.
10. Un'interfaccia può estendere una classe astratta e un'interfaccia.

Esercizio 6.j)

Riprendendo la soluzione dell'esercizio 5.ff, supponiamo di voler far sì che la nostra libreria venda anche album musicali. Astrarre quindi la classe `Album`, tenendo conto che anche per gli album musicali esiste un numero identificativo chiamato ISMN (anche se esistono altri modi di identificare un album come in lo standard europeo EAN), così come per i libri esiste il numero identificativo ISBN. Dopo verificare se sussiste una relazione di ereditarietà tra la classe `Libro` e la classe `Album`. Se esiste, implementare una soluzione.

Per ora, non implementare i controlli che il metodo `setGenere` della classe `Libro` esegua affinché il genere specificato come parametro appartenesse ad un insieme predefinito di generi letterari (vedi esercizio 5.ff). Questo vale sia per la classe `Album`, che per la classe `Libro`. Risolveremo la questione nel prossimo esercizio.



Esercizio 6.k)

Ripartendo dalle soluzioni dell'esercizio precedente e dell'esercizio 5.ff, rinominare la classe `GenereUtils` in `GenereLetterarioUtils` e creare la classe equivalente `GenereMusicaleUtils`. Verificare se esiste una relazione di ereditarietà tra le due classi, e se esiste, implementarla. Inoltre, anche la classe `Album` deve avere un genere appartenente ad un insieme predefinito di generi musicali (vedi metodo `setGenere` della classe `Libro`). Infine creare una classe `TestArticoli`, equivalente alla classe `TestLibro` dell'esercizio 5.ff, che testa il corretto funzionamento degli oggetti istanziati dalle classi `Libro` e `Album`.

Esercizio 6.l)

Tenendo presente tutti gli inserimenti di codice che il compilatore esegue implicitamente, riscrivere la seguente classe, aggiungendo tutte le istruzioni che il compilatore aggiungerebbe:

```
public class CompilatorePensaciTu {  
  
    private int var;  
  
    public void setVar(int v) {  
        var = v;  
    }  
  
    public int getVar() {  
        return var;  
    }  
}
```

Esercizio 6.m)

Considerando le seguenti classi:

```
public class Persona {  
  
    private String nome;  
  
    public void setName(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return this.nome;  
    }  
}  
  
public class Impiegato extends Persona {  
  
    private int matricola;  
  
    public void setMatricola(int matricola) {  
        this.matricola = matricola;  
    }  
  
    public int getMatricola () {  
        return this.matricola;  
    }  
  
    public String getDati() {  
        return getNome() + "\nnumero" + getMatricola();  
    }  
}
```

Quale sarà l'output del processo di compilazione (scegliere una sola opzione)?

1. Nessun output (compilazione corretta).
2. Errore nel metodo `getDati` di `Impiegato`.
3. Errore nel metodo `getNome` di `Persona`.
4. Errore nel metodo `getMatricola` di `Impiegato`.

Esercizio 6.n)

Se aggiungiamo alla classe `Impiegato` descritta nell'esercizio 6.m il seguente metodo:

```
public void setDati(String nome, int matricola) {  
    setName(nome);  
    setMatricola(matricola);  
}
```

Quale sarà l'output del processo di compilazione (scegliere una sola opzione)?

1. Nessun output (compilazione corretta).
2. Errore nel metodo `getDati` di `Impiegato`.
3. Errore nel metodo `setName` di `Persona`.
4. Errore nel metodo `setMatricola` di `Impiegato`.

Esercizio 6.o)

Quali di queste affermazioni sono vere?

1. L'ereditarietà permette di mettere in relazione di aggregazione più classi.
2. L'ereditarietà permette di mettere in relazione di aggregazione più interfacce.
3. L'ereditarietà permette di mettere in relazione di estensione più classi ed interfacce.
4. L'ereditarietà permette di mettere in relazione di aggregazione più classi ed interfacce.

Esercizio 6.p)

Considerato il seguente codice:

```
class Animale {}  
interface Felino {}  
class Leone {}
```

Se volessimo mettere in relazione di ereditarietà i tipi precedenti, quali dei seguenti snippet sono validi dal punto di vista del compilatore?

1. `class Animale extends Felino {}`
2. `interface Felino extends Animale {}`
3. `class Leone extends Felino {}`
4. `class Leone extends Animale implements Felino {}`
5. `class Animale extends Leone implements Felino {}`

Esercizio 6.q)

Dato che un'interfaccia può dichiarare oltre a metodi astratti, anche metodi statici pubblici e privati, e implementati (di default) pubblici e privati, e visto che con le interfacce possiamo implementare l'ereditarietà multipla, perché mai dovremmo preferire una classe astratta ad un'interfaccia?

Esercizio 6.r)

Dati i seguenti tipi:

```
interface Volante {}  
class Aereo implements Volante {}
```

quali tra i seguenti snippet sono corretti?

1. `Aereo a = new Aereo();`
2. `Volante v = new Volante();`
3. `aereo1.equals(aereo2);` (dove `aereo1` e `aereo2` sono oggetti di tipo `Aereo`)
4. `Volante.aereo = new Aereo();`

Esercizio 6.s)

Quali modificatori sono implicitamente aggiunti a tutti i metodi di un'interfaccia (è possibile scegliere più di una risposta)?

1. `public`
2. `protected`
3. `private`
4. `static`
5. `default`
6. `abstract`
7. `final`

Esercizio 6.t)

Data la seguente gerarchia:

```
interface A {  
    void metodo();  
}  
  
interface B extends A {}  
  
abstract class C implements B {}  
  
public final class D extends C {  
    public void metodo() {}  
}
```

Quali delle seguenti affermazioni sono false (è possibile scegliere più di un'affermazione):

1. La classe `C` non può essere dichiarata astratta perché implementa un'interfaccia, quindi questo codice non compila.
2. L'interfaccia `B` non può estendere un'altra interfaccia.
3. La classe `C` implementando `B`, eredita anche il metodo `metodo()` astratto di `A`.
4. La classe `D` non compila perché non può essere dichiarata `final`.
5. La classe `D` non compila perché è dichiarata `public`.
6. La classe `D` non compila perché il suo metodo è dichiarato `public`.

Esercizio 6.u)

Quali delle seguenti affermazioni sono vere:

1. I metodi statici dichiarati in un'interfaccia non vengono ereditati nelle sotto-interfacce.
2. Non è possibile dichiarare una classe `abstract` e `final` perché i due modificatori non sono compatibili tra loro.
3. Una classe astratta deve dichiarare per forza metodi astratti.
4. Le interfacce possono anche dichiarare costruttori.
5. Le classi astratte possono dichiarare metodi statici.

Esercizio 6.v)

Supponiamo di voler definire un tipo `Atleta`. Supponiamo che ogni atleta definisca i metodi `corri` e `allenati`. Supponiamo anche di voler definire delle sottoclassi più specifiche come

Calciatore, Corridore e Tennista. Come definireste `Atleta`, come interfaccia o classe astratta? Aggiungereste altri tipi?

Esercizio 6.w)

Quali delle seguenti affermazioni riguardanti il modificatore `protected` sono corrette:

1. Una classe dichiarata `protected` può essere istanziata solo all'interno dello stesso package in cui è definita, e in tutte le sue sottoclassi anche se definite in package differenti.
2. Se una classe ha un costruttore dichiarato `protected`, questa potrà essere istanziata solo dalle classi che appartengono allo stesso package.
3. Un metodo dichiarato `protected` è ereditato da una sottoclasse indipendentemente dal package in cui quest'ultima è definita.
4. Una variabile dichiarata `protected` può essere utilizzata da tutte le classi che appartengono allo stesso package della classe che definisce tale variabile.
5. Una variabile dichiarata `protected` è ereditata da una sottoclasse indipendentemente dal package in cui quest'ultima è definita.
6. Un costruttore dichiarato `protected` è ereditato da una sottoclasse indipendentemente dal package in cui quest'ultima è definita.
7. Una sottoclasse definita in un package diverso da quello a cui appartiene la sua superclasse, può istanziare quest'ultima per poi utilizzare i suoi membri `protected`.

Esercizio 6.x)

Partendo dalla soluzione dell'esercizio 4.z:

- Incapsulare le classi `Contatto` e `Rubrica`.
- Implementare per la classe `Rubrica` il pattern Singleton.
- Modificare la classe `RicercaContatti`, per fare in modo che funzioni come prima.
- Creare una classe `Speciale` che estende `Contatto`. Supponiamo che un contatto speciale, debba avere anche una suoneria personalizzata (creando la relativa variabile d'istanza di tipo `stringa`).
- Nella classe `Rubrica` creare anche un array di oggetti `Speciale`, simile all'array `contatti` che già esiste. Creare anche un metodo `cercaContattiSpecialiPerNome`, equivalente al metodo `cercaContattiPerNome` già presente nella classe.
- Creare una classe `RicercaContattiSpeciali` equivalente alla classe `RicercaContatti` dell'esercizio 4.z, per testare il corretto funzionamento del codice scritto.



Esercizio 6.y)

Si sconsiglia di eseguire il seguente esercizio utilizzando Blocco Note e riga di comando. Con un IDE (o almeno con EJE) si risparmieranno molti minuti per terminare l'esercizio.

Partendo dall'esercizio 6.x, eseguire le seguenti tecniche di refactoring per usare i paradigmi di riuso, ereditarietà ed astrazione:

- Fare in modo che le classi create non contengano codice duplicato (riutilizzare tutto il codice che si riesce a riutilizzare).
- Creare un'interfaccia `Dato` da far implementare a tutte le classi che rappresentano dati nell'applicazione, e l'interfaccia `Identificabile` che dichiara un metodo astratto `getID` che ritorna un `int`.
- Astrarre il concetto di entità implementando la classe astratta `Entita` (senza l'accento perché non è possibile utilizzare lettere accentate in Java), che implementa le interfacce `Dato` e `Identificabile`. Un oggetto `Entita` deve essere caratterizzato da un numero identificativo incrementale generato automaticamente. Tale caratteristica ci servirà per far evolvere le nostre astrazioni nei futuri esercizi.
- Mettere in relazione di ereditarietà questa nuova classe (ed eventualmente le altre interfacce) con le altre classi del progetto dove applicabile.
- Creare i package `rubrica.dati`, `rubrica.interfaccia` e `rubrica.business`, dove intendiamo con `rubrica.dati`, un package che deve contenere classi che rappresentano i dati dell'applicazione. Il package `rubrica.business` invece, deve contenere le classi che contengono la logica di business dell'applicazione. Infine `rubrica.interfaccia` deve contenere le classi che rappresentano l'interfaccia con cui può interagire l'utente.
- Spostare di conseguenza le classi create sotto i package che sembrano più adeguati. Se lo ritenete necessario, è possibile creare altre classi e package e modificare quelle esistenti (è possibile anche creare superclassi, sottoclassi e classi di utilità).
- Testare che le classi `RicercaContattiSpeciali` e `RicercaContatti` continuino a funzionare.

Esercizio 6.z)

Riprendiamo il caso di studio definito nell'appendice 5.7. Avevamo fatto una serie di passi, che rappresentavano un possibile processo da seguire per poter creare infine del codice di qualità. In particolare avevamo definito i seguenti passi:

- Analisi degli use case.
- Definizione degli scenari per ogni use case.
- Definire un high level deployment diagram di architettura.
- Individuare le key abstraction.
- Verificare la validità delle key abstraction utilizzando i diagrammi di iterazione per convalidare i flussi degli scenari, con gli oggetti istanziati dalla key abstraction.
- Abbiamo visto negli esercizi finali relativi al capitolo 5, che altri passi da eseguire sono:
- Definire le key abstraction su un class diagram.
- Rivalutare il class diagram aggiungendo i dettagli essenziali e soprattutto ragionando sulle responsabilità.

Completare l'intero programma Logos è troppo impegnativo (ci vorrebbero settimane se non mesi di lavoro), ma possiamo concentrarci su particolari use case, e portare avanti il nostro processo solo su questi use case.

Il lettore potrebbe poi anche iterare i ragionamenti fatti su ogni use case per completare pezzo dopo pezzo il programma.

Anche se nell'analisi iniziale di Logos non è stato individuato il processo di autenticazione, in realtà ci deve essere. Infatti, dall'analisi degli use case erano stati definiti due attori: l'amministratore (che aveva compiti di configurazione del sistema) e il commesso (che aveva compiti operativi). Sembra scontato che per potersi far riconoscere dal sistema, un meccanismo di login sia indispensabile. Quindi possiamo dire di aver scoperto un nuovo caso d'uso, che andiamo a definire come "autenticazione". Facciamo evolvere allora il diagramma dei casi d'uso della figura 5.7.2 dell'approfondimento 5.7 nel diagramma di figura 6.z.1, dove introduciamo il nuovo caso d'uso trovato.

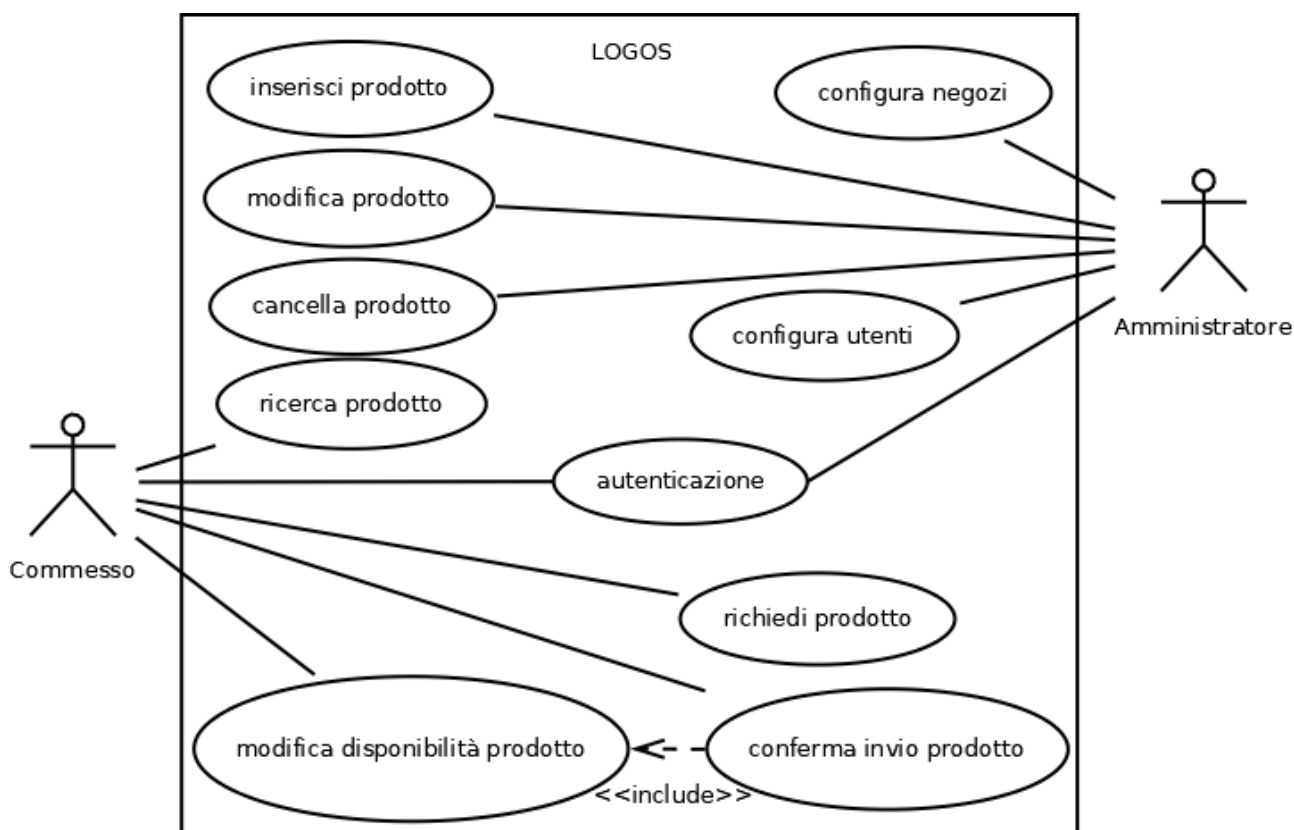


Figura 6.z.1 - Use case diagram aggiornato di Logos.

Per ora concentriamoci su questo caso d'uso calato nel contesto di Logos. Abbiamo il vantaggio di aver già lavorato su un programma che gestisce l'autenticazione con un certo flusso, vediamo se riusciamo ad evolverlo.

Quindi, tenendo presente che abbiamo già creato la classe `Utente`, vogliamo definire le classi `Commesso` e `Amministratore`. Conviene crearle? E perché?

Esercizi del capitolo 7

Polimorfismo

Per questo capitolo simuleremo la costruzione di un IDE di programmazione in maniera molto semplificata. Anche in questo caso sarà un esercizio incrementale quindi bisogna completare ogni esercizio (e leggerne la soluzione) per poter passare al successivo.

Creeremo classi ed interfacce andando per gradi. In particolare creeremo le astrazioni di `IDE`, `Editor`, `FileSorgente`, `File`, `TipoFile` e così via. L'esercizio è guidato, quindi il lettore è sollevato dalla responsabilità di decidere quali classi devono comporre la nostra applicazione. Per tale ragione si tratta di esercizi di programmazione più che di analisi e progettazione.

Con i prossimi esercizi ci sarà solo da applicare le definizioni che abbiamo imparato sino ad ora, non è richiesto quasi nessun tipo di algoritmo. Lo scopo finale è di concentrarsi sull'Object Orientation e non sugli algoritmi. Inoltre sono presentati anche altri tipi di esercizi come quelli che supportano la preparazione alla certificazione Oracle.

Esercizio 7.a) *Polimorfismo per metodi, Vero o Falso:*

1. L'overload di un metodo implica scrivere un altro metodo con lo stesso nome e diverso tipo di ritorno.
2. L'overload di un metodo implica scrivere un altro metodo con nome differente e stessa lista di parametri.
3. La segnatura (o firma) di un metodo è costituita dalla coppia identificatore - lista di parametri.
4. Per sfruttare l'override bisogna che sussista l'ereditarietà.
5. Per sfruttare l'overload bisogna che sussista l'ereditarietà.
6. Supponiamo che in una classe `B`, la quale estende la classe `A`, ereditiamo il metodo:

```
public int m(int a, String b) { ... }
```


Se nella classe `B` scriviamo il metodo:

```
public int m(int c, String b) { ... }
```


stiamo facendo overload e non override.
7. Se nella classe `B` scriviamo il metodo:

```
public int m(String a, String b) { ... }
```


stiamo facendo overload e non override.
8. Se nella classe `B` scriviamo il metodo:

```
public void m(int a, String b) { ... }
```


otterremo un errore in compilazione.
9. Se nella classe `B` scriviamo il metodo:

```
protected int m(int a, String b) { ... }
```


otterremo un errore in compilazione.
10. Se nella classe `B` scriviamo il metodo:

```
public int m(String a, int c) { ... }
```


otterremo un override.

Esercizio 7.b) Polimorfismo per dati, Vero o Falso:

1. Considerando le classi introdotte in questo capitolo, il seguente frammento di codice non produrrà errori in compilazione:
`Veicolo v [] = {new Automobile(), new Aereo(), new Aereo()};`
2. Considerando le classi introdotte in questo capitolo, il seguente frammento di codice non produrrà errori in compilazione:
`Object o [] = {new Automobile(), new Aereo(), "ciao"};`
3. Considerando le classi introdotte in questo capitolo, il seguente frammento di codice non produrrà errori in compilazione:
`Aereo a [] = {new Veicolo(), new Aereo(), new Aereo()};`
4. Considerando le classi introdotte in questo capitolo, e se il metodo della classe `Viaggiatore` fosse questo:

```
public void viaggia(Object o) {  
    o.accelera();  
}
```

potremmo passargli un oggetto di tipo `Veicolo` senza avere errori in compilazione. Per esempio:
`claudio.viaggia(new Veicolo());`
5. Considerando le classi introdotte in questo capitolo, il seguente frammento di codice non produrrà errori in compilazione:
`PuntoTridimensionale ogg = new Punto();`
6. Considerando le classi introdotte in questo capitolo, il seguente frammento di codice non produrrà errori in compilazione:
`PuntoTridimensionale ogg = (PuntoTridimensionale)new Punto();`
7. Considerando le classi introdotte in questo capitolo, il seguente frammento di codice non produrrà errori in compilazione:
`Punto ogg = new PuntoTridimensionale();`
8. Considerando le classi introdotte in questo capitolo, e se la classe `Piper` estende la classe `Aereo`, il seguente frammento di codice non produrrà errori in compilazione:
`Veicolo a = new Piper();`
9. Considerando le classi introdotte in questo capitolo, il seguente frammento di codice non produrrà errori in compilazione:
`String stringa = fiat500.toString();`
10. Considerando le classi introdotte in questo capitolo. Il seguente frammento di codice non produrrà errori in compilazione:

```
public void pagaDipendente(Dipendente dip) {  
    if (dip instanceof Dipendente) {  
        dip.stipendio = 1000;  
    }  
    else if (dip instanceof Programmatore) {  
        ...  
    }  
}
```

Esercizio 7.c)

Per iniziare, creare un'interfaccia `TipoFile` che definisca delle costanti statiche che rappresentano i tipi di file sorgente che vorremo gestire con l'IDE. Di sicuro una di queste costanti deve chiamarsi `JAVA`, tutte le altre sono a piacere. Scegliere anche il tipo delle costanti a piacere.

Esercizio 7.d)

Creare una classe `File` che astrae il concetto di file generico e che definisce un nome e un tipo.

Esercizio 7.e)

Creare una classe `FileSorgente` che astrae il concetto di file sorgente (estendendo la classe `File`) che definisce anche un contenuto di tipo stringa.

Esercizio 7.f)

Aggiungere un metodo `aggiungiTesto` che aggiunga una stringa di testo alla fine del contenuto del file sorgente.

Esercizio 7.g)

Aggiungere un overload del metodo `aggiungiTesto` che aggiunga una stringa di testo in un punto specificato del contenuto del file sorgente (consultare la documentazione della classe `String`).

Esercizio 7.h)

Creare una classe `TestFileSorgente` che testi il funzionamento corretto della classe `FileSorgente`.

Esercizio 7.i)

Creare un'interfaccia `Editor` che astrae il concetto di editor di testo. Bisogna definire metodi per aprire, chiudere, salvare e modificare un file.

Esercizio 7.j)

Creare un'interfaccia `IDE` che astrae il concetto di IDE di sviluppo. Si tenga presente che un IDE è anche un editor. Bisogna definire metodi per compilare ed eseguire un file.

Esercizio 7.k)

Creare una semplice implementazione `JavaIDE` dell'interfaccia `IDE`. Aggiungere un'implementazione per il metodo `modifica` (e a piacere è possibile reimplementare tutti i metodi che si ritiene utile).

Esercizio 7.l)

Creare una classe di test `TestIDE` che esegue delle operazioni sul file tramite `IDE`.

L'esercizio potrebbe poi continuare estendendo ulteriormente queste classi, sentitevi liberi di compiere altre iterazioni di programmazione dopo aver completato tutto. Dovrete darvi delle specifiche, capire come implementarle e implementarle: tre passi.

Esercizio 7.m)

Partendo dalle soluzioni dell'esercizio 6.k, nel quale abbiamo creato delle classi per simulare una libreria, eseguire i seguenti passi:

1. Creare i package `libreria.dat`, `libreria.business`, `libreria.util` e `libreria.interfaccia`, e posizionare le classi esistenti all'interno dei package che si ritengono più opportuni.
2. Creare per le classi `Libro` e `Album`, il metodo `toString`, che restituisca una riga descrittiva dell'articolo. In particolare, il metodo `toString` dovrà stampare queste informazioni: `tipo_articolo: (isbn) - "titolo" di autore, genere, prezzo`. Per esempio: `Libro: (2345) "I pilastri della terra" di Ken Follet, Romanzo, 18€`.
3. Modificare la classe `TestArticoli`, per testare anche il funzionamento del metodo `toString`.

Esercizio 7.n)

Partendo dalla soluzione dell'esercizio precedente, creare una classe `Carrello`, che rappresenti un carrello della spesa. Tale astrazione deve dichiarare i metodi: `aggiungi` per aggiungere un elemento al carrello, `calcolaPrezzo` che restituisce il prezzo totale degli articoli presenti nel carrello, `toString` che stampi la descrizione degli elementi degli articoli contenuti nel carrello. Il carrello potrà contenere un massimo di quattro articoli (libri o album). Fare in modo che il metodo `aggiungi` stampi un errore quando si prova ad aggiungere il quinto articolo, e che l'elemento non venga aggiunto.

Creare la classe `TestCarrello`, per testare il corretto funzionamento della classe `Carrello`, invocando tutti i metodi definiti, e provare ad aggiungere undici articoli per verificare il corretto funzionamento implementato nel metodo precedente. Correggere eventuali bug.

Esercizio 7.o) Varargs, Vero o Falso:

1. I varargs permettono di utilizzare i metodi come se fossero degli overload.
2. La seguente dichiarazione è compilabile correttamente:

```
public void myMethod(String... s, Date d) {  
    ...  
}
```


3. La seguente dichiarazione è compilabile correttamente:

```
public void myMethod(String... s, Date d...) {  
    ...  
}
```
4. Considerando il seguente metodo:

```
public void myMethod(Object... o) {  
    ...  
}
```

la seguente invocazione è corretta:

```
oggetto.myMethod();
```
5. La seguente dichiarazione è compilabile correttamente:

```
public void myMethod(Object o, Object os...) {  
    ...  
}
```
6. Considerando il seguente metodo:

```
public void myMethod(int i, int... is) {  
    ...  
}
```

la seguente invocazione è corretta:

```
oggetto.myMethod(new Integer(1));
```
7. Le regole dell'override cambiano con l'introduzione dei varargs.
8. Il metodo di `java.io.PrintStream printf` è basato sul metodo `format` della classe `java.util.Formatter`.
9. Il metodo `format` di `java.util.Formatter` non ha overload perché definito con un varargs.
10. Nel caso in cui si passi un array come varargs al metodo `printf` di `java.io.PrintStream`, questo verrà trattato non come oggetto singolo ma come se fossero stati passati ad uno ad uno ogni suo elemento.

Esercizio 7.p)

Data la seguente gerarchia:

```
interface A {  
    void metodo();  
}  
  
interface B extends A {  
    static void metodoStatico() {}  
}  
  
final class C implements B {}  
  
public abstract class D implements A {  
    @Override  
    void metodo() {}  
}
```

Scegliere tutte le affermazioni vere:

1. La classe `C` non eredita il metodo `metodoStatico`.
2. L'interfaccia `B` non può implementare un'altra interfaccia.
3. La classe `C` implementando `B` eredita anche il metodo `metodo` astratto di `A`, e non essendo dichiarata astratta non può essere compilata.

4. La classe `D` non compila perché non può essere dichiarata `abstract`. Infatti non dichiara alcun metodo astratto.
5. La classe `D` non compila perché il metodo che dichiara non è dichiarato `public`.
6. La classe `D` compila solo perché il metodo è annotato con `Override`.

Esercizio 7.q)

Quali delle seguenti affermazioni è corretta (scegliere tutte quelle corrette):

1. Un'interfaccia estende la classe `Object`.
2. Un metodo che prende come parametro un reference di tipo `Object`, può prendere in input qualsiasi oggetto di qualsiasi tipo, anche di tipo interfaccia.
3. Un metodo che prende come parametro un reference di tipo `Object`, può prendere in input qualsiasi oggetto di qualsiasi tipo, anche un array.
4. Un metodo che prende come parametro un reference di tipo `Object`, può prendere in input qualsiasi oggetto di qualsiasi tipo, anche una collezione eterogenea.
5. Tutti i cast di oggetti, sono valutati al tempo di compilazione.

Esercizio 7.r)

Considerando le seguenti classi:

```
public class StampaNumero {
    public void stampa(double numero) {
        System.out.print(numero);
    }
}

public class StampaIntero extends StampaNumero {
    public void stampa(int numero) {
        System.out.print(numero);
    }

    public static void main(String args[]) {
        StampaNumero stampaNumero = new StampaIntero();
        stampaNumero.stampa(1);
    }
}
```

Eseguendo `StampaIntero`, qual è l'output di questo programma?

1. 1.2
2. 1
3. 1.0
4. 11.2

Esercizio 7.s)

Considerate la seguente gerarchia:

```
public interface Satellite {
    void orbita();
}
```

```
public class Luna implements Satellite {
    @Override
    public void orbita() {
        System.out.println("Luna che orbita");
    }
}

public class SatelliteArtificiale implements Satellite {
    @Override
    public void orbita() {
        System.out.println("Satellite artificiale che orbita");
    }
}
```

E la seguente classe di test:

```
public class TestSatelliti {
    public static void main(String args[]) {
        test(new Luna(), new SatelliteArtificiale());
        Satellite[] satelliti = {
            new Luna(), new SatelliteArtificiale()
        };
        test(satelliti);
        test();
        test(new Object());
    }

    public static void test(Satellite... satelliti) {
        for (Satellite satellite : satelliti) {
            satellite.orbita();
        }
    }
}
```

Scegliere tutte le affermazioni corrette:

1. L'applicazione compila e viene eseguita senza errori
2. L'applicazione non compila per l'istruzione `test(satelliti);`
3. L'applicazione non compila per l'istruzione `test();`
4. L'applicazione non compila per l'istruzione `test(new Object());`
5. L'applicazione compila ma al runtime si blocca per un'eccezione



Esercizio 7.t)

Definire l'overload e l'override. E fare un esempio di una sottoclasse, che implementa entrambi i concetti.

Esercizio 7.u)

Tenendo presente che `Number` è superclasse della classe `Integer`, consideriamo la seguente gerarchia:

```
public abstract class SommaNumero {
    public abstract Number somma (Number n1, Number n2);
}
```

```
}  
  
public class SommaIntero extends SommaNumero {  
    @Override  
    public Integer somma(Number n1, Number n2) {  
        return (Integer)n1 + (Integer)n2;  
    }  
}
```

Scegliere tutte le affermazioni corrette:

1. La classe `SommaIntero` compila senza errori.
2. La classe `SommaIntero` non compila perché l'override non è corretto: i tipi di ritorno non coincidono.
3. La classe `SommaIntero` non compila perché non è possibile utilizzare l'operatore `+` se non con numeri di tipo primitivo.
4. La classe `SommaIntero` potrebbe causare un'eccezione al runtime.



Esercizio 7.v)

Rendere *robusto* il metodo `somma` della classe `SommaIntero` definito nell'esercizio 7.u, in modo che al runtime funzioni senza generare eccezioni.

Esercizio 7.w)

Definire brevemente cos'è un parametro polimorfo, cosa sono le collezioni eterogenee, e cosa è una chiamata virtuale ad un metodo

Esercizio 7.x)

Partendo dalla soluzione dell'esercizio 6.y, facciamo evolvere la nostra applicazione che simula una rubrica:

1. Sostituire il metodo `stampaDettagli` con l'override del metodo `toString`, nelle classi `Contatto` e `Speciale`. Questa modifica è coerente, perché tali classi rappresentano dati dell'applicazione, e quindi non dovrebbero "stampare dettagli". Fare in modo che il metodo `toString` restituisca anche l'identificativo (variabile `id`).
2. Nella classe `Rubrica` aggiungere all'array `contatti` anche gli elementi dell'array `contattiSpeciali`, ed eliminare la variabile `contattiSpeciali`. Modificare i metodi `getContatti` e `getContattiSpeciali` per fare in modo che il primo restituisca tutti i contatti (speciali e non), mentre il secondo restituisca solo i contatti speciali.
3. Se ce n'è bisogno, modificare la classe `Utente` per farla funzionare in maniera coerente alla nuova implementazione della classe `Rubrica`.
4. Modificare le classi `RicercaContatti`, e `RicercaContattiSpeciali`, affinché funzionino coerentemente con le modifiche effettuate.

Esercizio 7.y)

Partendo dalla soluzione dell'esercizio precedente:

1. Aggiungere nella classe `Rubrica` un metodo chiamato `getContattiOrdinari` che restituisca tutti i contatti ordinari, ovvero che non sono di tipo `Speciale`.
2. Aggiungere un metodo chiamato `cercaContattiOrdinariPerNome` nella classe `Utente`, che cerca per nome solo i contatti che non sono speciali.
3. Creare una classe `RicercaContattiOrdinari` omologa alle altre classi di ricerca che abbiamo già creato.



Esercizio 7.z)

Partendo dalla soluzione dell'esercizio precedente:

1. Sino ad ora, abbiamo delegato alla classe che contengono il metodo `main` la responsabilità di creare gli oggetti `Contatto` esplicitamente. Creiamo quindi una classe `ContattoFactory` che abbia questa responsabilità, dichiarando uno o più metodi statici `getContatto`, a cui vengono passati i campi necessari per istanziare gli oggetti.
2. Nella classe `Utente`, aggiungere un metodo chiamato `aggiungi` che aggiunge un oggetto `Contatto` all'array `contatti`. Attenzione che nella classe `Rubrica`, l'array `contatti` può contenere un numero finito di oggetti di tipo `Contatto`. Aggiungere un numero di posti liberi per poter inserire nuovi oggetti.
3. Testare nella classe `RicercaContatti` il corretto funzionamento del metodo `aggiungi` della classe `Rubrica`. Creare gli oggetti da passare al metodo sfruttando la classe `ContattoFactory`.

Esercizi del capitolo 8

Eccezioni ed asserzioni

La gestione delle eccezioni è un argomento fondamentale, è molto importante imparare ogni dettaglio dell'argomento (non sono molti in fondo). Le asserzioni sono molto meno usate, ma potrebbero essere usate con profitto.

Esercizio 8.a) *Gestione delle eccezioni e degli errori, Vero o Falso:*

1. Ogni eccezione che estende una `ArithmeticException` è una unchecked exception.
2. Un `Error` si differenzia da una `Exception` perché non può essere lanciato; infatti non estende la classe `Throwable`.
3. Il seguente frammento di codice:

```
int a = 10;
int b = 0;
try {
    int c = a/b;
    System.out.println(c);
}
catch (ArithmeticException exc) {
    System.out.println("Divisione per zero...");
}
catch (NullPointerException exc) {
    System.out.println("Reference nullo...");
}
catch (Exception exc) {
    System.out.println("Eccezione generica...");
}
finally {
    System.out.println("Finally!");
}
```

produrrà il seguente output:

```
Divisione per zero...
Eccezione generica...
Finally!
```

4. Il seguente frammento di codice:

```
int a = 10;
int b = 0;
try {
    int c = a/b;
    System.out.println(c);
}
catch (Exception exc) {
    System.out.println("Eccezione generica...");
}
catch (ArithmeticException exc) {
    System.out.println("Divisione per zero...");
}
catch (NullPointerException exc) {
    System.out.println("Reference nullo...");
}
```

```
}  
finally {  
    System.out.println("Finally!");  
}
```

produrrà un'eccezione al runtime.

5. La parola chiave `throw` permette di lanciare “a mano” solo le sottoclassi di `Exception` che crea il programmatore.
6. La parola chiave `throw` consente di lanciare “a mano” solo le sottoclassi di `Exception`.
7. Se un metodo fa uso della parola chiave `throw`, affinché la compilazione abbia buon esito nello stesso metodo deve essere gestita l'eccezione che si vuole lanciare, o il metodo stesso deve utilizzare una clausola `throws`.
8. Non è possibile estendere la classe `Error`.
9. Se un metodo `m2` fa override di un altro metodo `m2` posto nella superclasse, non potrà dichiarare con la clausola `throws` eccezioni nuove che non siano sottoclassi rispetto a quelle che dichiara il metodo `m2`.
10. Dalla versione 1.4 di Java è possibile “includere” in un'eccezione un'altra eccezione.

Esercizio 8.b) Gestione delle asserzioni, Vero o Falso:

1. Se in un'applicazione un'asserzione non viene verificata, si deve parlare di bug.
2. Un'asserzione che non viene verificata provoca il lancio da parte della JVM di un `AssertionError`.
3. Le precondizioni servono per testare la correttezza dei parametri di metodi pubblici.
4. È sconsigliato l'utilizzo di asserzioni laddove si vuole testare la correttezza di dati inseriti da un utente.
5. Una postcondizione serve per verificare che al termine di un metodo sia verificata un'asserzione.
6. Un'invariante interna permette di testare la correttezza dei flussi all'interno dei metodi.
7. Un'invariante di classe è una particolare invariante interna che deve essere verificata per tutte le istanze di una certa classe, in ogni momento del loro ciclo di vita, tranne che durante l'esecuzione di alcuni metodi.
8. Un'invariante sul flusso di esecuzione, è solitamente un'asserzione con una sintassi del tipo:
`assert false;`
9. Non è in alcun modo possibile compilare un programma che fa uso di asserzioni con il JDK 1.3.
10. Non è in alcun modo possibile eseguire un programma che fa uso di asserzioni con il JDK 1.3.

Esercizio 8.c)

Consideriamo le classi create negli esercizi del capitolo 7: `File`, `FileSorgente`, `Editor`, `IDE` e `JavaIDE`. Consideriamo inoltre il metodo `aggiungiTesto(String)` della classe `FileSorgente` che abbiamo creato nell'esercizio 7.f, che avevamo codificato nel seguente modo:

```
public void aggiungiTesto(String testo) {  
    if (contenuto != null && testo != null) {  
        contenuto += testo;  
    }  
}
```

Il controllo della clausola `if` si può sicuramente migliorare. Cosa usereste in questo caso, asserzioni o eccezioni?

Esercizio 8.d)

(ALLARME SPOILER: le prossime righe rivelano la soluzione del precedente esercizio)

Dopo aver letto la soluzione dell'esercizio precedente, usare le parole chiave `throw` ed eventualmente `throws` per gestire eventuali eccezioni nel metodo `aggiungiTesto(String)` citato nell'esercizio precedente.



Esercizio 8.e)

Consideriamo ora il metodo `aggiungiTesto(String, int)` che abbiamo creato nell'esercizio 7.g e che avevamo codificato nel seguente modo:

```
public void aggiungiTesto(String testo, int posizione) {  
    final int length = contenuto.length();  
    if (contenuto != null && testo != null && posizione > 0  
        && posizione < length) {  
        contenuto = contenuto.substring(0, posizione) + testo +  
            contenuto.substring(posizione);  
    }  
}
```

Gestire un'eccezione (o più eccezioni) tramite le parole chiave `try-catch`, e se possibile anche `finally`.



Esercizio 8.f)

Dopo aver svolto l'esercizio precedente, creare una classe di test `TestFileSorgente` per verificare che la gestione delle eccezioni funzioni correttamente.

Esercizio 8.g)

Consideriamo i sorgenti creati con gli esercizi realizzati per il capitolo 5. Creare delle eccezioni personalizzate per gestire le situazioni impreviste. In particolare creare un'eccezione che scatti nel costruttore della classe `PortaMonete` quando vengono specificate troppe monete, chiamiamola `PortaMonetePienoException`. Gestire l'eccezione direttamente nel costruttore istanziando comunque l'oggetto con un numero limitato di elementi (ovvero senza cambiare il comportamento

già definito nell'esercizio del capitolo 5). Gestire anche la `NullPointerException` (quale istruzione potrebbe generare tale eccezione?).



Esercizio 8.h)

Consideriamo i sorgenti creati con gli esercizi realizzati per il capitolo 5. Usare la gestione delle eccezioni personalizzate per gestire le situazioni impreviste. In particolare usare l'eccezione `PortaMonetePienoException` affinché scatti nel metodo `aggiungi` che avevamo definito nel modo seguente:

```
public void aggiungi(Moneta moneta) {
    System.out.println("Proviamo ad aggiungere una " +
        moneta.getDescrizione());
    int indiceLibero = primoIndiceLibero();
    if (indiceLibero == -1) {
        System.out.println("Portamonete pieno! La moneta " +
            moneta.getDescrizione() + " non è stata aggiunta...");
    } else {
        monete[indiceLibero] = moneta;
        System.out.println("E' stata aggiunta una " +
            moneta.getDescrizione());
    }
}
```

Fare in modo che l'eccezione `PortaMonetePienoException` sia lanciata opportunamente. Gestire anche eventuali altre eccezioni.



Esercizio 8.i)

Consideriamo i sorgenti creati con gli esercizi realizzati per il capitolo 5. Creare delle eccezioni personalizzate per gestire le situazioni impreviste. In particolare usare l'eccezione `MonetaNonTrovataException` affinché scatti nel metodo `preleva` che avevamo definito nel modo seguente:

```
public Moneta preleva(Moneta moneta) {
    System.out.println("Proviamo a prelevare una " +
        moneta.getDescrizione());
    Moneta monetaTrovata = null;
    int indiceMonetaTrovata = indiceMonetaTrovata(moneta);
    if (indiceMonetaTrovata == -1) {
        System.out.println("Moneta non trovata!");
    } else {
        monetaTrovata = moneta;
        monete[indiceMonetaTrovata] = null;
        System.out.println("Una " + moneta.getDescrizione()
            + " prelevata");
    }
    return monetaTrovata;
}
```

```
}
```

Fare in modo che l'eccezione `MonetaNonTrovataException` sia lanciata opportunamente. Gestire anche eventuali altre eccezioni.



Esercizio 8.l)

Consideriamo i sorgenti creati con gli esercizi realizzati per il capitolo 5. Modificare la classe `TestMonete` per gestire correttamente l'eccezione `PortaMonetePienoException`.

Esercizio 8.m)

Aggiungere delle asserzioni nel costruttore della classe `PortaMonete`.

Esercizio 8.n)

Quali delle seguenti affermazioni sono corrette?

1. Le `RuntimeException` sono equivalenti alle checked exception.
2. `ArithmeticException` è una checked exception.
3. `ClassCastException` è una unchecked exception.
4. `NullPointerException` è una checked exception.

Esercizio 8.o)

Quali delle seguenti affermazioni sono corrette?

1. Nella clausola `throws` è possibile dichiarare solo le checked exception.
2. Nella clausola `throws` è possibile dichiarare solo le unchecked exception.
3. Nella clausola `throws` è possibile dichiarare una `NullPointerException`.
4. Con la clausola `throw` è possibile lanciare solo checked exception.
5. Con la clausola `throw` è possibile lanciare solo unchecked exception.
6. La clausola `throws` è obbligatoria se nel nostro metodo potrebbe essere lanciata una checked exception.
7. Un metodo che dichiara una clausola `throws` può essere invocato solo se si gestisce all'interno di un blocco `try-catch`.

Esercizio 8.p)

Quali delle seguenti affermazioni sono corrette?

1. È possibile definire nuove eccezioni solo di tipo checked exception.

2. Se definiamo una sottoclasse `MyException` di `NullPointerException`, sarà possibile gestire un'eccezione di tipo `NullPointerException` all'interno di una clausola `catch` a `MyException`.
3. Se definiamo una sottoclasse di `NullPointerException`, questa verrà catturata al posto della `NullPointerException`.
4. Se definiamo una sottoclasse di `ArithmeticException`, questa verrà lanciata nel caso ci sia un problema in un'operazione aritmetica.

Esercizio 8.q)

Non tenendo conto del costrutto `try-with-resources`, il blocco `finally` è obbligatorio (scegliere tutte le affermazioni valide):

1. Quando non ci sono blocchi `catch` dopo un blocco `try`.
2. Quando non ci sono blocchi `try` prima di un blocco `catch`.
3. Quando ci sono almeno due blocchi `catch` dopo un blocco `try`.
4. Mai.

Esercizio 8.r)

Considerando il seguente metodo:

```
public void metodoCheLanciaEccezione() throws ArrayIndexOutOfBoundsException {  
    //INSERIRE CODICE QUI  
}
```

Scegli tra i seguenti snippet quelli che potrebbero essere inseriti nel metodo `metodoCheLanciaEccezione` affinché il codice precedente sia valido:

1. `throw new ArrayIndexOutOfBoundsException();`
2. `int i = 0, j = 0;`
 `try {`
 `i = i/j;`
 `} catch(ArithmeticException e) {`
 `throw new ArrayIndexOutOfBoundsException ();`
 `}`
3. `int i = 0;`
4. `System.out.println();`



Esercizio 8.s)

Considerando la seguente classe:

```
public class Esercizio8S {  
    public static void main(String args[]) throws NullPointerException {  
        Esercizio8S e = new Esercizio8S();  
        e.metodo();  
    }  
  
    public NullPointerException metodo() throws Exception {
```

```
String s = null;
try {
    s.toString();
} catch(ArithmeticException e) {
    throw new NullPointerException ();
}
return null;
}
```

Scegliere tutte le affermazioni corrette:

1. Il codice non compila perché il metodo `metodo` non può ritornare `NullPointerException`.
2. Il codice non compila perché il metodo `metodo` ritorna `null` e non una `NullPointerException`.
3. Il codice non compila perché il metodo `main` non dichiara la giusta eccezione nella sua clausola `throws`.
4. Il codice compila ma al runtime termina con una `NullPointerException`.
5. Il codice compila ma al runtime termina con una `Exception`.
6. Il codice compila ma al runtime termina con una `ArithmeticException`.

Esercizio 8.t)

Creare una classe `PortaAutomatica` che dichiari due metodi, `open` e `close`, dove quest'ultimo deve essere compatibile per essere chiamato con la tecnica del `try-with-resources`.

Esercizio 8.u)

Considerando la soluzione dell'esercizio 8.t (ovvero la classe `PortaAutomatica`): scrivere una semplice classe che ne testi il funzionamento mediante il costrutto `try-with-resources`.

Esercizio 8.v)

Riprendiamo l'esercizio 7.u dove avevamo verificato che le seguenti classi compilavano senza errori, ma che `SommaIntero` poteva lanciare un'eccezione al runtime.

```
public abstract class SommaNumero {
    public abstract Number somma (Number n1, Number n2);
}

public class SommaIntero extends SommaNumero {
    @Override
    public Integer somma(Number n1, Number n2) {
        return (Integer)n1 + (Integer)n2;
    }
}
```

Infatti con le seguenti istruzioni:

```
SommaIntero sommaIntero = new SommaIntero();
sommaIntero.somma(1.0, 1.0);
```

otterremo quest'eccezione al runtime:

```
Exception in thread "main" java.lang.ClassCastException:
  java.base/java.lang.Double cannot be cast to
  java.base/java.lang.Integer
    at SommaIntero.somma(SommaIntero.java:4)
    at SommaIntero.main(SommaIntero.java:9)
```

Nell'esercizio 7.v, avevamo chiesto di rendere robusta l'implementazione della classe `SommaIntero`, ed il risultato è stato il seguente:

```
public class SommaIntero extends SommaNumero {
    @Override
    public Integer somma(Number n1, Number n2) {
        if (n1 == null || n2 == null) {
            System.out.println("Impossibile sommare un operando null, " +
                "restituisco il valore di default");
            return Integer.MIN_VALUE;
        } else if (!(n1 instanceof Integer && n2 instanceof Integer)) {
            System.out.println("Passa solo variabili di tipo intere, " +
                "restituisco il valore di default");
            return Integer.MIN_VALUE;
        }
        return (Integer)n1 + (Integer)n2;
    }
}
```

Ora che si conosce la teoria delle eccezioni, riprogettare la classe `SommaIntero` usando la gestione delle eccezioni.

Esercizio 8.z)

Creare una semplice classe di test per la classe `SommaIntero` che abbiamo creato nell'esercizio 8.v.

Esercizi riassuntivi Parte II

Un esempio guidato alla programmazione ad oggetti

Basandoci sull'approfondimento 6.1 intitolato “Un esempio guidato alla programmazione ad oggetti”, abbiamo creato esercizi di riepilogo su tutti gli argomenti che abbiamo studiato sino ad ora. Tutti gli esercizi prevedono programmazione. In particolare, nella prima parte creeremo un'applicazione partendo dalla classe `Punto` che abbiamo definito in questo capitolo, fino a disegnare graficamente linee spezzate (polilinee) su un piano cartesiano. Poi creeremo un'applicazione che capace di convertire i valori espressi in scala Kelvin, Celsius e Fahrenheit. Infine creeremo un'applicazione che simula un gioco testuale configurabile e multi-utente. Dal prossimo capitolo in poi, studieremo argomenti più complessi iniziando la terza parte del “Caratteristiche avanzate del linguaggio”, quindi cerchiamo di rinforzare le nostre basi di programmazione.

Esercizio 8.aa)

Creare:

- La classe `Punto` con coordinate x e y , i relativi metodi setter e getter, un costruttore e un metodo `toString` (che ricordiamo essere un metodo ereditato dalla classe `Object`, e quindi ereditato da tutte le classi).
- La classe `Righello` che dichiara un metodo `calcolaDistanza` che prende in input due oggetti di tipo `Punto` e restituisce il valore della distanza geometrica di tipo `double`. Leggere la documentazione e sfruttare i metodi della classe `Math` per eseguire i calcoli. Inoltre valutare se dichiarare il metodo statico oppure no.
- Creare una classe `Esercizio8AA`, che stampa la distanza di due punti e controllare se il risultato è corretto utilizzando un'asserzione come visto nel capitolo 8.

Per poter utilizzare le asserzioni bisogna abilitarle nel momento dell'esecuzione utilizzando l'opzione `-ea` (vedi capitolo 8). Se utilizzate EJE le asserzioni dovrebbero già essere abilitate di default. È possibile controllare all'interno delle opzioni di EJE come mostrato in figura 8.aa.1. Se invece utilizzate un altro IDE, consultare la documentazione dell'IDE.

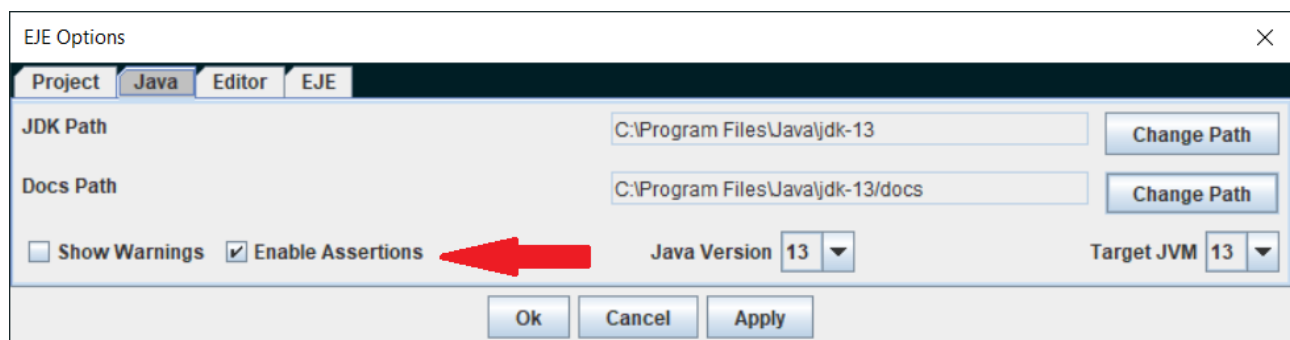


Figura 8.aa.1 – Abilitazione delle asserzioni in EJE.

Esercizio 8.bb)

Partendo dall'esercizio precedente, creare la classe `Esercizio8BB`. Questa volta parametrizziamo l'applicazione utilizzando l'array di stringhe `args`, parametro di input del metodo `main`. In particolare, quando lanciamo la classe `Esercizio8BB`, bisogna passare in input le quattro coordinate dei due punti. Per esempio così:

```
java Esercizio8BB 2 1 1 2
```

Siccome queste variabili all'interno del metodo `main` saranno di tipo `String`, usando la documentazione della classe `Integer`, trovare il metodo corretto che ci permetterà di tramutare una stringa in un intero, e gestire eventuali eccezioni.

Non è richiesto il controllo fatto con le asserzioni nell'esercizio precedente perché non possiamo sapere a priori i valori delle coordinate che saranno passati al metodo `main`.

Esercizio 8.cc)

Partendo dalla soluzione dell'esercizio precedente, creare la classe `Esercizio8CC` utilizzando la classe `Scanner`, per acquisire da riga di comando le coordinate dei due punti. Tale classe deve:

- Chiedere l'inserimento delle coordinate (una per volta).
- Acquisire le coordinate mediante il metodo della classe `Scanner` più opportuno (consultare la relativa documentazione).
- Se l'utente inserisce una coordinata in un formato non corretto (ricordiamo che le coordinate devono essere numeri interi), il programma deve gestire l'eccezione, stampare un messaggio di errore e terminare.

Segue un esempio di output che dovrebbe generare l'applicazione:

```
Inserisci coordinata x per il punto 1
2
Inserisci coordinata y per il punto 1
1
Creato punto:(2,1)
Inserisci coordinata x per il punto 2
1
Inserisci coordinata y per il punto 2
2
Creato punto:(1,2)
Distanza tra i punti: (2,1) e (1,2) = 1.4142135623730951
```

Esercizio 8.dd)

Partendo dalla soluzione dell'esercizio precedente, creare la classe `Esercizio8DD`, eseguendo un refactoring sulla classe `Esercizio8CC`, cercando di limitare il codice duplicato e renderlo più leggibile.



Esercizio 8.ee)

Partendo dalla soluzione dell'esercizio precedente, creare la classe `Esercizio8EE`, facendo in modo che, se l'utente inserisce una coordinata con un formato sbagliato (per esempio inserendo una lettera), l'applicazione deve segnalare l'errore a permettere all'utente di reinserire la coordinata. Il programma deve terminare solo nel momento in cui verranno inserite tutte le coordinate correttamente e la distanza verrà stampata. Segue un esempio di output che potrebbe generare l'applicazione:

```
Inserisci coordinata x per il punto 1
f
Coordinata non valida f!, Le coordinate devono essere di tipo intero!
Prego reinserire
2
Inserisci coordinata y per il punto 1
1
Creato punto:(2,1)
Inserisci coordinata x per il punto 2
1
Inserisci coordinata y per il punto 2
y
Coordinata non valida y!, Le coordinate devono essere di tipo intero!
Prego reinserire
pippo
Coordinata non valida pippo!, Le coordinate devono essere di tipo intero!
Prego reinserire
3
Creato punto:(1,3)
Distanza tra i punti: (2,1) e (1,3) = 2.23606797749979
```

Esercizio 8.ff)

Partendo dalla soluzione dell'esercizio precedente:

- creare la classe `Segmento`, che astrae il concetto di linea retta compresa tra due punti. Tale classe deve dichiarare come variabili d'istanza i due estremi e la distanza. Creare anche i metodi setter e getter che si ritiene di dover implementare, un costruttore e un metodo `toString`.
- creare la classe `Esercizio8FF`, sulla falsariga della classe `Esercizio8EE`, che chiede le coordinate dei due estremi del segmento e ne stampa la lunghezza.

Segue un esempio di output che potrebbe generare l'applicazione:

```
Definiamo un segmento specificando i due estremi:
Inserisci coordinata x per l'estremo 1
2
Inserisci coordinata y per l'estremo 1
pippo
Coordinata non valida pippo!, Le coordinate devono essere di tipo intero!
Prego reinserire
3
Creato estremo:(2,3)
```



```
Inserisci coordinata x per l'estremo 2
1
Inserisci coordinata y per l'estremo 2
1
Creato estremo:(1,1)
Segmento da P1(2,3) a P2(1,1) con lunghezza = 2.23606797749979
```

Esercizio 8.gg)

In questo esercizio utilizzeremo una classe chiama `CartesianPlane` (in italiano “piano cartesiano”), che ci permetterà di visualizzare i nostri punti su un piano cartesiano.

Purtroppo, a meno che non abbiate già studiato tutti i capitoli di questo libro, non dovreste essere in grado di capirne tutto il codice. Tuttavia per completezza ne riportiamo il codice di seguito (trovate comunque la classe già pronta nel file degli esercizi nella cartella 8.gg):

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JFrame;
import javax.swing.SwingUtilities;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Dimension;
import java.util.*;

public class CartesianPlane {
    private final static int LENGTH = 600;
    private final static int MID_LENGTH = LENGTH/2;
    private final static int GAP = LENGTH/10;
    private JFrame frame;
    private JPanel panel;
    private ArrayList<Punto> points;

    public CartesianPlane() {
        frame = new JFrame("Cartesian Plane");
        panel = new CartesianPlanePanel();
        points = new ArrayList<>();
        setup();
        addDetails();
    }

    private void setup() {
        frame.add(panel);
    }

    private void addDetails() {
        frame.getContentPane().setPreferredSize(new Dimension(LENGTH , LENGTH));
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }

    public void drawPoint(Punto point) {
        points.add(point);
        panel.repaint();
    }
}
```

```
private class CartesianPlanePanel extends JPanel {
    public CartesianPlanePanel() {
        setBackground(Color.lightGray);
    }

    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        drawAxes(g);
        g.setColor(Color.RED);
        drawPoints(g);
    }

    private void drawPoints(Graphics g) {
        for (Punto point : points) {
            drawPoint(point, g);
        }
    }

    private void drawPoint(Punto point, Graphics g) {
        int x = getX(point.getX(), 2);
        int y = getY(point.getY(), 2);
        g.drawOval(x,y,5,5);
        g.fillOval(x,y,5,5);
        g.drawString(point.toString(), x-15, y-8);
    }

    private int getX(int x, int delta) {
        return MID_LENGTH + (gap(x)-delta);
    }

    private int getY(int y, int delta) {
        return MID_LENGTH - (gap(y)+delta);
    }

    private void drawAxes(Graphics g) {
        drawReferencePoints(g);
        g.setColor(Color.BLACK);
        g.drawLine(MID_LENGTH, 0, MID_LENGTH, LENGTH);
        g.drawLine(0, MID_LENGTH, LENGTH, MID_LENGTH);
    }

    private void drawReferencePoints(Graphics g) {
        g.setColor(Color.YELLOW);
        for(int i = 1; i < 10; i++) {
            g.drawLine(GAP*i, 0, GAP*i, LENGTH);
        }
        for(int i = 1; i < 10; i++) {
            g.drawLine(0, GAP*i, LENGTH, GAP*i);
        }
    }

    private int gap(int value) {
        return value*GAP;
    }
}
```

È il primo esercizio di programmazione dove vedrete un'applicazione che utilizza la grafica (a questo argomento sono dedicati gli ultimi due capitoli del libro). In qualsiasi caso, lo scopo del

nostro esercizio non è quello di comprendere il codice della classe `CartesianPlane`, quanto quello di utilizzare questa classe creando la classe `Esercizio8GG` a partire dalla classe `Esercizio8FF`.

La classe `Esercizio8GG` dovrà aprire il pannello del cartello cartesiano con al seguente istruzione:

```
SwingUtilities.invokeLater() -> cartesianPlane = new CartesianPlane();
```

(importando `javax.swing. SwingUtilities`).

Inoltre dovrà:

- Aggiungere alla classe `Punto` una nuova variabile d'istanza `nome`, con relativi metodi setter e getter e un nuovo costruttore.
- Costruire un punto con le coordinate inserite dall'utente. Settare anche il nome del punto, con un meccanismo automatico (per esempio il primo punto deve chiamarsi `P1`, il secondo `P2`, il terzo `P3`, e così via).
- Invece di stampare i dettagli del punto sulla console, la classe deve invocare il metodo `drawPoint` della classe `CartesianPlane`.

Verificare che i punti vengano visualizzati correttamente sul piano cartesiano (vedi figura 8.gg.1).

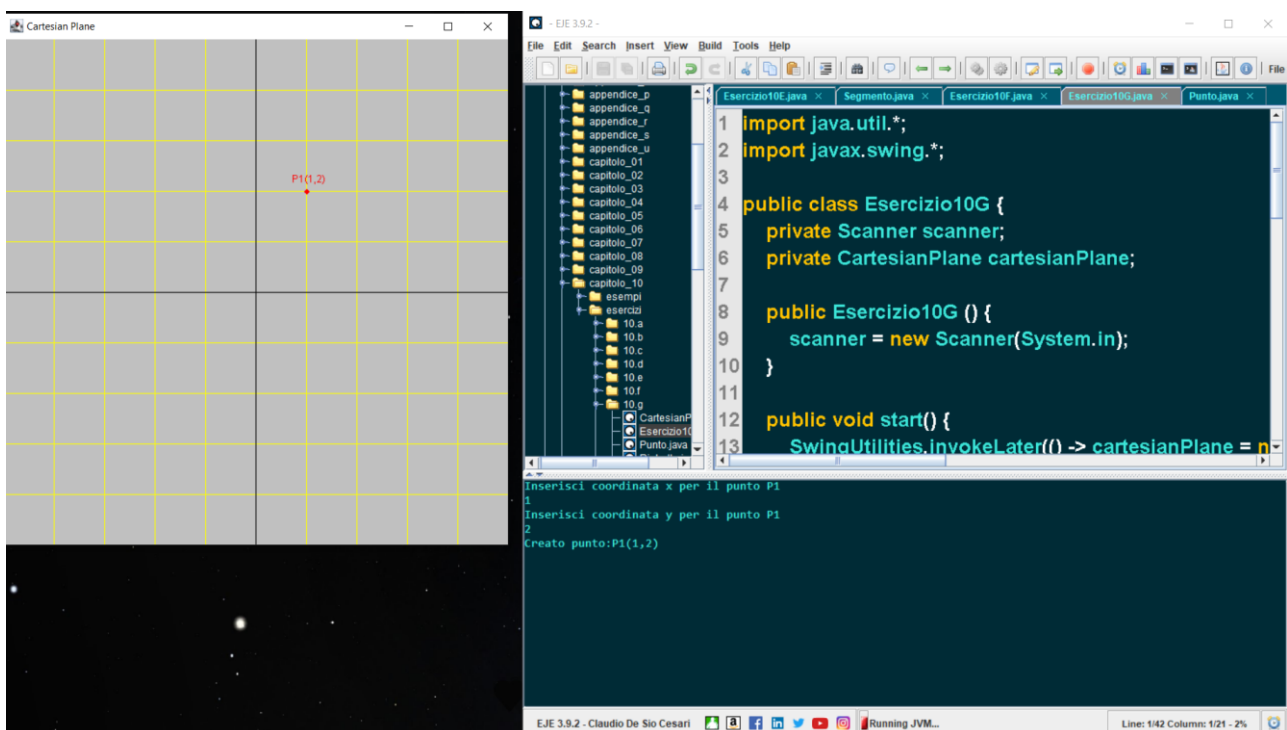


Figura 8.gg.1 – Esempio di funzionamento dell'applicazione.

Esercizio 8.hh)

Partendo dall'esercizio precedente far evolvere la classe `Esercizio8GG`, nella classe `Esercizio8HH`. Questa deve continuare a chiedere all'utente di inserire le coordinate di un punto all'infinito, sino a quando l'applicazione non verrà chiusa chiudendo la finestra del piano cartesiano. In questo modo l'utente potrà visualizzare più punti sull'asse cartesiano.

La risoluzione di questo esercizio è molto semplice.



Esercizio 8.ii)

Partendo dall'esercizio precedente, e utilizzando la classe `Segmento` realizzata nell'esercizio 8.ff, forniamo una nuova versione della classe `CartesianPlane`, a cui abbiamo aggiunto la funzionalità di poter visualizzare segmenti (basta invocare il metodo `drawSegment`). La nuova classe `CartesianPlane` è riportata di seguito con le modifiche in grassetto (trovate comunque la classe già pronta nel file degli esercizi nella cartella 8.ii):

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JFrame;
import javax.swing.SwingUtilities;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Dimension;
import java.util.*;

public class CartesianPlane {
    private final static int LENGTH = 600;
    private final static int MID_LENGTH = LENGTH/2;
    private final static int GAP = LENGTH/10;
    private JFrame frame;
    private JPanel panel;
    private ArrayList<Punto> points;
    private ArrayList<Segmento> segments;

    public CartesianPlane() {
        frame = new JFrame("Cartesian Plane");
        panel = new CartesianPlanePanel();
        points = new ArrayList<>();
        segments = new ArrayList<>();
        setup();
        addDetails();
    }

    private void setup() {
        frame.add(panel);
    }

    private void addDetails() {
        frame.getContentPane().setPreferredSize(new Dimension(LENGTH , LENGTH));
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }

    public void drawPoint(Punto point) {
        points.add(point);
        panel.repaint();
    }

    public void drawSegment(Segmento segment) {
        segments.add(segment);
        panel.repaint();
    }
}
```

```
}

private class CartesianPlanePanel extends JPanel {
    public CartesianPlanePanel() {
        setBackground(Color.lightGray);
    }

    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        drawAxes(g);
        g.setColor(Color.RED);
        drawPoints(g);
        g.setColor(Color.BLUE);
        drawSegments(g);
    }

    private void drawSegments(Graphics g) {
        for (Segmento segment : segments) {
            Punto estremo1 = segment.getEstremo1();
            drawPoint(estremo1, g);
            Punto estremo2 = segment.getEstremo2();
            drawPoint(estremo2, g);
            g.setColor(Color.BLUE);
            g.drawLine(getX(estremo1.getX()), getY(estremo1.getY()),
                getX(estremo2.getX()), getY(estremo2.getY()));
        }
    }

    private void drawPoints(Graphics g) {
        for (Punto point : points) {
            drawPoint(point, g);
        }
    }

    private void drawPoint(Punto point, Graphics g) {
        int x = getX(point.getX(), 2);
        int y = getY(point.getY(), 2);
        g.drawOval(x,y,5,5);
        g.fillOval(x,y,5,5);
        g.drawString(point.toString(), x-15, y-8);
    }

    private int getX(int x) {
        return getX(x, 0);
    }

    private int getY(int y) {
        return getY(y, 0);
    }

    private int getX(int x, int delta) {
        return MID_LENGTH + (gap(x)-delta);
    }

    private int getY(int y, int delta) {
        return MID_LENGTH - (gap(y)+delta);
    }

    private void drawAxes(Graphics g) {
        drawReferencePoints(g);
    }
}
```

```
        g.setColor(Color.BLACK);
        g.drawLine(MID_LENGTH, 0, MID_LENGTH, LENGTH);
        g.drawLine(0, MID_LENGTH, LENGTH, MID_LENGTH);
    }

    private void drawReferencePoints(Graphics g) {
        g.setColor(Color.YELLOW);
        for(int i = 1; i < 10; i++) {
            g.drawLine(GAP*i, 0, GAP*i, LENGTH);
        }
        for(int i = 1; i < 10; i++) {
            g.drawLine(0, GAP*i, LENGTH, GAP*i);
        }
    }

    private int gap(int value) {
        return value*GAP;
    }
}
```

Sulla falsariga della classe `Esercizio8HH`, creare una classe `Esercizio8II` che permette all'utente specificare gli estremi di segmenti che devono essere visualizzati sul piano cartesiano.



Esercizio 8.jj)

Partendo dalla soluzione dell'esercizio precedente, creare una classe `Polilinea`, che astrae il concetto di polilinea (conosciuta anche come *linea spezzata*). Questa deve dichiarare un `ArrayList` di segmenti. Una polilinea ha il vincolo che il secondo estremo di un segmento, deve coincidere con il primo estremo del segmento successivo. Tenere presente che inoltre una polilinea è sempre costituita da almeno due segmenti. Quindi creare uno o più costruttori, il metodo `aggiungiSegmento` che permette di aggiungere un segmento alla polilinea, e il metodo `toString`. Gestire eventuali problemi con le eccezioni, come studiato nel capitolo 8. Creare inoltre una classe di test che chiameremo `TestPolilinea` che avrà il seguente metodo `main`:

```
public static void main(String args[]) {
    testPolilineaCorretta();
    testPolilineaCorrettaConQuattroPunti();
    testPolilineaSenzaSegmenti();
    testPolilineaConUnSoloSegmento();
    testPolilineaConDueSegmentiNonConsecutiviNelCostruttore();
    testPolilineaConTreSegmentiNonConsecutiviNelCostruttore();
    testPolilineaCorrettaAggiungendoUnSegmentoConsecutivo();
    testPolilineaScorrettaAggiungendoUnSegmentoNonConsecutivo();
}
```

Implementare tali metodi con delle asserzioni (se il test fallisce, deve essere lanciato un errore tramite un'asserzione).



Esercizio 8.kk)

Nell'approfondimento 8.2, abbiamo eseguito questo test con JUnit:

```
import org.junit.Assert;
import org.junit.Test;

public class TestPunto{
    @Test
    public void testDammiDistanzaSullAscissa() {
        Punto p1 = new Punto(1,1);
        Punto p2 = new Punto(1,2);
        double distanza = p1.dammiDistanza(p2);
        Assert.assertTrue(distanza == 1);
    }
    @Test
    public void testDammiDistanzaConNull() {
        Punto p1 = new Punto(1,1);
        Punto p2 = null;
        double distanza = p1.dammiDistanza(p2);
        Assert.assertTrue(distanza == -1);
    }
    @Test
    public void testDammiDistanzaDalloStessoPunto() {
        Punto p1 = new Punto(1,1);
        Punto p2 = new Punto(1,1);
        double distanza = p1.dammiDistanza(p2);
        Assert.assertTrue(distanza == 0);
    }
    @Test
    public void testDammiDistanzaDaUnPunto3D() {
        Punto p1 = new Punto(1,1);
        Punto p2 = new Punto3D(1,2,2);
        double distanza = p1.dammiDistanza(p2);
        Assert.assertTrue(distanza == -1);
    }
}
```

sulle classi Punto:

```
public class Punto {
    private int x, y;
    public Punto() {
        //Costruttore senza parametri
    }
    public Punto(int x, int y) {
        this.setXY(x, y); //Il this è facoltativo
        //riutilizziamo codice
    }
    public void setX(int x) {
        this.x = x; //Il this non è facoltativo
    }
    public void setY(int y) {
        this.y = y; //Il this non è facoltativo
    }
    public void setXY(int x, int y) {
        this.setX(x); //Il this è facoltativo
        this.setY(y);
    }
    public int getX() {
        return this.x; //Il this è facoltativo
    }
    public int getY() {
```

```
        return this.y; //Il this è facoltativo
    }
    public double dammiDistanza(Punto p) {
        //quadrato della differenza delle x dei due punti
        int tmp1 = (x - p.x)*(x - p.x);
        //quadrato della differenza della y dei due punti
        int tmp2 = (y - p.y)*(y - p.y);
        //radice quadrata della somma dei due quadrati
        return Math.sqrt(tmp1 + tmp2);
    }
}
```

e Punto3D:

```
public class Punto3D extends Punto {
    private int z;
    public Punto3D() {
        //Costruttore senza parametri
    }
    public Punto3D(int x, int y, int z) {
        this.setXYZ(x, y, z); //Riuso di codice
    }
    public void setZ(int z) {
        this.z = z; //Il this non è facoltativo
    }
    public void setXYZ(int x, int y, int z) {
        this.setXY(x, y); //Riuso del codice
        this.setZ(z); //Il this è facoltativo
    }
    public int getZ() {
        return this.z; //Il this è facoltativo
    }
    @Override
    public double dammiDistanza(Punto p) {
        if (p instanceof Punto3D) {
            //Chiamata ad un metodo privato tramite casting
            return this.calcolaDistanza((Punto3D)p);
        }
        else {
            return -1; //distanza non valida!
        }
    }

    private double calcolaDistanza(Punto3D p1) {
        //quadrato della differenza della x dei due punti
        int tmp1=(getX()-p1.getX())*(getX()-p1.getX());
        //quadrato della differenza della y dei due punti
        int tmp2=(getY()-p1.getY())*(getY()-p1.getY());
        //quadrato della differenza della z dei due punti
        int tmp3=(z-p1.z)*(z-p1.z);
        //radice quadrata della somma dei tre quadrati
        return Math.sqrt(tmp1+tmp2+tmp3);
    }
}
```

ottenendo il fallimento nel 50% dei test. In questo esercizio modificheremo la classe `TestPunto`, risolvendo i problemi evidenziati dal test.

Per semplicità si consiglia di utilizzare un IDE come Eclipse che già integra lo strumento JUnit.

Esercizio 8.11)

Partendo dalla soluzione dell'esercizio 8.jj, trasformare la classe `TestPolilinea`, utilizzando invece delle asserzioni il tool JUnit.

Per semplicità si consiglia di utilizzare un IDE come Eclipse che già integra lo strumento JUnit.



Esercizio 8.mm)

Partendo dalla soluzione dell'esercizio 8.ii, ed aggiungendo le classi `Polilinea` e `PolilineaNonValidaException` definite nell'esercizio 8.jj, creare sulla falsariga della classe `Esercizio8II`, la classe `Esercizio8MM` che consente all'utente di creare polilinee sul piano cartesiano. L'applicazione dovrà richiedere all'utente di inserire punto dopo punto gli estremi dei segmenti che costituiscono una polilinea (che nel frattempo verrà visualizzata dalla classe `CartesianPlane`). Solo per il primo segmento verrà specificato il primo estremo, mentre per i successivi, è scontato che il primo estremo coincide con il secondo estremo del segmento precedente.

Segue un esempio dell'output che dovrà generare la nostra applicazione:

```
Definiamo i primi due segmenti di una polilinea :
Definiamo il primo estremo del primo segmento della polilinea:
Inserisci coordinata x per il punto P1
1
Inserisci coordinata y per il punto P1
2
Creato punto:P1(1,2)
Definiamo il secondo estremo del primo segmento della polilinea:
Inserisci coordinata x per il punto P2
3
Inserisci coordinata y per il punto P2
4
Creato punto:P2(3,4)
Definiamo il secondo estremo del secondo segmento della polilinea:
Inserisci coordinata x per il punto P3
5
Inserisci coordinata y per il punto P3
6
Creato punto:P3(5,6)
Inserisci coordinata x per il punto P4
3
Inserisci coordinata y per il punto P4
3
Creato punto:P4(3,3)
Inserisci coordinata x per il punto P5
```



Esercizio 8.nn)

Testando la soluzione dell'esercizio 8.mm, abbiamo notato che specificando un segmento di estremi coincidenti, il sistema disegna due punti uno sopra l'altro, quindi non si dovrebbe neanche parlare di segmento. Gestire quindi l'eccezione affinché il sistema non permetta di creare segmenti con due estremi coincidenti.

Esercizio 8.oo)

Creiamo la classe `Esercizio800` che rappresenterà un convertitore dalla scala Kelvin alle scale Celsius e Fahrenheit. La nostra applicazione deve chiedere all'utente di inserire un valore in Kelvin, e il programma dovrà convertire il valore inserito in gradi Celsius e Fahrenheit.

Ecco un esempio di output che dovrà generare la nostra applicazione:

```
Inserisci un valore in Kelvin
0
Il valore di 0.0 gradi Kelvin equivale a:
-273.15 gradi Celsius
-459.66998 gradi Fahrenheit
```

Utilizzare il tipo `float` per fare tutti i calcoli.

Ricordiamo che la formula per convertire i gradi Kelvin (che rappresentiamo con "K") in Celsius (che rappresentiamo con "C") è: $0\text{ K} - 273,15 = -273,1\text{ C}$. Ricordiamo che la formula per convertire i gradi Kelvin in Fahrenheit (che rappresentiamo con "F") è: $(0\text{ K} - 273,15) \times 9/5 + 32 = -459,7\text{ F}$.

Esercizio 8.pp)

L'output dell'esercizio precedente non ci soddisfa in quanto l'utilizzo del tipo `float` non ci permette di definire una formattazione univoca. Far evolvere la classe `Esercizio800` nella classe `Esercizio8PP`, utilizzando la classe `BigDecimal` in modo tale che i valori vengano formattati specificando solo le prime due cifre decimali. È necessario studiare la documentazione della classe `BigDecimal`.

Ecco un esempio di output che dovrà generare la nostra applicazione:

```
Inserisci un valore in Kelvin
0
Il valore di 0 gradi Kelvin equivale a:
-273.15 gradi Celsius
-459.67 gradi Fahrenheit
```

Esercizio 8.qq)

Far evolvere la classe `Esercizio8PP` nella classe `Esercizio8QQ`, in modo tale che l'utente possa partire da una qualsiasi scala ed ottenere nelle altre scale.

Ecco un esempio di output che dovrà generare la nostra applicazione:

```
Inserisci un valore in Kelvin (per esempio 12K), Celsius (25C) o Fahrenheit (451F)
0k
Il valore di 0 gradi Kelvin equivale a:
-273.15 gradi Celsius
-459.67 gradi Fahrenheit
```

oppure:

```
Inserisci un valore in Kelvin (per esempio 12K), Celsius (25C) o Fahrenheit (451F)
25C
Il valore di 25 gradi Celsius equivale a:
298.15 gradi Kelvin
77.00 gradi Fahrenheit
```

o anche:

```
Inserisci un valore in Kelvin (per esempio 12K), Celsius (25C) o Fahrenheit (451F)
500F
Il valore di 500 gradi Fahrenheit equivale a:
260.00 gradi Celsius
533.15 gradi Kelvin
```

8Esercizio 8.rr)

Far evolvere la classe `Esercizio8QQ` nella classe `Esercizio8RR`, cercando di migliorare l'astrazione del nostro codice. Creiamo quindi una classe `Convertitore` che avrà la responsabilità di eseguire tutte le conversioni aritmetiche. Dichiarerà quindi tutti i metodi (statici) che servono per convertire una scala in un'altra. Per esempio ecco la dichiarazione del metodo di conversione da Kelvin a Fahrenheit:

```
public static BigDecimal convertiKelvinInFahrenheit(BigDecimal kelvin) {
    //scrivi il tuo codice qui
}
```

Ovviamente la classe `Esercizio8RR` dovrà ora utilizzare i metodi della classe `Convertitore`.

Esercizio 8.ss)

Far evolvere la classe `Esercizio8RR` nella classe `Esercizio8SS`, cercando di migliorare l'astrazione del nostro codice. Creiamo quindi una classe `Stampante` che avrà la responsabilità di stampare tutti i messaggi. Dichiarerà quindi tutti i metodi (statici) che servono per stampare i messaggi. Per esempio ecco la dichiarazione del metodo che stampa il messaggio di errore quando si inserisce un simbolo non valido per scegliere la scala del valore inserito:

```
public static void stampaScalaErrata(String scala) {
    //scrivi il tuo codice qui
}
```

Ovviamente la classe `Esercizio8SS` dovrà ora utilizzare i metodi della classe `Stampante`.

Esercizio 8.tt)

In questo esercizio proveremo ad utilizzare per la prima volta la libreria `java.logging`, in realtà introdotta solo nell'approfondimento 14.1, che potete anche già consultare. A differenza di quanto visto nell'approfondimento 14.1, andremo a configurare il nostro log tramite il seguente file di configurazione (che trovate nella cartella del 8.tt del codice del libro):

```
handlers=java.util.logging.FileHandler
.level=ALL
java.util.logging.FileHandler.pattern=./esercizio8tt.log
java.util.logging.FileHandler.formatter=java.util.logging.SimpleFormatter
Esercizio8TT.level=ALL
```

Dove viene specificato che il log verrà generato all'interno di un file nella stessa cartella chiamato `esercizio8tt.log`. Inoltre viene specificato che verranno stampati tutti i messaggi di log a qualsiasi livello (istruzione `Esercizio8TT.level=ALL`).

Inserire solo all'interno del file `Esercizio8TT` (evoluzione del file `Esercizio8SS`) il seguente metodo:

```
private static void initLogging() {
    try {
        LogManager.readConfiguration(
            new FileInputStream("logging.properties"));
    } catch (IOException exception) {
        LOGGER.log(Level.SEVERE,
            "Problema con la lettura del file di configurazione", exception);
    }
}
```

e richiamarlo come prima istruzione del metodo `main`, affinché la configurazione sia tramite file sia correttamente eseguita. Inoltre inserire a proprio piacimento alcune istruzioni di log con livelli diversi. Testare che funzioni tutto correttamente utilizzando l'applicazione e controllando il file di log. Testare più volte l'applicazione specificando nel file di configurazione, al posto della parola `ALL` altri livelli di configurazione come `SEVERE`, `INFO` etc. verificando che vengano stampati solo i messaggi con i livelli di log consentiti dalla configurazione.



Esercizio 8.uu)

Ore implementeremo un semplice gioco dopo tutta questa matematica e fisica! La nostra applicazione chiederà all'utente di indovinare un numero casuale compreso tra 1 e 100 contando il numero di tentativi.

Un esempio dell'output che dovrà essere generato da questa applicazione è il seguente:

```
Ho pensato ad un numero tra 1 e 100, indovinalo!
50
Troppo alto, riprova
25
Troppo basso, riprova
37
Troppo basso, riprova
42
Troppo basso, riprova
```

```
48
Troppo basso, riprova
49
Complimenti! Hai indovinato dopo 6 tentativi
```

Proviamo a scrivere le nostre classi in inglese (in fondo ci dobbiamo abituare), e creare la classe `GuessNumber` che contiene un unico metodo statico che genera il numero casuale tra 1 e il numero massimo che gli viene passato in input. Ecco la dichiarazione del metodo:

```
public static int generateRandomNumber(int max) {
    //scrivi il tuo codice qui
}
```

Poi, data la seguente interfaccia:

```
public interface Game {
    public void init();
    public void start();
    public void play();
    public void end();
}
```

implementarla in una classe chiamata `GuessNumberGame`, che implementa i metodi dell'interfaccia nel modo più opportuno. Nella stessa classe possiamo anche mettere il metodo `main` che farà partire l'applicazione.

Esercizio 8.vv)

La soluzione dell'esercizio precedente, non teneva conto però della possibilità che l'utente inserisca dati non validi. Quindi far evolvere la classe `GuessNumberGame`, in modo tale da gestire un eventuale input scorretto da parte dell'utente.

Un esempio dell'output che dovrà essere generato da questa applicazione è il seguente:

```
Ho pensato ad un numero tra 1 e 100, indovinalo!
u
Input 'u' non valido. Puoi inserire solo numeri interi, riprova
```



Esercizio 8.ww)

Iniziamo a modificare la nostra applicazione per renderla un gioco multi-utente. Creiamo quindi la classe `Player` che deve definire:

- le variabili d'istanza `name` e `id` (che servirà per identificare univocamente un giocatore anche in caso di coincidenza di nomi).
- Un unico costruttore.
- Un semplice metodo `toString`.

Definire una classe astratta `MultiPlayerGame` (gioco multi-utente) che implementa l'interfaccia `Game` definita nell'esercizio 8.uu, e che definisce:

- come variabile d'istanza un `ArrayList` di oggetti `Player` chiamato `players`.
- Il metodo getter `getPlayers`, che restituisce la variabile d'istanza `players`.
- I metodi `addPlayer` e `removePlayer` che aggiungono o rimuovono un oggetto di tipo `Player` dalla variabile d'istanza `players`.

- Un metodo `getPlayer` che prende in input l'indice del player all'interno della lista `players`, e che ritorna l'oggetto `Player` corrispondente, oppure rilancia una `PlayerException` (da creare anch'essa), nei casi l'indice non sia valido.
- Un metodo `getPlayer` che prende in input il nome di un giocatore presente all'interno della lista `players`, e che ritorna l'oggetto `Player` corrispondente, oppure rilancia una `PlayerException`, nei casi in cui, non venga trovato il player con il nome specificato, o se sono presenti più giocatori con lo stesso nome all'interno della lista.
- Creare una classe concreta `MultiPlayerGameImpl` ("Impl" sta per implementazione) che estende `MultiPlayerGame`, e definisce un nuovo metodo `printPlayers` che stampa i dettagli di tutti i giocatori della lista `players`.
- Infine creare una classe di test `MultiPlayerGameTest` che istanzia un oggetto da `MultiPlayerGameImpl`, e tutti testa tutti i metodi definiti da `MultiPlayerGame`.



Esercizio 8.xx)

Considerando le classi implementate nella soluzione dell'esercizio 8.ww (escludendo `MultiPlayerGameImpl`), e la seguente interfaccia:

```
public interface Configurator {  
    public void config();  
}
```

L'obiettivo di questo esercizio, è di creare una implementazione dell'interfaccia `Configurator` chiamata `MultiPlayerGameConfigurator`. Questa classe deve avere la responsabilità di permettere la configurazione di un gioco di diventare multi-giocatore. In particolare deve permettere di inserire i nomi dei giocatori che devono partecipare al gioco, assegnando ad ognuno di essi un id progressivo. Questa classe deve dichiarare come variabile d'istanza quantomeno un oggetto di tipo `MultiPlayerGameImpl`, a cui devono essere aggiunti i giocatori.

Lanciando quindi la seguente classe:

```
public class MultiPlayerGameConfiguratorTest {  
    public static void main(String args[]) {  
        new MultiPlayerGameConfigurator();  
    }  
}
```

vogliamo che l'output del nostro programma sia simile al seguente:

```
Inserire nome player 1  
Claudio  
Inserire nome player 2 oppure 'i' per iniziare a giocare  
Pippo  
Inserire nome player 3 oppure 'i' per iniziare a giocare  
i  
Lista dei giocatori:  
Player 1: Claudio  
Player 2: Pippo
```



Esercizio 8.yy)

Ora, dopo aver realizzato delle implementazioni delle interfacce `MultiPlayer` e `Configurator`, non ci resta che unire il tutto e far evolvere la nostra applicazione definita nell'esercizio 8.vv in un gioco multi-utente e configurabile. Quindi per prima cosa, rendiamo la classe `Player` astratta, aggiungendogli il seguente metodo astratto `play`:

```
public abstract boolean play();
```

d'altronde un giocatore che non definisce un metodo `play` non ha molto senso.

Quindi definire l'implementazione della classe `Player` per il nostro gioco, che chiameremo

`GuessNumberPlayer`. Essa deve avere la responsabilità di:

- contare il numero di tentativi fatti per indovinare il numero.
- Implementare il metodo `play` sulla falsariga di quanto faceva prima la classe `GuessNumberGame`. Infatti ora sarà il giocatore a giocare, e quindi deve implementare il codice che gli permetta di giocare. Il metodo `play` deve ritornare `true`, solo quando il giocatore indovina il numero.

Inoltre bisogna ridefinire la classe `GuessNumberGame`, in modo tale che implementi l'interfaccia

`Configurator`, e gestisca il gioco in maniera multi-utente utilizzando oggetti di tipo

`GuessNumberPlayer`. In particolare essa deve:

- ridefinire il metodo `config` per gestire la configurazione dell'applicazione.
- Ridefinire il metodo `play` in modo che sfrutti il metodo `play` degli oggetti di tipo `GuessNumberGame`.

Esercizio 8.zz)

In questo ultimo esercizio, l'unica richiesta è quella di inserire ogni classe in un package con un nome significativo, in modo da rendere evidente la struttura architetturale della nostra applicazione.

Di seguito vengono elencati tutti i package dell'applicazione:

- `games.generic.exceptions`
- `games.generic.data`
- `games.guessnumber.data`
- `games.guessnumber.business`
- `games.guessnumber.util`
- `games.generic.business`
- `games.generic.business`

il vostro compito è quindi quello di assegnare il package giusto alla classe (o interfaccia giusta).

Esercizi del capitolo 9

Tipi Enumerazioni e Tipi Record

Il capitolo 9 è il primo dei capitoli della terza parte del libro “Caratteristiche avanzate”. Gli argomenti trattati in questo capitolo, non sono tutto sommato molto complessi. Ci sono alcune regole particolari che bisogna comprendere, ma per la programmazione, l'importante è capire quando conviene utilizzare record ed enumerazioni. Anche per questo capitolo troverete tanti esercizi a risposta multipla (alcuni molto difficili) che supportano alla preparazione dell'esame di certificazione Oracle, ma anche diverse implementazioni da codificare.

Esercizio 9.a) Record, Vero o Falso:

1. Un record viene trasformato in classe durante la compilazione. È possibile verificarlo mediante il tool `javap` fornito dal JDK.
2. Un record estende la classe `java.util.Record`, e quindi non può estendere altre classi.
3. Non è possibile estendere un record, infatti i record sono dichiarati di default `final`.
4. Un record non può implementare interfacce solo se sono di interfacce `sealed`.
5. Un record implementa automaticamente i metodi setter e getter.
6. Possiamo sempre evitare di scrivere un record, implementando una classe equivalente che però richiederà un maggior sforzo implementativo a livello di righe di codice.
7. Possiamo sempre evitare di scrivere una classe, implementando una record equivalente che però richiederà un maggior sforzo implementativo a livello di righe di codice.
8. Nel momento in cui implementiamo un'interfaccia in un record, dovremo implementare obbligatoriamente tutti i metodi astratti ereditati.
9. Ogni record ha un costruttore di default.
10. Non è possibile creare un metodo setter per un record. Infatti, le variabili d'istanza sono sempre automaticamente dichiarate `final`.



Per rispondere ad alcune domande del prossimo esercizio bisogna prima studiare l'approfondimento 9.5 sui tipi innestati (opzionale) che trovate sul file materiale extra online su www.nuovojava.it.

Esercizio 9.b) Enumerazioni e tipi innestati, Vero o Falso:

1. Le enumerazioni non si possono istanziare se non all'interno della definizione dell'enumerazione stessa. Infatti, possono avere solamente costruttori `private`.
2. Un'enumerazione può dichiarare metodi, e i suoi elementi possono essere estesi da classi che possono sottoporre a override i metodi dell'enumerazione. Non è però possibile che una `enum` estenda un'altra `enum`.
3. Il metodo `values` appartiene ad ogni enumerazione ma non alla classe `java.lang.Enum`.
4. Il seguente codice viene compilato senza errori:

```
public enum MyEnum {  
    public void metodo1() {
```



```
    }  
    public void metodo2() {  
  
    }  
    ENUM1, ENUM2;  
}
```

5. Il seguente codice viene compilato senza errori:

```
public enum MyEnum {  
    ENUM1 {  
        public void metodo() {  
  
        }  
    }, ENUM2;  
    public void metodo2() {  
  
    }  
}
```

6. Il seguente codice viene compilato senza errori:

```
public enum MyEnum {  
    ENUM1 (), ENUM2;  
    private MyEnum(int i) {  
  
    }  
}
```

7. Il seguente codice viene compilato senza errori:

```
public class Volume {  
    public enum Livello {  
        ALTO, MEDIO, BASSO  
    } ;  
    // implementazione della classe . . .  
    public static void main(String args[]) {  
        switch (getLivello()) {  
            case ALTO:  
                System.out.println(Livello.ALTO);  
                break;  
            case MEDIO:  
                System.out.println(Livello.MEDIO);  
                break;  
            case BASSO:  
                System.out.println(Livello.BASSO);  
                break;  
        }  
    }  
    public static Livello getLivello() {  
        return Livello.ALTO;  
    }  
}
```

8. Se dichiariamo la seguente enumerazione:

```
public enum MyEnum {  
    ENUM1 {  
        public void metodo1() {  
  
        }  
    },  
    ENUM2 {  
        public void metodo2() {  
  
        }  
    }  
}
```

```
    }  
}  
il seguente codice potrebbe essere correttamente compilato:  
MyEnum.ENUM1.metodo1();  
9. Non è possibile dichiarare enumerazioni con un unico elemento.  
10. Si possono innestare enumerazioni in enumerazioni in questo modo:  
public enum MyEnum {  
    ENUM1 (), ENUM2;  
    public enum MyEnum2 {a,b,c}  
}  
ed il seguente codice viene compilato senza errori:  
System.out.println(MyEnum.MyEnum2.a);
```



Esercizio 9.c)

Modificare i sorgenti creati con gli esercizi realizzati per il capitolo 8, dopo aver sostituito l'interfaccia `TipoFile`, creata nell'esercizio 7.c, con un'enumerazione. Tutto dovrà funzionare come in precedenza.

Esercizio 9.d)

Anche se ancora non solo introdotto nell'approfondimento 2.1 i **tipi innestati** (argomento che verrà ripreso solo nel capitolo 13), come tipi che si possono dichiarare all'interno di altri tipi, possiamo dire che è molto frequente dichiarare le enumerazioni, all'interno di classi. Questo perché spesso un'enumerazione viene pensata per un particolare contesto, e spesso deve essere utilizzata da un'unica classe.

Quindi, partendo dalla soluzione dell'esercizio precedente, innestare l'enumerazione `TipoFile` nella classe `File`. Che modifiche bisogna apportare all'applicazione per continuare a farla funzionare come prima?



Esercizio 9.e)

La soluzione di questo esercizio riporta una sintassi complessa che non è stata riportata nelle pagine del libro cartaceo. In pratica, ogni singolo elemento dell'enumerazione può ridefinire al volo i metodi definiti nella enumerazione stessa. Questo significa che lo stesso metodo può avere un'implementazione diversa per ogni elemento. Quindi se non si riesce risolvere l'esercizio non c'è da preoccuparsi. Tale sintassi è usata raramente, e si è preferito non introdurla per non appesantire troppo l'argomento con caratteristiche meno usate. Ricordiamo che è previsto un volume complementare che dovrebbe essere pubblicato contemporaneamente alla release 17 di Java, che sarà dedicato alle

novità introdotte dalla prossima versione LTS (cfr. appendice A), e alle caratteristiche avanzate.

Consideriamo i sorgenti creati con gli esercizi realizzati per il capitolo 5, e modificati con gli esercizi del capitolo 8. Attualmente, la classe `Moneta` potrebbe essere istanziata in maniera scorretta, e potrebbe essere necessario gestire eventuali eccezioni. Per esempio, come parametro al costruttore della classe `Moneta` potrebbe essere passato un valore negativo. Segue il codice che abbiamo sino ad ora sviluppato (commenti omissi):

```
public class Moneta {

    public final static String VALUTA = "EURO";

    private final int valore;

    public Moneta(int valore) {
        this.valore = valore;
        System.out.println("Creata una " + getDescrizione());
    }

    public int getValore() {
        return valore;
    }

    public String getDescrizione() {
        String descrizione = "moneta da " + formattaStringaDescrittiva(valore)
            + VALUTA;
        return descrizione;
    }

    private String formattaStringaDescrittiva(int valore) {
        String stringaFormattata = " centesimi di ";
        if (valore == 1) {
            stringaFormattata = " centesimo di ";
        } else if (valore > 99) {
            stringaFormattata = " ";
            valore /= 100;
        }
        return valore + stringaFormattata;
    }
}
```

Creare un'enumerazione che consenta di rendere il costruttore (e quindi l'intero programma) più robusto, e che ci permetta di evitare la gestione delle eccezioni. Riscrivere anche la classe `Moneta`.

Esercizio 9.f)

In base all'esercizio precedente, modificare la classe `PortaMonete` di conseguenza.

Esercizio 9.g)

In base ai due ultimi esercizi, modificare la classe `TestMonete` di conseguenza.

Esercizio 9.h)

Consideriamo la classe `PortaMonete` rifinita nell'esercizio 9.f, e concentriamoci sul metodo privato `indiceMonetaTrovata`, che abbiamo definito nel seguente modo:

```
private int indiceMonetaTrovata(Moneta moneta) {
    int indiceMonetaTrovata = -1;
    final int size = monete.size();
    for (int i = 0; i < size; i++) {
        if (monete.get(i) == null) {
            break;
        }
        Valore valoreMonetaNelPortaMoneta = monete.get(i).getValore();
        Valore valore = moneta.getValore();
        if (valore == valoreMonetaNelPortaMoneta) {
            indiceMonetaTrovata = i;
            break;
        }
    }
    return indiceMonetaTrovata;
}
```

Creare un metodo `equals` nella classe `Moneta`, in modo tale da semplificare la ricerca in questo metodo.

Esercizio 9.i)

Riscrivere il metodo `toString` del seguente record `Persona`:

```
public record Persona(String nome, String cognome, String dataDiNascita, String
    professione, String indirizzo) {}
```

il nostro requisito è che il metodo `toString` restituisca una stringa con il seguente formato:

```
Nome:           Arjen Anthony
Cognome:         Lucassen
Professione:     Compositore
Data di Nascita: 03/04/1960
Indirizzo:       Olanda
```

Per la formattazione è possibile sfruttare il carattere di escape `\t` introdotto nel capitolo 3.

Esercizio 9.j)

Qual è l'output della seguente classe?

```
public class Esercizio9J {
    public static void main(String args[]) {
        String stringa1 = "Claudio";
        String stringa2 = new String(stringa1);
        System.out.println(stringa2 == stringa1);
        System.out.println(stringa2.equals(stringa1));
        System.out.println("Claudio".equals(stringa1));
        System.out.println("Claudio" == stringa1);
        System.out.println("Claudio" == stringa2);
    }
}
```

Esercizio 9.k) Approfondimento `import static`, Vero o Falso:

1. Gli `import static` permettono di non referenziare i membri statici importati.
2. Non è possibile, dopo avere importato staticamente una variabile, referenziarla all'interno del codice.
3. La seguente importazione non è corretta perché `java.lang` è sempre importato implicitamente:

```
import static java.lang.System.out;
```
4. In alcuni casi gli `import static` potrebbero peggiorare la leggibilità dei nostri file.
5. Considerando la seguente enumerazione:

```
package mypackage;  
public enum MyEnum {  
    A, B, C  
}
```


il seguente codice è compilabile correttamente:

```
import static mypackage.MyEnum.*;  
public class MyClass {  
    public MyClass() {  
        out.println(A);  
    }  
}
```
6. Se utilizziamo gli `import static`, si potrebbero importare anche due membri statici con lo stesso nome. Il loro utilizzo all'interno del codice darebbe luogo ad errori in compilazione, se non referenziati.
7. Lo *shadowing* è un fenomeno che potrebbe verificarsi se si utilizzano gli `import static`.
8. Essenzialmente l'utilità degli `import static` risiede nella possibilità di scrivere meno codice probabilmente superfluo.
9. Non ha senso importare staticamente una variabile, se poi viene utilizzata una sola volta all'interno del codice.
10. Non è consigliato utilizzare gli `import static` per importare gli elementi di enumerazioni.



Per rispondere ad alcune domande del prossimo esercizio bisogna prima studiare l'approfondimento 9.5 sui tipi innestati (opzionale) che trovate nel file del materiale extra online su www.nuovojava.it.

Esercizio 9.q)

Dato il seguente codice:

```
public enum Esercizio9Q {  
    A, B, C;  
  
    private enum InnerEnum {  
        C, D, E;  
  
        protected enum InnerInnerEnum {  
            F, G, H  
        }  
    }  
}
```

```
}

public static void main(String args[]) {
    System.out.println(/*INSERISCI CODICE QUI*/);
}
}
```

Quali delle seguenti espressioni è possibile inserire al posto del commento `/*INSERISCI CODICE QUI*/` per stampare il valore `H` (è possibile scegliere una o più espressioni):

1. `Esercizio9Q.A.InnerEnum.C.InnerInnerEnum.H`
2. `Esercizio9Q.InnerEnum.InnerInnerEnum.H`
3. `InnerInnerEnum.H`
4. `InnerEnum.A.InnerInnerEnum.H`
5. `InnerEnum.InnerInnerEnum.H`
6. `InnerEnum.InnerInnerEnum.F.G.H`
7. `Esercizio9Q.A.C.H`
8. Nessuno: il codice non compila.



Per rispondere ad alcune domande del prossimo esercizio bisogna prima studiare l'approfondimento 9.5 sui tipi innestati (opzionale) che trovate sul file materiale extra online su www.nuovojava.it.

Esercizio 9.r)

Dato il seguente codice:

```
public enum Esercizio9R implements Interface {
    UNO {
        @Override
        public int metodo() {
            return 29 + 7 + 74;
        }
    },
    DUE,
    TRE {
        @Override
        public int metodo() {
            return 12 + 11 + 6;
        }
    };
    @Override
    public int metodo() {
        return 14 + 4 + 4;
    }

    public static void main(String args[]) {
        Interface i = Esercizio9R.TRE;
        System.out.println(i.metodo());
    }
}

interface Interface {
    int metodo();
}
```

```
}
```

Se viene eseguito il file `Esercizio9R`, cosa verrà stampato (scegliere una sola risposta)?

1. 22
2. 29
3. 110
4. 161
5. TRE
6. `TRE.metodo()`
7. `i.metodo()`
8. Niente, il codice non compila.



Esercizio 9.v)

Astrarre un mazzo di carte napoletane e creare una classe eseguibile (con metodo `main`) che istanzi tutte le 40 carte.



Esercizio 9.z)

Partendo dalla soluzione dell'esercizio precedente:

- Individuare i problemi di formattazione.
- Risolvere i problemi di formattazione.

Questo esercizio, testa la vostra capacità di risolvere i problemi con inventiva, coscienza e intraprendenza. Praticamente quello che ogni programmatore dovrebbe fare sempre.

Esercizi del capitolo 10

Tipi Generici e Collezioni

I tipi generici rappresentano un argomento che può diventare molto complesso. Tra questi esercizi, ed in particolare tra quelli a risposta multipla che supportano la certificazione Java, ne troverete alcuni davvero complicati, perché per rispondere correttamente dovrete avere una preparazione eccellente.

Esercizio 10.a) *Generics, Vero o Falso:*

1. I tipi generici e i tipi parametro sono la stessa cosa.
2. Un vantaggio principale dei generics, è che permettono di evitare bachi come quelli provocati da una `ClassCastException` al runtime, individuandoli in compilazione.
3. Le collection possono essere usate anche senza specificare i tipi parametro. In tal caso si parla di raw type.
4. L'ereditarietà ignora i tipi parametro.
5. Le wildcard si usano quando non abbiamo tipi parametrici a disposizione da usare.
6. Se creiamo un tipo generico con tipo parametro `<E>`, possiamo usare lo stesso parametro anche nei metodi dichiarati nel tipo.
7. Nei metodi generici il tipo parametro viene messo come parametro di input del metodo.
8. Il seguente codice:

```
import java.util.*;

public class MyGeneric <Pippo extends Number> {
    private List<Pippo> list;
    public MyGeneric () {
        list = new ArrayList<Pippo>();
    }
    public void add(Pippo pippo) {
        list.add(pippo);
    }
    public void remove(int i) {
        list.remove(i);
    }
    public Pippo get(int i) {
        return list.get(i);
    }
    public void copy(ArrayList<?> al) {
        Iterator<?> i = al.iterator();
        while (i.hasNext()) {
            Object o = i.next();
            add(o);
        }
    }
}
```

compila senza errori.

9. Il seguente codice:

```
public <N extends Number> void print(List<N> list) {
```



```
        for (Iterator<N> i = list.iterator();  
            i.hasNext(); ) {  
            System.out.println(i.next());  
        }  
    }
```

compila senza errori.

10. Per risolvere le wildcard capture bisogna usare un metodo helper definito nella libreria standard.

11. Il seguente codice:

```
List<String> strings = new ArrayList<String>();  
strings.add(new Character('A'));
```

compila senza errori.

12. Il seguente codice:

```
List<int> ints = new ArrayList<int>();
```

compila senza errori.

13. Il seguente codice:

```
List<int> ints = new ArrayList<Integer>();
```

compila senza errori.

14. Il seguente codice:

```
List<Integer> ints = new ArrayList<Integer>();  
ints.add(1);
```

compila senza errori.

15. Il seguente codice:

```
List ints = new ArrayList<Integer>();
```

compila senza errori.

Esercizio 10.b)

Consideriamo i sorgenti creati con gli esercizi realizzati per il capitolo 5 e poi ridefiniti nei capitoli 8 e 9. Sostituire l'array `monete` definito nella classe `PortaMonete` con `ArrayList` di oggetti `Moneta`. Eventualmente modificare le altre classi in modo tale che funzioni tutto come prima.

Suggerimento: nella documentazione della classe `ArrayList` potrebbero esserci metodi utili.



Esercizio 10.c)

Creare una classe astratta `Frutta` che definisce una variabile incapsulata `peso`. Per il nostro programma un oggetto `Frutta` senza `peso` non ha senso. Creare le sue sottoclassi `Mela`, `Pesca`, `Arancia`. Creare una classe generica `Cesto` che astrae il concetto di cesto della frutta. Questa definisce un array list di frutti. Inoltre deve esporre un metodo `getPeso` che ritorna il peso totale del contenuto della cesta; un metodo `aggiungi` per aggiungere un frutto alla volta, che rilancia un'eccezione (creata appositamente) nel caso l'aggiunta del frutto che si vuole aggiungere faccia superare il limite massimo di 5 chili di peso. Ogni `Cesta` deve avere un solo tipo di frutta per volta. Implementare una soluzione con i generics. Creare una classe di test per verificare l'effettivo funzionamento del programma.

Esercizio 10.d)

Quali dei seguenti statement è compilabile (scegliere tutti gli statement corretti):

1. `HashMap hm = new HashMap();`
2. `HashMap<> hm = new HashMap<>();`
3. `HashMap<Integer, String> hm = new HashMap ()<Integer, String>;`
4. `HashMap<Double> hm = new HashMap<Double>();`
5. `HashMap<Double> hm = new HashMap<>();`

Esercizio 10.e)

Quali dei seguenti statement è compilabile (scegliere tutti gli statement corretti):

1. `ArrayList al = new ArrayList<Integer>();`
2. `ArrayList<Integer> al = new ArrayList<>();`
3. `ArrayList<String, String> al = new ArrayList<String, String>();`
4. `ArrayList al = new ArrayList<>();`
5. `ArrayList al = new ArrayList<int>();`

Esercizio 10.f)

Quali dei seguenti statement è compilabile (scegliere tutti gli statement corretti):

1. `List al = new ArrayList<HashMap<String, String>>();`
2. `Map<ArrayList<String>> hm = new ArrayList<>();`
3. `List<String, String> al = new ArrayList<HashMap<String, String>>();`
4. `List<Map<List<List<Integer>>, String>> al = new ArrayList<>();`
5. `List<Map<String, String>> al = new ArrayList<HashMap<String, String>>();`

Esercizio 10.g)

Qual è l'output delle seguenti righe di codice?

```
import java.util.*;

public class Esercizio10G {

    public static void main(String args[]) {
        List list = new ArrayList();
        list.add("1");
        list.add('2');
        list.add(3);
        Iterator iterator = list.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next());
        }
    }
}
```

```
}
```

1. 123
2. 321
3. Nessuno, il programma non si può compilare perché non è possibile aggiungere elementi diversi alla collezione.
4. "1""2"3
5. Nessuno, il programma non si può compilare perché non è possibile recuperare un `Iterator` da una collezione eterogenea.
6. L'esecuzione sarà stoppata nel momento in cui si proverà a stampare il secondo valore della collezione.

Esercizio 10.h)

Qual è l'output delle seguenti righe di codice?

```
import java.util.*;

public class Esercizio10H {
    public static void main(String args[]) {
        List<Object> list = new ArrayList<>();
        list.add("1");
        list.add('2');
        list.add(3);
        Iterator<Object> iterator = list.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next());
        }
    }
}
```

1. 123
2. "1""2"3
3. Nessuno, il programma non si può compilare perché la collezione ammette solo oggetti di tipo `Object`.
4. Nessuno, il programma non si può compilare perché non è possibile recuperare un `Iterator` da una collezione eterogenea.
5. L'esecuzione sarà stoppata nel momento in cui si proverà a stampare il secondo valore della collezione.

Esercizio 10.i)

Qual è l'output delle seguenti righe di codice?

```
import java.util.*;

public class Esercizio10I {
    public static void main(String args[]) {
        List<String> list = new ArrayList<>();
        list.add("1");
        list.add(null);
        list.add('3');
        Iterator<String> iterator = list.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next());
        }
    }
}
```

```
    }  
    }  
}
```

1. Nessuno, il programma non si può compilare perché la collezione non ammette valori `null`.
2. Nessuno, il programma non si può compilare perché la collezione non ammette valori primitivi.
3. L'esecuzione sarà stoppata nel momento in cui si proverà ad aggiungere `null` alla collezione.
4. L'esecuzione sarà stoppata nel momento in cui si proverà ad aggiungere `'3'` alla collezione.
5. `1null3`

Esercizio 10.j)

Creare una classe che, dato un array di interi, li ordina dal più alto al più basso.

Esercizio 10.k)

Creare una classe `WrapperComparable` che abbia le seguenti caratteristiche:

- Abbia come costante incapsulata un `Integer` che deve essere sempre valorizzato.
- Abbia un metodo `toString()` che restituisca l'intero.
- Definisca un modo di ordinamento che vada dall'intero incapsulato più alto al più basso.

Infine creare una classe che testi che l'ordinamento funzioni come ci si aspetta.

Esercizio 10.l)

Qual è l'output delle seguenti righe di codice?

```
import java.util.*;  
  
public class Esercizio10L {  
  
    public static void main(String args[]) {  
        Map<Integer, Integer> map = new HashMap<>(1);  
        map.put(14, 12);  
        map.put('a', 'b');  
        map.put(07, 0x0ABC);  
    }  
}
```

1. Il codice non compila perché non è possibile aggiungere tre coppie chiave-valore ad una collezione di dimensione 1.
2. Il codice non compila perché i valori `07` e `0x0ABC` non sono valori interi.
3. Il codice non compila perché i valori `'a'` e `'b'` non sono valori interi.
4. Nessuna delle precedenti.

Esercizio 10.m)

Data la seguente dichiarazione:

```
Map<String, Float> map = new Hashmap<>(1);
```

Quali tra i seguenti statement sono validi?

1. `map.add("stringa", 1.1F);`
2. `map.add("stringa", 1.1D);`
3. `map.add("stringa", 1);`
4. Nessuno dei precedenti.



Esercizio 10.n)

Data la seguente classe:

```
public class Esercizio10N/*INSERISCI CODICE 1*/ {  
  
    public static void main(String[] args) {  
        Esercizio10N<String> g = new Esercizio10N/*INSERISCI CODICE 2*/();  
        Esercizio10N<Object> g2 = new Esercizio10N/*INSERISCI CODICE 3*/();  
    }  
}
```

Se volessimo inserire le tre istruzioni mancanti affinché la compilazione vada a buon fine, quali tra queste opzioni risulteranno essere valide?

1. Codice 1: `<Object>`; Codice 2: `<Object>`; Codice 3: `<Object>`.
2. Codice 1: `<>`; Codice 2: `<>`; Codice 3: `<>`.
3. Codice 1: nessun codice; Codice 2: `<>`; Codice 3: `<>`.
4. Codice 1: `<T extends Object>`; Codice 2: `<String>`; Codice 3: `<Object>`.
5. Codice 1: `<? extends Object>`; Codice 2: `<String>`; Codice 3: `<Object>`.
6. Codice 1: `<T>`; Codice 2: `<String>`; Codice 3: `<>`.
7. Codice 1: `<T>`; Codice 2: `<T>`; Codice 3: `<Object>`.

Esercizio 10.o)

Data la seguente classe:

```
import java.util.*;  
  
public class Esercizio10O {  
  
    public static int getSize(List/*INSERISCI CODICE QUI*/ list) {  
        return list.size();  
    }  
  
    public static void main(String args[]) {  
        System.out.println(getSize(new ArrayList<Integer>()));  
        System.out.println(getSize(  
            new ArrayList<HashMap<String, List<String>>>()));  
    }  
}
```

Quali tra i seguenti parametri generici potrebbero sostituire il commento `/*INSERISCI CODICE QUI*/` per far sì che il codice compili correttamente, e permetta di stampare la dimensione della lista passata in input, qualsiasi sia il tipo di lista?

1. `<Object>`
2. `<>`

3. <?>
4. <T extends Object>
5. <? extends Object>
6. <T>
7. Nessun codice, il file può essere compilato così com'è.

Esercizio 10.p)

Dato il seguente codice:

```
public class Esercizio10P<T> {  
  
    public static T[] getValues(T t) {  
        return t.values();  
    }  
  
    private enum T {  
        T1, T2, T3;  
    }  
}
```

Quale tra le seguenti affermazioni sono corrette?

1. Il codice non compila perché non è possibile chiamare l'enumerazione innestata `T` come il tipo parametro.
2. Il codice compila correttamente.
3. Il codice compila correttamente, ma mostra un warning.
4. Il codice non compila perché non è possibile dichiarare un'enumerazione come tipo parametro.
5. Il codice non compila perché un metodo statico non può accedere ad una enumerazione non statica.
6. Nessuna delle precedenti.

Esercizio 10.q)

Data la seguente classe:

```
public class Esercizio10Q/*INSERISCI CODICE QUI*/ {  
  
    public Integer max(N n1, N n2) {  
        return Integer.max(n1.intValue(), n2.intValue());  
    }  
}
```

Quale tra i seguenti parametri generici potrebbe sostituire il commento `/*INSERISCI CODICE QUI*/` per far sì che il codice compili correttamente:

1. <Object>
2. <Number>
3. <?>
4. <T>
5. <N>
6. <N extends Number>
7. <T>
8. Nessun codice, il file può essere compilato così com'è.



Esercizio 10.r)

Data la seguente classe:

```
import java.util.List;
import java.util.ArrayList;

public class Esercizio10R<T> {

    public static void main(String args[]) {
        Esercizio10R<Void> e = new Esercizio10R<>();
        ArrayList<Integer> a = new ArrayList<>();
        a.add(2);
        a.add(6);
        a.add(10);
        a.add(24);
        a.add(17);
        System.out.println("La somma degli elementi della lista è "
            + e.sumElements(a));
    }
}
```

Creare un metodo generico (cfr. paragrafo 10.3.4) che:

- prenda in input un parametro generico *L* definito dal metodo, che rappresenta una lista di interi.
- Abbia come tipo di ritorno un intero, risultato della somma di tutti gli elementi della lista di interi.

Si noti che abbiamo utilizzato il tipo `void` come generico per l'istanza di `Esercizio10R`, visto che comunque la classe `Esercizio10R` non utilizza nemmeno il parametro.

Esercizio 10.s)

Data la seguente gerarchia:

```
public interface Tecnologia {

    void stampa();
}

public class Inkjet implements Tecnologia {

    @Override
    public void stampa() {
        System.out.println("Stampa getto d'inchiostro");
    }
}

public class Laser implements Tecnologia {

    @Override
```

```
public void stampa() {  
    System.out.println("Stampa laser");  
}  
}
```

Creare una classe generica `Stampante` parametrizzata con una tecnologia, che dichiari a sua volta un metodo `stampa`, che però delega alla propria tecnologia l'effettivo messaggio da stampare. Creare anche una classe che testa l'effettivo funzionamento del nostro codice.

Esercizio 10.t)

Data la seguente enumerazione:

```
public enum Operatore {  
  
    SOMMA("+"), SOTTRAZIONE("-"), MOLTIPLICAZIONE("X"), DIVISIONE(":");  
  
    String segno;  
  
    Operatore(String segno) {  
        this.segno = segno;  
    }  
  
    public String toString() {  
        return segno;  
    }  
}
```

e la classe:

```
public class TestOperazione {  
    public static void main(String args[]) {  
        Operazione<Integer, Operatore> operazione =  
            new Operazione<>(1, Operatore.SOMMA, 1);  
        operazione.stampa();  
    }  
}
```

creare la classe `Operazione`, con un metodo `stampa` che stampi semplicemente:

```
1 + 1
```

Esercizio 10.u)

Data la seguente classe:

```
import java.util.List;  
import java.util.Iterator;  
  
public class Esercizio10U {  
    public static <T extends Number> void cicla(List<T> list) {  
        for (Iterator<T> i = list.iterator(); i.hasNext(); ) {  
            System.out.println(i.next());  
        }  
    }  
}
```

Quali delle seguenti affermazioni riguardo il metodo `cicla` sono corrette?

1. È un metodo che usa una wildcard lower bounded.
2. È un metodo che usa un parametro bounded.
3. È un metodo che usa una wildcard upper bounded.

4. È un metodo generico.
5. È un metodo che usa una wildcard capture.
6. È un metodo helper.

Esercizio 10.v)

Data la seguente classe:

```
import java.util.List;

public class Esercizio10V {
    protected final void modifica(List<?> list) {
        list.add(list.get(0));
    }
}
```

Quali delle seguenti affermazioni riguardo il metodo `modifica` sono corrette?

1. È un metodo che non compila.
2. È un metodo generico.
3. È un metodo helper.
4. Ha bisogno di un metodo helper.



Esercizio 10.w)

Creare un programma che simula il gioco noto come “sasso-carta-forbici”, noto anche come “morra cinese”. Le istruzioni del gioco potrete trovarle a questo link:

https://it.wikipedia.org/wiki/Morra_cinese

Lanciato il programma l'utente deve specificare se scegliere sasso, carta o forbice, e contemporaneamente il programma deve fare la sua scelta (casualmente).

Si cerchi di utilizzare tutti i concetti appresi sinora per creare quest'applicazione (comprese le enumerazioni). Ricordarsi che è difficile fare delle scelte, ma con i metodi di analisi che abbiamo visto in precedenza possiamo procedere con più rigore e sicurezza.



Esercizio 10.z)

Data la seguente classe:

```
public class Esercizio10Z {

    public static <E extends Exception> void printException(E e) {
        System.out.println(e.getMessage());
    }

    public static void main(String[] args) {
        /*INSERISCI CODICE QUI*/
    }
}
```

```
}  
}
```

Quale tra i seguenti statement potrebbe sostituire il commento `/*INSERISCI CODICE QUI*/` per far sì che il codice compili correttamente:

1. `Esercizio10Z.<Exception>printException(new Exception("Exception"));`
2. `Esercizio10Z.printException(new ClassCastException("ClassCastException"));`
3. `Esercizio10Z.printException(new RuntimeException("RuntimeException"));`
4. `Esercizio10Z.printException(new Throwable("Exception"));`

La soluzione di questo esercizio riporta una sintassi complessa che non è stata riportata nelle pagine del libro cartaceo. In pratica, possiamo anticipare che la sintassi al punto 1 è valida. Tale sintassi è usata raramente, e si è preferito non introdurla per non appesantire troppo l'argomento con caratteristiche meno usate. Ricordiamo che è previsto un volume complementare che dovrebbe essere pubblicato contemporaneamente alla release 17 di Java, che sarà dedicato alle novità introdotte dalla prossima versione LTS (cfr. appendice A), e alle caratteristiche avanzate.

Esercizi del capitolo 11 e dell'appendice E

Tipi Annotazioni, Reflection e la variabile d'ambiente CLASSPATH

Con questi esercizi cercheremo di capire cosa significa sfruttare le annotazioni. Creeremo un'annotazione da zero, la sfrutteremo con un'applicazione creata ad hoc. Creeremo un verificatore di codice, che in base a delle annotazioni deciderà se le nostre classi rispettano determinati requisiti. Poi seguiranno anche tanti altri esercizi che supportano la preparazione alla certificazione, con quiz a risposta multipla. Dedicheremo alcuni esercizi anche agli altri argomenti del capitolo come la reflection, e alla variabile *classpath* introdotta nell'appendice E (questi hanno una numerazione diversa che inizia con la lettera E).

Sino ad ora, per semplificare il nostro lavoro abbiamo fatto poco uso dei package, che però solitamente si usano sempre. Con le annotazioni useremo sempre i package per i nostri esercizi. Questo è necessario perché le annotazioni non sono rilevabili tramite reflection se non si trovano in un package. Quindi si consiglia di utilizzare un IDE come Eclipse, Netbeans o IntelliJ.

Esercizio 11.a) Annotazioni, dichiarazioni ed uso, Vero o Falso:

1. Un'annotazione è un modificatore.
2. Un'annotazione è un'interfaccia.
3. Gli elementi di un'annotazione sembrano metodi astratti ma sottintendono un'implementazione implicita.
4. La seguente è una dichiarazione di annotazione valida:

```
public @interface MiaAnnotazione {  
    void metodo();  
}
```
5. La seguente è una dichiarazione di annotazione valida:

```
public @interface MiaAnnotazione {  
    int metodo(int valore) default 5;  
}
```
6. La seguente è una dichiarazione di annotazione valida:

```
public @interface MiaAnnotazione {  
    int metodo() default -99;  
    enum MiaEnum{VERO, FALSO};  
    MiaEnum miaEnum();  
}
```
7. Supponiamo che l'annotazione `MiaAnnotazione` definita nel punto 6 sia corretta. Con il seguente codice essa viene utilizzata correttamente:

```
public @MiaAnnotazione (  
    MiaAnnotazione.MiaEnum.VERO  
)  
MiaAnnotazione.MiaEnum m() {  
    return MiaAnnotazione.MiaEnum.VERO;  
}
```

8. Supponiamo che l'annotazione `MiaAnnotazione` definita nel punto 6 sia corretta. Con il seguente codice essa viene utilizzata correttamente:

```
public @MiaAnnotazione (  
    miaEnum=MiaAnnotazione.MiaEnum.VERO  
)  
MiaAnnotazione.MiaEnum m() {  
    return @MiaAnnotazione.miaEnum;  
}
```

9. Consideriamo la seguente annotazione:

```
public @interface MiaAnnotazione {  
    int valore();  
}
```

con il seguente codice essa viene utilizzata correttamente:

```
public @MiaAnnotazione (  
    5  
)  
void m()  
    //...
```

10. Consideriamo la seguente annotazione:

```
public @interface MiaAnnotazione {}
```

con il seguente codice essa, viene utilizzata correttamente:

```
public @MiaAnnotazione void m() {  
    //...  
}
```

Esercizio 11.b) Annotazioni e libreria, Vero o Falso:

1. La seguente annotazione è anche una meta-annotazione:

```
public @interface MiaAnnotazione ()
```
2. La seguente annotazione è anche una meta-annotazione:

```
@Target (ElementType.SOURCE)  
public @interface MiaAnnotazione()
```
3. La seguente annotazione è anche una meta-annotazione:

```
@Target (ElementType.@INTERFACE)  
public @interface MiaAnnotazione()
```
4. La seguente annotazione, se applicata ad un metodo, sarà documentata nella relativa documentazione Javadoc:

```
@Documented  
@Target (ElementType.ANNOTATION_TYPE)  
public @interface MiaAnnotazione()
```
5. La seguente annotazione sarà ereditata se e solo se applicata ad una classe:

```
@Inherited  
@Target (ElementType.METHOD)  
public @interface MiaAnnotazione()
```
6. Per la seguente annotazione è anche possibile creare un processore di annotazioni che riconosca al runtime il tipo di annotazione, per implementare un particolare comportamento:

```
@Documented  
@Target (ElementType.TYPE)  
@Retention(RetentionPolicy.RUNTIME)  
public @interface MiaAnnotazione()
```
7. `Override` è un'annotazione standard per segnalare al runtime di Java che un metodo effettua l'override di un altro.

8. `Deprecated` può essere considerata anche una meta-annotazione perché applicabile ad altre annotazioni.
9. `SuppressWarnings` è una annotazione a valore singolo. `Deprecated` e `Override` invece sono entrambe annotazioni marcatrici.
10. Non è possibile utilizzare contemporaneamente le `SuppressWarnings`, `Deprecated` e `Override` su di un'unica classe.



Esercizio 11.c)

Creare un'annotazione marcatrice per classi, di nome `Breve`, che deve essere usata per marcare classi che non hanno più di tre metodi. Questa annotazione deve appartenere ad un package `metadati`. Poi creare due classi `ClasseBreve` e `ClasseLunga`, la prima con un solo metodo, e l'altra con quattro metodi. Annotiamo entrambe le classi con `Breve`, e facciamole appartenere ad un package `dati`. Creiamo anche un'eccezione che chiameremo `AnnotationException` da far scattare quando una classe non rispetta la specifica dell'annotazione, che deve appartenere al package `eccezioni`.



Esercizio 11.d)

Creare una classe `VerificatoreInterattivo` che contiene un metodo `main` che verifica se una classe specificata al runtime tramite l'utilizzo di una classe `Scanner` (già incontrata in precedenti esercizi) e annotata con `Breve`, soddisfa il requisito che abbiamo specificato nell'esercizio 11.c. Ovvero deve essere possibile specificare una classe, battere il tasto "Invio" e il programma deve stampare se la verifica è andata a buon fine o un eventuale messaggio di errore. Questa classe deve appartenere ad un package chiamato `test`.



Esercizio 11.e)

Creare un'annotazione a valore unico di nome `Specifica`, che deve essere usata per marcare le classi che hanno un numero preciso di variabili d'istanza. Inserire anche questa annotazione nel package `metadati`. Usare anche questa annotazione per le classi `ClasseBreve` e `ClasseLunga`, che per l'occasione dichiareranno delle variabili d'istanza. In particolare `ClasseBreve` dichiarerà una variabile d'istanza incapsulata, mentre `ClasseLunga` dichiarerà due variabili d'istanza non incapsulate.



Esercizio 11.f)

Modificare la classe `VerificatoreInterattivo` affinché verifichi la correttezza dell'utilizzo dell'annotazione `Specifica` come fatto per l'annotazione `Breve`.

**Esercizio 11.g)**

Creare un'annotazione completa di nome `Bean`, che deve essere usata per marcare le classi che hanno un costruttore senza parametri, le variabili incapsulate, un numero di metodi non superiore a un numero da specificare, e un numero di variabili non inferiore a un numero da specificare. Inserire anche questa annotazione nel package `metadati`. Usare anche questa annotazione per le classi `ClasseBreve` e `ClasseLunga`.

**Esercizio 11.h)**

Modificare la classe `VerificatoreInterattivo` affinché verifichi la correttezza dell'utilizzo dell'annotazione `Bean` come fatto per l'annotazione `Breve` e l'annotazione `Specifica`.

Esercizio 11.i)

Quali tra le seguenti affermazioni sono corrette?

1. L'annotazione `Override` è utile solo nella fase di compilazione.
2. Un'annotazione `Override` che annota un metodo precede sempre gli eventuali modificatori del metodo.
3. L'annotazione `Override` appartiene al package `java.lang`.
4. L'annotazione `Override` può annotare solo metodi.
5. L'annotazione `Override` è una annotazione marcatrice (marker).

**Esercizio 11.j)**

Creare una classe `ReflectionInterattiva` che contiene un metodo `main` il quale stampa i metodi di una classe specificata al runtime tramite l'utilizzo di una classe `Scanner` (già incontrata più volte in precedenza). Deve essere possibile specificare una classe, battere il tasto “Invio” e il programma deve stampare le firme di tutti i metodi della classe specificata.

Questo programma potrebbe non essere eseguito correttamente su EJE, si consiglia di lanciarlo da riga di comando.



Esercizio 11.k)

Creare una classe `CompilatoreInterattivo` che contiene un metodo `main` il quale compila i file specificati specificata al runtime tramite l'utilizzo di una classe `Scanner` (già incontrata in più esercizi precedentemente). Deve essere possibile specificare una classe, battere il tasto “Invio” e il programma deve compilare la classe specificata.

Questo programma dovrebbe essere eseguito su riga di comando o tramite EJE, ma non tramite Eclipse o Netbeans. Infatti questi IDE compilano automaticamente i file e quindi non si riuscirebbe a capire bene se il file viene compilato correttamente dal nostro programma o dall'IDE.

Esercizio 11.l)

Considerata la seguente gerarchia:

```
public interface Giocatore {  
    default void gioca() {}  
}  
  
public class Bambino implements Giocatore {  
    /*INSERISCI CODICE QUI*/  
}
```

Cosa è possibile inserire al posto del commento `/*INSERISCI CODICE QUI*/` tra i seguenti statement?

1. `@Override void gioca() {}`
2. `@Override public void gioca() {}`
3. `@Override public boolean equals(Object o) {return false;}`
4. `@Override public int hashCode() {return 1;}`
5. `@Override public String toString() {return "";}`

Esercizio 11.m)

Quali tra le seguenti affermazioni sono corrette?

1. L'annotazione `Deprecated` va utilizzata al posto del tag javadoc `@Deprecated`.
2. Un'annotazione `Deprecated` è di tipo annotazione completa, e dichiara due elementi: `since`, e `forRemoval`.
3. L'annotazione `Deprecated` appartiene al package `java.lang.annotation`.
4. L'annotazione `Deprecated` è una meta-annotazione.

Esercizio 11.n)

Quali tra le seguenti affermazioni sono corrette?

1. L'annotazione `FunctionalInterface` è utile solo nella fase di compilazione.
2. Un'annotazione `FunctionalInterface` dovrebbe annotare solo interfacce che hanno un unico metodo (che viene chiamato metodo SAM).
3. L'uso dell'annotazione `FunctionalInterface` è obbligatoria se l'interfaccia annotata ha un unico metodo.
4. L'annotazione `FunctionalInterface` è un'annotazione marcatrice (marker).
5. L'uso dell'annotazione `FunctionalInterface` implica sicuramente l'uso dell'annotazione `@Override`.

**Esercizio 11.o)**

Se compiliamo la seguente classe:

```
import java.util.*;

public class Esercizio11o {

    List objects;

    public Esercizio11o () {
        objects = new ArrayList();
    }

    public void rimuovi(Object object) {
        Iterator iterator = objects.iterator();
        if (iterator.hasNext()) {
            Object item = iterator.next();
            if (object.toString().equals(item.toString())) {
                iterator.remove();
            }
        }
    }
}
```

otterremo il seguente output che ci segnala tre warning:

```
Esercizio11o.java:4: warning: [rawtypes] found raw type: List
    List objects;
    ^
missing type arguments for generic class List<E>
where E is a type-variable:
  E extends Object declared in interface List
Esercizio11o.java:7: warning: [rawtypes] found raw type: ArrayList
    objects = new ArrayList();
                   ^
missing type arguments for generic class ArrayList<E>
where E is a type-variable:
  E extends Object declared in class ArrayList
Esercizio11o.java:12: warning: [rawtypes] found raw type: Iterator
    Iterator iterator = objects.iterator();
    ^
```



```
missing type arguments for generic class Iterator<E>
where E is a type-variable:
  E extends Object declared in interface Iterator
3 warnings
```

È possibile aggiungere un'annotazione per ottenere:

- Solo due warning
- Solo un warning
- Nessun warning

Scrivere le tre versioni della classe che realizzano quanto richiesto.

Esercizio 11.p)

Quali tra le seguenti affermazioni sono corrette?

1. L'annotazione `Native` può essere usata per interfacciare Java con altri linguaggi.
2. Il termine “linguaggio nativo” significa linguaggio Java.
3. L'annotazione `Native` appartiene al package `java.lang`.
4. L'annotazione `Native` può annotare solo costanti.
5. L'annotazione `Native` è una annotazione marcatrice (marker).

Esercizio 11.q)

Quali tra le seguenti affermazioni sono corrette?

1. L'annotazione `Target` è una meta-annotazione che annota sé stessa.
2. L'annotazione `Target` è di tipo annotazione ordinaria. Infatti può specificare vari parametri.
3. `ElementType` è un'interfaccia che definisce i vari elementi di programmazione Java a cui si possono applicare le annotazioni.
4. L'annotazione `Target` annota l'annotazione `Target`.

Esercizio 11.r)

Quali tra le seguenti affermazioni sono corrette?

1. L'annotazione `Retention` è annotata a sua volta da `Target`.
2. L'annotazione `Target` è annotata a sua volta da `Retention`.
3. L'annotazione `Documented` è annotata a sua volta da `Target`.
4. L'annotazione `Target` è annotata a sua volta da `Documented`.
5. L'annotazione `Retention` è annotata a sua volta da `Documented`.
6. L'annotazione `Documented` è annotata a sua volta da `Retention`.

Esercizio 11.s)

Quali tra le seguenti affermazioni sono corrette?

1. Con l'annotazione `Retention` decidiamo se l'annotazione annotata deve o meno essere conservata all'interno della classe compilata.
2. `RetentionPolicy` è un'enumerazione che dichiara solo due elementi: `SOURCE` e `CLASS`.

3. L'annotazione `Retention` e l'enumerazione `RetentionPolicy` appartengono al package `java.lang.annotation`.
4. L'annotazione `Retention` può annotare solo annotazioni.
5. L'annotazione `Retention` è di tipo annotazione ordinaria.

Esercizio 11.t)

Quali tra le seguenti affermazioni sono corrette?

1. L'annotazione `Documented` permette di includere all'interno della documentazione javadoc l'annotazione `Documented` stessa.
2. Un'annotazione `Documented` è annotata da sé stessa.
3. L'annotazione `Documented` è ereditata di default nelle sottoclassi.
4. L'annotazione `Documented` può annotare solo annotazioni che annotano classi.
5. L'annotazione `Documented` è una annotazione marcatrice (marker).

Esercizio 11.u)

Considerato il seguente codice:

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Inherited
@Documented
public @interface Annotation11U {

}

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Inherited
@Documented
public @interface DifferentAnnotation {

}

@Annotation11U
public interface InterfacellU {

}

@DifferentAnnotation
public class Esercizio11U implements InterfacellU {

}
```

Qual è l'output del seguente programma?

```
import java.lang.reflect.*;
import java.util.*;
import java.lang.annotation.*;
public class AnnotationsReflection {
```

```
public static void main(String[] args) throws Exception {
    Annotation[] dcs=Esercizio11U.class.getAnnotations();
    for (Annotation dc : dcs) {
        System.out.println(dc);
    }
}
```

Esercizio 11.v)

Creare un inizializzatore statico (cfr. paragrafo 5.4.3) all'interno della classe `Moneta`, che abbiamo utilizzato l'ultima volta nella soluzione dell'esercizio 10.b, e facciamogli stampare una frase qualsiasi. Come è possibile usare la reflection per far eseguire il blocco?

Esercizio 11.x)

L'utilizzo di un IDE implica (scegliere tutte le risposte che si ritiene siano corrette):

1. Iniziale rallentamento dell'apprendimento dello sviluppo, perché richiede lo studio dello stesso IDE.
2. La possibilità di utilizzare un debugger. Fondamentale strumento per sviluppare.
3. La possibilità di integrarsi con altri strumenti di sviluppo con un database o un application server.
4. La possibilità di utilizzare un editor evoluto che ci permette di automatizzare la creazione di costrutti di programmazione ed utilizzare tecniche di refactoring del codice.
5. L'impossibilità di utilizzare i package, che devono comunque essere gestiti da riga di comando.

Esercizio 11.y)

L'architettura si occupa (scegliere tutte le risposte che si ritiene siano corrette):

1. Di definire diagrammi di attività UML che modellano le funzionalità del sistema.
2. Di migliorare le prestazioni del software.
3. Di ottimizzare l'utilizzo delle risorse da parte del software.
4. Di ottimizzare il partizionamento del software in modo tale da semplificare la procedura di installazione.

Esercizio 11.z)

Un deployment diagram (scegliere tutte le affermazioni che si ritiene siano corrette):

1. È un diagramma statico.
2. Mostra i flussi di esecuzione dell'applicazione.
3. Mostra come i nodi hardware ospitano i componenti software.
4. Evidenziano le dipendenze tra componenti software.
5. Il principale elemento del diagramma è un *nodo*, che è rappresentato da un rettangolo con due piccoli rettangoli che fuoriescono dall'angolo superiore sinistro.

Esercizio 11.aa)

Abbiamo asserito che la conoscenza di base di argomenti quali XML e l'interazione con database è essenziale per lavorare in un'azienda informatica. Questo perché la quasi totalità delle applicazioni utilizza in qualche modo queste due tecnologie. Se si lavora ad un'applicazione web, quali sono le conoscenze di base che bisogna avere?

Esercizio 11.bb)

Definire brevemente i concetti di:

1. client
2. server
3. applicazione standalone
4. applicazione mobile
5. applicazione web
6. client web
7. server web
8. applicazione enterprise.

Esercizio 11.cc)

Definire i compiti dei seguenti ruoli aziendali:

1. Capo progetto
2. Business analyst
3. IT manager
4. Release manager
5. DBA
6. Grafico

Esercizio 11.dd)

Definire cos'è una metodologia object oriented.

Esercizio E.a)

Impacchettare in un file JAR, il package di autenticazione progettato e realizzato negli esercizi 5.w.

Esercizi del capitolo 12

Gestione dei thread

Abbiamo visto nel capitolo 12 come la gestione dei thread sia un argomento molto complesso. Tuttavia gli esercizi che sono presentati di seguito dovrebbero risultare alquanto fattibili.

Esercizio 12.a) Creazione di Thread, Vero o Falso:

1. Un thread è un oggetto istanziato dalla classe `Thread` o dalla classe `Runnable`.
2. Il multithreading è solitamente una caratteristica dei sistemi operativi e non dei linguaggi di programmazione.
3. In ogni applicazione al runtime esiste almeno un thread in esecuzione.
4. A parte il thread principale, un thread ha bisogno di eseguire codice all'interno di un oggetto la cui classe implementa `Runnable` o estende `Thread`.
5. Il metodo `run` deve essere chiamato dal programmatore per attivare un thread.
6. Il thread corrente non si identifica solitamente con il reference `this`.
7. Chiamando il metodo `start` su di un thread, questo viene immediatamente eseguito.
8. Il metodo `sleep` è statico e permette di far dormire per un numero specificato di millisecondi il thread che legge tale istruzione.
9. Assegnare le priorità ai thread è una attività che può produrre risultati diversi su piattaforme diverse.
10. Lo scheduler della JVM non dipende dalla piattaforma su cui viene eseguito.

Esercizio 12.b) Gestione del multi-threading, Vero o Falso:

1. Un thread astrae un processore virtuale che esegue codice su determinati dati.
2. La parola chiave `synchronized` può essere utilizzata sia come modificatore di un metodo sia come modificatore di una variabile.
3. Il monitor di un oggetto può essere identificato con la parte sincronizzata dell'oggetto stesso.
4. Affinché due thread che eseguono lo stesso codice e condividono gli stessi dati non abbiano problemi di concorrenza, è necessario sincronizzare il codice comune.
5. Si dice che un thread ha il lock di un oggetto se entra nel suo monitor.
6. I metodi `wait`, `notify` e `notifyAll` rappresentano il principale strumento per far comunicare più thread.
7. I metodi `suspend` e `resume` sono attualmente deprecati.
8. Il metodo `notifyAll`, invocato su di un certo oggetto `o1`, risveglia dallo stato di pausa tutti i thread che hanno invocato `wait` sullo stesso oggetto. Tra questi verrà eseguito quello che era stato fatto partire per primo con il metodo `start`.
9. Il deadlock è una condizione di errore bloccante generata da due thread che stanno in reciproca dipendenza in due oggetti sincronizzati.

10. Se un thread `t1` esegue il metodo `run` nell'oggetto `o1` della classe `C1`, e un thread `t2` esegue il metodo `run` nell'oggetto `o2` della stessa classe `C1`, la parola chiave `synchronized` non serve a niente.



Esercizio 12.c)

Simulare con del codice funzionante la seguente situazione con quanto appreso in questo capitolo. Un gruppo di 10 persone è all'osservatorio astronomico per ammirare il passaggio di una cometa sfruttando il potente telescopio messo a disposizione dalla struttura. Solo una persona alla volta può usare il telescopio, e i responsabili concedono solo tre minuti a persona per completare l'osservazione. Non esiste una fila, i partecipanti accederanno al telescopio in maniera casuale. Ogni partecipante quindi attraverserà degli stati:

- lo stato "In attesa" quando starà aspettando il suo turno. Dura un tempo indefinito, che dipende da quando inizierà il turno del partecipante;
- lo stato "Osservazione" quando starà osservando la cometa tramite il telescopio. Dura esattamente 3 minuti (ma è possibile abbreviare questo tempo per eseguire l'esercizio);
- lo stato "Finito" quando il turno è finito.

Gli stati possono essere caratterizzati da delle stampe significative.



Esercizio 12.d)

Simulare con del codice funzionante la seguente situazione con quanto appreso in questo capitolo. Supponiamo di trovarci allo sportello del comune che rilascia le carte d'identità. Supponiamo che ci siano 10 richiedenti in fila pronti per richiedere il documento. Quando arriva il proprio turno, sarà consegnato al richiedente un modulo da compilare. Per non bloccare la fila, il richiedente successivo potrà nel frattempo richiedere il servizio allo stesso sportello. Quindi a tutti i richiedenti sarà consegnato il modulo da compilare, la fila sarà molto veloce, e in parallelo diverse persone saranno occupate a compilare il modulo. Ogni richiedente potrebbe metterci un tempo variabile da 5 a 10 secondi per compilare il modello, il primo che finirà (indipendentemente dalla sua posizione nella fila iniziale) potrà richiedere la stampa della propria carta d'identità. Questa sarà stampata in 3 secondi in ogni caso.

Anche in questo caso usare delle stampe significative per rendere l'esecuzione del programma auto esplicativa.

Esercizio 12.e)

Consideriamo le seguenti classi:

```
import java.util.ArrayList;
```

```
class RunnableArrayList extends ArrayList implements Runnable {
```

```
    public void run(String stringa) {  
        System.out.println("Nel metodo run(): " + stringa);  
    }  
}  
  
public class Esercizio12E {  
    public static void main(String args[]) {  
        RunnableArrayList g = new RunnableArrayList();  
        Thread t = new Thread(g);  
        t.start();  
    }  
}
```

Se eseguiamo la classe `Esercizio12E`, quale sarà l'output?

1. Nel metodo `run()`: null
2. Nel metodo `run()`:
3. Il codice viene interrotto con un'eccezione nella classe `Esercizio12E`.
4. Il codice viene interrotto con un'eccezione nella classe `RunnableArrayList`.
5. Nessun output. Il codice non compila per un errore nella classe `Esercizio12E`.
6. Nessun output. Il codice non compila per un errore nella classe `RunnableArrayList`.

Esercizio 12.f)

Quali delle seguenti affermazioni sono corrette:

1. Una classe, per creare oggetti che abbiano un monitor, deve estendere `Thread`.
2. Una classe, per creare oggetti che abbiano un monitor, deve implementare `Runnable`.
3. Quando "un thread entra nel monitor dell'oggetto" allora possiede il lock di tale oggetto. Questo significa che nessun altro thread può entrare nel monitor di quell'oggetto.
4. La classe `Monitor` è definita dai metodi sincronizzati di una classe.

Esercizio 12.g)

Quali delle seguenti affermazioni sono corrette?

1. La classe `Thread` estende `Runnable`.
2. `Runnable` è una functional interface.
3. La classe `Thread` definisce un metodo `run` vuoto.
4. La classe `Thread` definisce i metodi `wait` e `notify`.

Esercizio 12.h)

Creare una classe `ContoAllaRovescia` che una volta attivata fa partire un conto alla rovescia da 10 a 0 prima di terminare.

Esercizio 12.i)

Quali delle seguenti affermazioni sono corrette:

1. Per istanziare un thread basta usare i suoi costruttori.
2. Specificare la priorità dei thread non rappresenta un modo efficace per decidere l'ordine di esecuzione di più thread.
3. Per eseguire il metodo `run` definito in una classe che estende `Thread` basta chiamare il suo metodo `run`.
4. Per eseguire un thread bisogna invocare il metodo `start` della classe `Object`.
5. In ogni programma ci sono almeno tre thread in esecuzione.



Esercizio 12.l)

Creare un programma che simuli la seguente situazione. Tramite comandi interattivi (`corri`, `cammina`, `fermati` e `basta`) letti dalla classe `Scanner`, l'utente reciterà la parte dell'allenatore di un corridore virtuale, dando dei comandi al volo mentre il corridore si allena.

Creare quindi una classe `Corridore` che estende `Thread`. Essa deve dichiarare i metodi: `corri` che gli permetterà di simulare una corsa, `cammina` che gli permetterà di simulare una camminata, `fermati` che gli permetterà di simulare l'azione del fermarsi, e `basta` che farà terminare l'allenamento (e il programma).

Suggerimento: possiamo sfruttare variabili booleane per gestire un ciclo infinito che gestisce il ciclo dell'allenamento.

Esercizio 12.m)

È meglio implementare `Runnable` o estendere la classe `Thread`? Scegliere tutte le affermazioni corrette.

1. Implementando `Runnable` che è un'interfaccia, possiamo anche estendere altre classi.
2. Estendendo la classe `Thread` possiamo anche implementare altre interfacce.
3. Dal punto di vista dell'astrazione dei dati, un oggetto di tipo `Thread` non dovrebbe possedere variabili private che rappresentano i dati da gestire.
4. Un oggetto `Runnable` che implementa il metodo `run`, può essere passato in input al costruttore di un oggetto `Thread`. Se viene invocato su quest'ultimo il metodo `start`, viene attivato il thread che esegue il metodo `run`.

Esercizio 12.n)

Quali delle seguenti affermazioni sono corrette?

1. Con il time slicing o round-robin scheduling un thread può trovarsi in esecuzione solo per un certo periodo di tempo.
2. Il time slicing o round-robin scheduling è il comportamento di default della maggior parte dei sistemi Linux.
3. Il preemptive scheduling è il comportamento di default della maggior parte dei sistemi Linux.

4. Con il preemptive scheduling la priorità è un elemento più deterministico rispetto al caso del round robin. Infatti lanciati due thread in contemporanea, il thread a maggiore priorità entrerà nello stato di esecuzione, e ne uscirà solo quando avrà terminato il suo lavoro, oppure nel caso sia chiamato su di esso un metodo come `wait` o `suspend`, oppure si metta in attesa di risorse esterne (come nel caso di attesa di risorse di input-output).

Esercizio 12.o)

Quali delle seguenti affermazioni sono corrette?

1. Il modificatore `volatile` si può usare solo su metodi e variabili.
2. Dichiarare `volatile` una variabile d'istanza, implica che saranno considerate `volatile` anche tutte le altre variabili della stessa istanza che vengono usate dal medesimo thread.
3. Per una variabile `volatile`, tutti gli accessi in lettura e scrittura sono atomici.
4. Per una variabile intera non `volatile`, tutti gli accessi in lettura e scrittura sono atomici.

Esercizio 12.p)

Quali delle seguenti affermazioni sono corrette?

1. La parola chiave `synchronized` permette di dichiarare metodi atomici.
2. Il monitor di un oggetto è costituito dalla parte dell'oggetto sincronizzata. Quindi se in un certo istante, un thread sta eseguendo codice all'interno di uno dei metodi sincronizzati di tale oggetto, qualsiasi altro thread che vuole accedere al codice di uno qualsiasi dei metodi sincronizzati dello stesso oggetto, dovrà attendere che il primo thread termini di eseguire codice sincronizzato.
3. È possibile utilizzare blocchi sincronizzati per rendere sincronizzati solo determinate righe di codice. Questo rappresenta una soluzione più flessibile della sincronizzazione.
4. È possibile utilizzare `synchronized` come modificatore solo con i metodi.
5. È possibile dichiarare `synchronized` il metodo `run`.

Esercizio 12.q)

Quali delle seguenti affermazioni sono corrette?

1. I metodi `wait`, `notify` e `notifyAll` sono definiti nella classe `Thread`.
2. Se un thread incontra il metodo `wait` abbandona il monitor dell'oggetto di cui stava eseguendo codice.
3. Il metodo `notifyAll` fa ripartire tutti i thread che avevano eseguito il metodo `wait`.
4. I metodi `suspend` e `resume` sono definiti nella classe `Thread`.

Esercizio 12.r)

Rendere immutabile la seguente classe:

```
import java.util.Date;

public class Esercizio12RNonImmutabile {
```

```
private Integer intero;

private Date date;

public void setDate(Date date) {
    this.date = date;
}

public Date getDate() {
    return date;
}

public void setIntero(Integer intero) {
    this.intero = intero;
}

public Integer getIntero() {
    return intero;
}
}
```

Esercizio 12.s)

Quali delle seguenti affermazioni sono corrette riguardo i package

`java.util.concurrent.atomic` e `java.util.concurrent.locks`?

1. `ReentrantLock` è una classe che, se istanziata, può sostituire l'utilizzo di un blocco sincronizzato. Però bisogna utilizzare obbligatoriamente un blocco `try-catch-finally`.
2. Il concetto di *fairness* degli oggetti di tipo `ReentrantLock` permette alla JVM di creare una gerarchia di priorità di esecuzione dei thread, basata sul tempo di creazione del `ReentrantLock`.
3. Un oggetto di tipo `Lock` può specificare un timeout per uscire da un blocco sincronizzato.
4. L'interfaccia `AtomicInteger` definisce metodi atomici che compiono più di un'operazione.

Esercizio 12.t)

Considerato il seguente snippet:

```
Callable<String> callable = new Callable<>() {public void call(){} };
Future<String> future = Executors.newFixedThreadPool(3).start(callable);
String result = future.get();
```

Che messaggio restituirà il compilatore?

Esercizio 12.u)

Realizzare una sveglia con le classi `Timer` e `TimerTask` del package `java.util`. In particolare deve essere possibile passare da riga di comando un numero che rappresenterà il numero di secondi che devono passare affinché “suoni” la sveglia.

Esercizio 12.v)

Quali delle seguenti affermazioni sono corrette?

1. Lo statement:
`new Semaphore().acquire();`
permette ad un semaforo di acquisire il lock su un oggetto.
2. Il metodo `tryAcquire` permette di specificare un timeout.
3. `Semaphore` è un'interfaccia.
4. I *permits* rappresentano una variabile d'istanza di `Semaphore`.

Esercizio 12.z)

Quali delle seguenti affermazioni sono corrette?

1. Istanziato un oggetto `cyclicBarrier` di tipo `CyclicBarrier`, lo statement:
`cyclicBarrier.await;`
equivale a chiamare il metodo `wait` sull'oggetto che legge questa istruzione.
2. Il costruttore di `CyclicBarrier` permette di specificare il numero di thread che devono "accumularsi" in un certo punto di codice prima di essere rilasciati.
3. Il metodo `signalAll` di `CyclicBarrier` è equivalente al metodo `notifyAll` di `Object`.
4. L'utilizzo di `CyclicBarrier` può sempre sostituire l'utilizzo di `wait`, `notify` e `notifyAll`.

Esercizi del capitolo 13 e dell'appendice F

Espressioni Lambda e package java.lang

Le espressioni lambda e i reference a metodi non rappresentano un argomento semplice, ma sono indubbiamente una soluzione molto utile a certi problemi di programmazione. I seguenti esercizi dovrebbero permettere al lettore di comprendere meglio gli argomenti trattati nel capitolo 13. In questa sezione inoltre troverete anche esercizi riguardanti la libreria `java.lang` così come descritta nell'appendice F. Tali esercizi hanno una numerazione diversa che inizia con la lettera F. Anche in questo capitolo sono stati introdotti diversi esercizi che supportano la certificazione Oracle.

Esercizio 13.a) *Espressioni lambda e reference a metodi:*

1. Con un'espressione lambda è possibile fare tutto quello che si può fare con una classe anonima.
2. Un'espressione lambda può utilizzare in maniera thread-safe le variabili d'istanza della classe in cui è dichiarata.
3. Un'espressione lambda può utilizzare le variabili d'istanza della classe in cui è definita solo se dichiarate `final`.
4. La seguente espressione lambda è legale:

```
Consumer <String> c = ((x)->System.out.println(x));
```
5. La seguente espressione lambda è legale:

```
(x, y) -> System.out.println(x);  
        System.out.println(y);
```
6. Il seguente reference a metodo è legale:

```
System.out::println()
```
7. Il seguente statement è legale:

```
System.out.println(Math::random)
```
8. Il seguente statement è legale:

```
System.out::println(Math::random)
```
9. Il seguente statement è legale:

```
Supplier<Chitarra> chitarraSupplier = Chitarra::new;
```
10. Supponendo che la classe `Chitarra` abbia un costruttore senza parametri, allora il seguente codice è legale:

```
Chitarrista chitarrista = new Chitarrista();  
chitarrista.suonaChitarra(Chitarra::new);
```



Esercizio 13.b)

Creare una classe `TestComparators` contenente un metodo `main` che dichiari un array di stringhe (di cui almeno due con la stessa lunghezza). Creare degli oggetti `Comparator` con espressioni lambda, per produrre ordinamenti secondo i seguenti criteri:

- in ordine di lunghezza (dalla stringa più lunga alla più breve);

- in ordine di lunghezza al contrario (dalla stringa più breve alla più lunga);
- in ordine alfabetico (usare comunque un'espressione lambda o un reference a metodo, anche se non ce ne sarebbe bisogno);
- in ordine alfabetico inverso;
- in ordine di lunghezza e in caso di stessa lunghezza in ordine alfabetico.

Ordinare l'array mediante il metodo `sort` della classe `Arrays`, che prende in input come primo parametro l'array dichiarato e come secondo parametro un oggetto di tipo `Comparator`. Dopo ogni ordinamento stampare il risultato. Creare questa classe in un package come per esempio `com.claudiodesio.lambda.test`.

Esercizio 13.c)

Creare una classe `Città` (non è possibile utilizzare le lettere accentate negli identificatori in Java), che astrae il concetto di città, che dichiara come variabili incapsulate la stringa `nome`, e i booleani `capoluogo`, e `diMare`.

Si ricorda che i metodi getter per i booleani solitamente usano come prefisso “is” in luogo di “get”. Quindi i metodi `getCapoluogo` e `getDiMare`, si dovrebbero scrivere come `isDiMare` e `isCapoluogo`.

Creare anche eventuali metodi di utilità come `toString` (affinché ritorni solo il nome) e un costruttore. Questa classe apparterrà ad un package come per esempio `com.claudiodesio.lambda.dat`.

Creare una classe `Esercizio13C` con un metodo `main`, che stampi:

- la lista di città di mare (per esempio Città di mare: [Siracusa, Napoli, Pescara, Taranto])
- la lista di città capoluogo (per esempio Città capoluogo: [Milano, Potenza, Perugia, Napoli])

sfruttando i metodi a reference e l'interfaccia `Predicate`.

Questa classe apparterrà ad un package come per esempio `com.claudiodesio.lambda.test`.

Esercizio 13.d)

Dopo aver svolto l'esercizio precedente, creare una classe `Esercizio13D` alternativa a `Esercizio13C`, che esegue le stesse operazioni usando le espressioni lambda.

Esercizio 13.e)

Creare la classe `Esercizio13E` modificando la classe `Esercizio13C` aggiungendo un metodo `stampaDettagli` che stampi la lista delle città, anche con le informazioni definite dalle variabili `capoluogo` e `diMare`. Per esempio l'output potrebbe essere simile al seguente:

```
Milano è capoluogo,  
Pescara è città di mare,  
Napoli è capoluogo, è città di mare,
```

...

Per ottenere tale risultato, non bisogna cambiare la classe `Citta`, ma usare un'espressione lambda in `Esercizio13E`, sfruttando una delle interfacce funzionali standard, quale?



Esercizio 13.f)

Supponiamo di voler scrivere un programma e di voler riutilizzare la seguente classe `Persona`, ereditata da un programma già scritto e non modificabile:

```
public class Persona {  
  
    private String nome;  
    private String cognome;  
    private String dataDiNascita;  
    private String professione;  
    private String indirizzo;  
  
    public Persona(String nome, String cognome) {  
        this.nome = nome;  
        this.cognome = cognome;  
    }  
  
    public Persona(String nome, String cognome, String dataDiNascita,  
        String professione, String indirizzo) {  
        this.nome = nome;  
        this.cognome = cognome;  
        this.dataDiNascita = dataDiNascita;  
        this.professione = professione;  
        this.indirizzo = indirizzo;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    public String getCognome() {  
        return cognome;  
    }  
  
    public void setCognome(String cognome) {  
        this.cognome = cognome;  
    }  
  
    public String getDataDiNascita() {  
        return dataDiNascita;  
    }  
  
    public void setDataDiNascita(String dataDiNascita) {
```

```
        this.dataDiNascita = dataDiNascita;
    }

    public String getProfessione() {
        return professione;
    }

    public void setProfessione(String professione) {
        this.professione = professione;
    }

    public String getIndirizzo() {
        return indirizzo;
    }

    public void setIndirizzo(String indirizzo) {
        this.indirizzo = indirizzo;
    }

    @Override
    public String toString() {
        return "Persona{" + "nome=" + nome + ", cognome=" + cognome + '}';
    }
}
```

Purtroppo nel nostro contesto, avremmo bisogno di ridefinire il metodo `toString` in modo che stampi non solo le informazioni sul nome e il cognome della persona, ma anche la data di nascita, l'indirizzo e la professione. Come già detto però, la classe è già in uso e non è possibile modificarla. In particolare il nostro requisito è che il metodo `toString` restituisca una stringa con il seguente formato:

```
Nome:           Arjen Anthony
Cognome:        Lucassen
Professione:    Compositore
Data di Nascita: 03/04/1960
Indirizzo:      Olanda
```

Si crei quindi una classe `TestPersona` che ridefinisca in qualche modo il metodo `toString` della classe `Persona` e stampi l'output di cui sopra.

Per la formattazione è possibile sfruttare il carattere di escape `\t` introdotto nel capitolo 3.

Esercizio 13.g)

Consideriamo la classe `Osservazione` della soluzione dell'esercizio 12.c, che riportiamo di seguito per comodità:

```
package com.claudiodesio.osservatorio.test;

import com.claudiodesio.osservatorio.dat.Partecipante;
import com.claudiodesio.osservatorio.dat.Telescopio;

public class Osservazione {

    public static void main(String args[]) {
        Telescopio telescopio = new Telescopio();
        Partecipante[] partecipanti = getPartecipanti(telescopio);
    }
}
```

```
        for (Partecipante partecipante : partecipanti) {
            partecipante.start();
        }
    }

    private static Partecipante[] getPartecipanti(Telescopio telescopio) {
        Partecipante[] partecipanti = {
            new Partecipante("Ciro", telescopio),
            new Partecipante("Gianluca", telescopio),
            new Partecipante("Pierluigi", telescopio),
            new Partecipante("Gigi", telescopio),
            new Partecipante("Nicola", telescopio),
            new Partecipante("Pino", telescopio),
            new Partecipante("Maurizio", telescopio),
            new Partecipante("Raffaele", telescopio),
            new Partecipante("Fabio", telescopio),
            new Partecipante("Vincenzo", telescopio)};
        return partecipanti;
    }
}
```

È possibile usare un'espressione lambda per implementare un override del metodo `run` della classe `Partecipante`, per uno dei partecipanti?

Esercizio 13.h)

Considerato il seguente codice:

```
new Thread(()->System.out.print("Java"));
System.out.print("Java");).start();
```

quali delle seguenti affermazioni sono corrette?

1. Eseguendo questo snippet con JShell viene stampata la stringa `JavaJava`.
2. Eseguendo questo snippet con JShell viene stampata la stringa `Java`.
3. Questo snippet non supererà la compilazione.
4. Questo snippet non lancerà un'eccezione al runtime.
5. Questo snippet produrrà un warning in compilazione.

Esercizio 13.i)

Considerato il seguente codice:

```
new Thread((int a)->{
    int b = 0;
    b = a/b;
}).start();
```

quali delle seguenti affermazioni sono corrette?

1. Il codice non compila perché bisogna gestire una `ArithmeticException`.
2. Il codice compila e viene eseguito correttamente.
3. Il codice compila ma lancerà una `ArithmeticException` durante l'esecuzione.
4. Il codice produrrà un warning in compilazione.

Esercizio 13.j) Tipi innestati, Vero o Falso:

1. Una classe innestata è una classe che viene dichiarata all'interno di un'altra classe.
2. Una classe anonima è anche innestata, ma non ha nome. Inoltre, per essere dichiarata, deve per forza essere istanziata.
3. Le classi innestate non sono necessarie per l'Object Orientation.
4. Una classe innestata deve essere per forza istanziata.
5. Per istanziare una classe innestata pubblica a volte bisogna istanziare prima la classe esterna.
6. Una classe innestata dichiarata `private` deve dichiarare anche i metodi "set" e "get" per poter essere utilizzata da una terza classe.
7. Una classe innestata non può avere lo stesso nome della classe che la contiene.
8. Una classe anonima può avere lo stesso nome della classe che la contiene.
9. Una classe innestata può accedere a membri statici della classe che la contiene solo se è dichiarata statica.
10. Una classe innestata non può essere dichiarata astratta.

Esercizio 13.k)

Data la seguente classe:

```
public class Esterna {  
    public int a = 1;  
    private int b = 2;  
  
    public void metodoEsterno(final int c) {  
        int d = 4;  
        class Interna {  
            private void metodoInterno(int e) {  
  
            }  
        }  
    }  
}
```

Quali delle seguenti affermazioni sono corrette?

1. All'interno del metodo `metodoInterno` è possibile referenziare la variabile `a`.
2. All'interno del metodo `metodoInterno` è possibile referenziare la variabile `b`.
3. All'interno del metodo `metodoInterno` è possibile referenziare la variabile `c`.
4. All'interno del metodo `metodoInterno` è possibile referenziare la variabile `d`.
5. All'interno del metodo `metodoInterno` è possibile referenziare la variabile `e`.
6. La classe non può essere compilata.

Esercizio 13.l)

Supponiamo di avere un'espressione lambda che usa un metodo che lancia una checked exception senza gestirla. Quali delle seguenti affermazioni è corretta?

1. Il codice non compila perché bisogna gestire una checked exception nel blocco di codice dell'espressione lambda.
2. Il codice non compila, ma se è possibile, possiamo renderlo compilabile ridefinendo il metodo SAM dell'interfaccia funzionale gestendo l'eccezione all'interno del metodo stesso.
3. Il codice compila tranquillamente.

4. Dal momento che dobbiamo gestire l’eccezione, perdiamo le capacità di deduzione del compilatore, che ci porta a scrivere meno codice.



Esercizio 13.m)

Creare l’interfaccia funzionale che possa soddisfare il seguente uso di espressioni lambda:

```
Operazione operazione1 = (double a, double b) -> a + b;  
Operazione operazione2 = (double a, double b) -> a - b;  
Operazione operazione3 = (double a, double b) -> a / b;  
Operazione operazione4 = (double a, double b) -> a * b;
```

Esercizio 13.n)

Esiste un’interfaccia funzionale standard, capace di sostituire l’interfaccia funzionale `Operazione` dell’esercizio precedente?

Usare la documentazione per risolvere l’esercizio.

Esercizio 13.o)

Riscrivere la soluzione dell’esercizio 13.b, sostituendo le espressioni lambda con reference a metodi.

Esercizio 13.p)

Come si chiama l’interfaccia funzionale che si adatta meglio a fungere da “Factory”?

1. Predicate
2. Factory
3. Function
4. Supplier
5. Consumer

Esercizio 13.q)

Selezionare le affermazioni corrette.

Un’interfaccia funzionale:

1. Deve essere annotata con l’annotazione `FunctionalInterface`.
2. Deve dichiarare un unico metodo.
3. Deve essere per forza implementata.
4. Non può essere estesa da un’altra interfaccia funzionale.
5. Non può essere estesa da un’altra interfaccia.

Esercizio 13.r)

Quali delle seguenti affermazioni sono corrette riguardo i reference a costruttore?

1. La sintassi è `NomeClasse::New`.
2. La sintassi è `NomeClasse::new()`.
3. La sintassi è `NomeClasse::new`.
4. Un reference a un costruttore si può assegnare ad un reference di un'interfaccia funzionale, il cui metodo SAM ritorna `void`.
5. Con un reference ad un costruttore, possiamo sostituire l'implementazione di un'interfaccia funzionale.

Esercizio 13.s)

Creare una classe `Persona` che dichiari le variabili `nome` e `anni` (nel senso di anni di età) e che implementi l'interfaccia `Comparable` in modo tale che il metodo ereditato `compareTo` ordini per età crescente, e, nel caso di persone della stessa età, in ordine alfabetico (considerando il nome). Si faccia override anche del metodo `toString`.

Poi, considerata la seguente classe:

```
import java.util.Arrays;

public class Esercizio13S {
    public static void main(String args[]) {
        Persona [] persone = {
            new Persona("Antonio",21),
            new Persona("Bruno",20),
            new Persona("Giorgio",19),
            new Persona("Martino",22),
            new Persona("Daniele",21)
        };
        Arrays.sort(persone, /*INSERISCI IL CODICE QUI*/);
        System.out.println(Arrays.toString(persone));
    }
}
```

si inserisca il codice corretto al posto del commento `/*INSERISCI IL CODICE QUI*/`, in modo tale da generare il seguente output:

```
[Giorgio, Bruno, Antonio, Daniele, Martino]
```



Esercizio 13.t)

Quale interfaccia funzionale potrebbe essere utilizzata al posto del seguente metodo?

```
boolean haQuestoNome(Persona persona, String nome) {
    return nome.equals(persona.getNome());
}
```

Esercizio 13.u)

Partendo dalla classe `Persona` dell'esercizio 13.s, e considerando la seguente classe:

```
import java.util.Arrays;
import java.util.function.BiPredicate;

public class Esercizio13U {
    public static void main(String args[]) {
        Persona [] persone = {
            new Persona("Antonio",21),
            new Persona("Bruno",20),
            new Persona("Giorgio",19),
            new Persona("Martino",22),
            new Persona("Daniele",21)
        };
        Persona personaCheIniziaPerD = getPersonaCheIniziaPer("D",
            persone, /*INSERISCI CODICE QUI*/);
        System.out.println(personaCheIniziaPerD);
    }

    static Persona getPersonaCheIniziaPer(String iniziale,
        Persona[] persone, BiPredicate<String, Persona> biPredicate) {
        for(Persona persona : persone) {
            if (biPredicate.test(iniziale, persona)) {
                return persona;
            }
        }
        return null;
    }
}
```

Quale espressione lambda è possibile inserire al posto del commento `/*INSERISCI CODICE QUI*/` per recuperare il primo oggetto `Persona` dell'array `persone` che ha un nome che inizia con la "D"?



Esercizio 13.v)

Quali delle seguenti affermazioni sono corrette?

1. Con le espressioni lambda il reference `this` si riferisce direttamente alla classe in cui è inclusa l'espressione.
2. Per le espressioni lambda valgono le stesse regole che valgono per le classi anonime.
3. La sintassi di un'espressione lambda permette di utilizzare altre espressioni lambda innestate.
4. La sintassi di un'espressione lambda permette di utilizzare reference a metodi innestati.
5. La sintassi di un reference a metodo permette di utilizzare altre espressioni lambda innestate.
6. La sintassi di un reference a metodo permette di utilizzare reference a metodi innestati.

Esercizio 13.w)

Ripetere l'esercizio 13.f, utilizzando una espressione lambda in luogo della classe anonima. Riportiamo il testo dell'esercizio 13.f per comodità qui di seguito.

Supponiamo di voler scrivere un programma e di voler riutilizzare la seguente classe `Persona`, ereditata da un programma già scritto e non modificabile:

```
public class Persona {  
  
    private String nome;  
    private String cognome;  
    private String dataDiNascita;  
    private String professione;  
    private String indirizzo;  
  
    public Persona(String nome, String cognome) {  
        this.nome = nome;  
        this.cognome = cognome;  
    }  
  
    public Persona(String nome, String cognome, String dataDiNascita,  
        String professione, String indirizzo) {  
        this.nome = nome;  
        this.cognome = cognome;  
        this.dataDiNascita = dataDiNascita;  
        this.professione = professione;  
        this.indirizzo = indirizzo;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    public String getCognome() {  
        return cognome;  
    }  
  
    public void setCognome(String cognome) {  
        this.cognome = cognome;  
    }  
  
    public String getDataDiNascita() {  
        return dataDiNascita;  
    }  
  
    public void setDataDiNascita(String dataDiNascita) {  
        this.dataDiNascita = dataDiNascita;  
    }  
  
    public String getProfessione() {  
        return professione;  
    }  
  
    public void setProfessione(String professione) {  
        this.professione = professione;  
    }  
  
    public String getIndirizzo() {  
        return indirizzo;  
    }  
}
```

```
public void setIndirizzo(String indirizzo) {
    this.indirizzo = indirizzo;
}

@Override
public String toString() {
    return "Persona{nome=" + nome + ", cognome=" + cognome + "}";
}
}
```

Purtroppo nel nostro contesto, avremmo bisogno di ridefinire il metodo `toString` in modo che stampi non solo le informazioni sul nome e il cognome della persona, ma anche la data di nascita, l'indirizzo e la professione. Come già detto però, la classe è già in uso e non è possibile modificarla. In particolare il nostro requisito è che il metodo `toString` restituisca la seguente stringa:

```
Nome:           Arjen Anthony
Cognome:        Lucassen
Professione:    Compositore
Data di Nascita: 03/04/1960
Indirizzo:      Olanda
```

Creare quindi una classe `TestPersona` che ridefinisca in qualche modo il metodo `toString` della classe `Persona` e stampi l'output di cui sopra.



Esercizio 13.x)

Data la seguente classe:

```
public class Esterna {
    private int intero = 5;

    public static void main(String[] args) {
        Esterna.Interna interna = new Esterna().new Interna();
        interna.metodoInterno();
    }

    private class Interna {
        private int intero = 10;
        protected void metodoInterno() {
            System.out.println(super.intero);
        }
    }
}
```

Quali delle seguenti affermazioni sono vere?

1. Eseguita la classe `Esterna`, viene lanciata una `NullPointerException`.
2. Eseguita la classe `Esterna`, viene stampato 50.
3. Eseguita la classe `Esterna`, viene stampato 10.
4. Eseguita la classe `Interna`, viene stampato 5.
5. Eseguita la classe `Interna`, viene stampato 0.
6. Impossibile compilare questo codice.

Esercizio 13.y)

Tenendo conto della risposta dell'esercizio precedente, modificare il codice (con una modifica minimale) in modo tale da far stampare il valore 5. Poi modificarlo nuovamente per fargli stampare anche il valore 10.

Esercizio 13.z)

Quali delle seguenti affermazioni sono corrette?

1. L'interfaccia funzionale che dichiara due tipi generici come parametri in input e ne ritorna un altro, si chiama `BiFunction`.
2. L'interfaccia funzionale che non dichiara tipi generici come parametri in input, ma ne ritorna un altro, si chiama `Provider`.
3. L'interfaccia funzionale che non dichiara tipi generici come parametri in input, ma ne ritorna un altro, si chiama `Supplier`.
4. L'interfaccia funzionale che dichiara un tipo generico come parametro in input e ne ritorna un altro, si chiama `Function`.
5. L'interfaccia funzionale che dichiara tre tipi generici come parametri in input e ne ritorna un altro, si chiama `TriFunction`.
6. L'interfaccia funzionale che dichiara un tipo generico come parametro in input e ritorna lo stesso tipo, si chiama `UnaryOperator`.
7. È possibile concatenare più `UnaryOperator` tramite il metodo di default `and`.
8. L'interfaccia funzionale che dichiara un tipo generico come parametro in input e non ritorna nulla, si chiama `Consumer`.

Esercizio 13.aa)

Data la seguente classe:

```
public class Esercizio13AA {  
  
    public static void main(String args[]) {  
        new Interfaccia() {  
            public void metodo() {  
                System.out.println("Classe anonima");  
            }  
        }.metodo();  
    }  
  
    private interface Interfaccia {  
        void metodo();  
    }  
}
```

Quali delle seguenti affermazioni è vera?

1. Eseguita la classe `Esercizio13AA`, viene lanciata una `NullPointerException`.
2. Eseguita la classe `Esercizio13AA`, viene lanciata una `CannotInstantiateInterfaceException`.
3. Eseguita la classe `Esercizio13AA`, viene stampato "Classe Anonima".
4. Eseguita la classe `Esercizio13AA`, viene stampato "Null".
5. Impossibile compilare questo codice.

Esercizio 13.bb)

Data la seguente classe:

```
public class Esterna {
    private class Interna {
        private static int effectivelyFinalVariable = 10;

        Interna() {
            effectivelyFinalVariable = 11;
        }

        protected void metodo () {
            System.out.println(effectivelyFinalVariable);
        }
    }
}
```

Quali delle seguenti affermazioni sono vere?

1. Impossibile compilare questo codice.
2. La variabile `effectivelyFinalVariable` non è “effettivamente final”.
3. La variabile `effectivelyFinalVariable` non ha importanza che sia o meno “effettivamente final” perché è una variabile d’istanza.
4. La variabile `effectivelyFinalVariable` non ha importanza che sia o meno “effettivamente final” perché appartiene alla classe interna e non alla classe esterna.

Esercizio 13.cc)

Data la seguente classe:

```
public class Esterna {
    private final static String stringa = "Classe Innestata";

    protected Esterna() {
        private static class Innestata {
            protected void metodo() {
                System.out.println(stringa);
            }
        }
    }
}
```

Quali delle seguenti affermazioni sono vere?

1. Impossibile compilare questo codice.
2. La costante statica `stringa` non è “effettivamente final” perché è anche statica, e quindi non può essere utilizzata all’interno della classe interna `Innestata`.
3. La costante statica `stringa` non è accessibile al metodo `metodo` perché dichiarata statica.
4. La classe `Innestata`, essendo dichiarata all’interno di un costruttore `protected`, può essere utilizzata solo all’interno del costruttore.



Esercizio 13.dd)

Dato questo codice:

```
public enum Esercizio13DD {  
    A, B, C;  
    public class Interna {  
        public enum EnumInterna {  
            D, E, F;  
        }  
    }  
    public static void main(String args[]) {  
        for (Esercizio13DD.Interna.EnumInterna item :  
            Esercizio13DD.Interna.EnumInterna.values()) {  
            System.out.println(item);  
        }  
    }  
}
```

Una volta eseguita l'enumerazione `Esercizio13DD`, l'output sarà:

1. A, B, C
2. D, E, F
3. A B C
4. D E F
5. ABC
6. DEF
7. Nessuna delle precedenti: viene lanciata una `NullPointerException`.
8. Nessuna delle precedenti: il codice non compila per un errore nel metodo `main`.
9. Nessuna delle precedenti: il codice non compila per un errore nella classe `Interna`.

La risposta a questo esercizio dipende dalla versione di Java che stiamo utilizzando (vedi nota nella risposta).

Esercizio 13.ee)

(ALLERTA SPOILER: le prossime righe rivelano la soluzione del precedente esercizio).

Dopo aver letto la soluzione dell'esercizio precedente (in particolare l'output della compilazione), si modifichi il codice per renderlo compilabile, risolvendo i vari errori che si presentano dopo le varie compilazioni. Eseguendo la versione compilabile definitiva il programma dovrà stampare il seguente output:

```
D  
E  
F
```

Questo esercizio testa la vostra capacità di interpretare i messaggi di errore del compilatore, e risolverli. Praticamente quello che ogni programmatore fa in continuazione.

Esercizio 13.ff)

Cosa dobbiamo utilizzare per far eseguire algoritmi diversi (anche parziali) ad un metodo (scegliere tutte le risposte corrette) senza aver prima creato il codice dell'algoritmo?

1. Un oggetto istanziato da una classe già esistente.
2. Un'interfaccia.
3. Un'enumerazione.
4. Una classe interna.
5. Una classe innestata.
6. Una classe anonima.
7. Un costrutto `switch`.
8. Niente, semplicemente non è possibile.

per tutti gli esercizi di questo libro, è possibile consultare la documentazione della libreria standard per trovare le soluzioni. Questo vale soprattutto nei capitoli come questi dedicati proprio alle librerie.

Esercizio F.a) Autoboxing, autounboxing e java.lang, Vero o Falso:

1. Il seguente codice compila senza errori:
`char c = new String("Pippo");`
2. Il seguente codice compila senza errori:
`int c = new Integer(1) + 1 + new Character('a');`
3. Le regole dell'overload non cambiano con l'introduzione dell'autoboxing e dell'autounboxing.
4. Le istanze della classe `Integer` sono immutabili, e quindi non è possibile mutare il loro stato interno una volta istanziate.
5. La classe `Runtime` dipende strettamente dal sistema operativo su cui gira.
6. La classe `Class` permette di leggere i membri di una classe (ma anche le superclassi ed altre informazioni) partendo semplicemente dal nome della classe grazie al metodo `forName`.
7. Tramite la classe `Class` è possibile istanziare oggetti di una certa classe conoscendone solo il nome.
8. È possibile dalla versione 1.4 di Java sommare un tipo primitivo e un oggetto della relativa classe wrapper, come nel seguente esempio:
`Integer a = new Integer(30);`
`int b = 1;`
`int c = a+b;`
9. La clonazione di oggetti richiede obbligatoriamente una chiamata al metodo `clone` di `Object`.
10. La classe `Math` non si può istanziare perché dichiarata `abstract`.

Esercizio F.b)

Quali delle seguenti affermazioni sono corrette?

1. Un oggetto immutabile non può essere modificato.
2. Le stringhe sono oggetti immutabili.
3. Le istanze delle classi wrapper sono oggetti immutabili.
4. Un oggetto immutabile può essere puntato da più reference.

5. Un oggetto immutabile non permette di modificare il suo stato interno.
6. Un oggetto immutabile non permette di modificare il suo stato esterno.

Esercizio F.c)

Quali tra le seguenti affermazioni sono corrette rispetto all'argomento delle compact strings?

1. È possibile specificare quali stringhe devono essere rese compatte.
2. Una stringa non compatta utilizza 16 bit.
3. Una stringa compatta utilizza solo 8 bit.
4. Per rendere in un programma tutte le stringhe compatte è necessario compilarlo utilizzando l'opzione `-XX:-CompactStrings`.
5. Utilizzando le compact string le performance del programma saranno sempre incrementate del 50%.

Esercizio F.d)

Qual è l'output del seguente file?

```
public class EsercizioFD {  
    public static void main(String args[]) {  
        String stringa = "**** Java ****";  
        stringa.toUpperCase();  
        stringa.trim();  
        stringa.substring(3, 8);  
        stringa.trim();  
        stringa.concat(String.format("Stringa = %n", stringa.length()));  
        stringa += "!";  
        System.out.println(stringa.length);  
    }  
}
```

1. 12
2. 13
3. 11
4. 10
5. 23
6. 22
7. 24
8. Nessun output, sarà lanciata un'eccezione al runtime.
9. Nessun output, il file non è compilabile.



Esercizio F.e)

Qual è l'output del seguente file?

```
public class EsercizioFE {  
    public static void main(String args[]) {  
        String stringa = "Java";  
        stringa = stringa.concat(" ");  
    }  
}
```

```
        stringa += 9;
        String risultato = "";
        if (stringa.intern() == "Java 9") {
            risultato += "intern()";
        }
        if (stringa == "Java 9") {
            risultato += "==";
        }
        if (stringa.equals("Java 9")) {
            risultato += "equals()";
        }
        System.out.println(risultato);
    }
}
```

1. Java 9
2. intern()
3. intern() ==
4. intern() equals()
5. null
6. intern() == equals()
7. Una stringa vuota.
8. Nessun output, sarà lanciata un'eccezione al runtime.
9. Nessun output, il file non è compilabile.



Esercizio F.f)

Qual è l'output del seguente file?

```
public class EsercizioFF {
    public static void main(String args[]) {
        String stringa1 = "123789";
        String stringa2 = stringa1.concat(System.lineSeparator());
        char [] array1 = stringa2.toCharArray();
        char [] array2 = {'4', '5', '6'};
        System.arraycopy(array2, 0, array1, 3, 3);
        System.out.println(array1);
        System.exit(0);
    }
}
```

Esercizio F.g)

Scrivere una classe che rappresenta un testo di tipo RTF (Rich Text Format), ovvero un testo a cui è possibile modificare per esempio il carattere, il colore di sfondo, l'interlinea, la sottolineatura e così via (scegliete voi cosa deve definire). Rendere tale classe clonabile e creare un programma di test che verifica l'effettivo funzionamento.

Esercizio F.h)

Partendo dall'esercizio 5.w, Creare una classe con metodo `main` che legge username e password, come properties di sistema. Testare la classe utilizzando le giuste opzioni da riga di comando (specificarle).

Esercizio F.i)

Quali delle seguenti affermazioni sono corrette riguardo la garbage collection:

1. La JVM implementa solo due algoritmi per deallocare la memoria: G1GC e ParallelGC.
2. Con ParallelGC gli interventi sulla memoria sono più numerosi e intensi, e questo rende l'algoritmo meno efficiente rispetto a G1GC.
3. Era possibile utilizzare G1GC già dalla versione 7 di Java.
4. La “finalizzazione” consiste nell'eliminare gli oggetti non più utilizzati dall'applicazione.
5. La “finalizzazione” può essere invocata mediante la chiamata del metodo `runFinalization` della classe `Object`.
6. Per utilizzare ParallelGC, è necessario lanciare l'applicazione con l'applicazione specificando l'opzione `-XX:+UseParallelGC`.

Esercizio F.j)

Data la seguente classe:

```
public class EsercizioFJ {  
  
    public static void main(String args[]) {  
        int raggio = 7;  
        /*INSERISCI CODICE QUI*/  
        System.out.println("L'area della circonferenza di raggio 7 è " + area);  
    }  
}
```

definire la riga che deve sostituire il commento `/*INSERISCI CODICE QUI*/` per ottenere il corretto calcolo dell'area.

Ricordiamo che l'area di una circonferenza viene calcolata come:

pi greco per raggio al quadrato, ovvero se chiamiamo il raggio r , e A l'area della circonferenza, e indichiamo il numero *pi greco* con π , avremo:

$$A = \pi r^2$$

Consultare la documentazione ufficiale della classe `Math` prima di scrivere il codice.

Esercizio F.k)

Qual è l'output del seguente file?

```
public class EsercizioFK {  
  
    public static void main(String args[]) {  
        double e = Math.E;  
        Math.floor(e);  
    }  
}
```

```
        boolean b = check(e, 2.0);
        System.out.println(b);
    }

    public static Boolean check(Double a, Double b){
        Boolean equals = null;
        if (a.equals(b)){
            equals = true;
        }
        return equals;
    }
}
```

1. true
2. false
3. null
4. 2.0
5. 2.718281828459045
6. Nessun output, sarà lanciata un'eccezione al runtime.
7. Nessun output, il file non è compilabile.

Esercizio F.1)

Qual è l'output della seguente classe?

```
public class EsercizioFL {
    public static void main(String args[]) {
        EsercizioFL e = new EsercizioFL();
        e.metodo(128);
    }

    public void metodo(Integer numero) {
        System.out.println("Integer " + numero);
    }

    public void metodo(long numero) {
        System.out.println("long " + numero);
    }

    public void metodo(byte numero) {
        System.out.println("byte " + numero);
    }

    public void metodo(Byte numero) {
        System.out.println("Byte " + numero);
    }

    public void metodo(short numero) {
        System.out.println("short " + numero);
    }

    public void metodo(Double numero) {
        System.out.println("Double " + numero);
    }

    public void metodo(double numero) {
        System.out.println("double " + numero);
    }
}
```

```
}
```

Esercizi del capitolo 14 e dell'appendice H

Stream API e Framework Collections

I prossimi esercizi riguardano sia il capitolo 14 che l'appendice H. Infatti, gli argomenti riguardanti la Stream API, sono spesso collegati con le collection, e quindi non distingueremo i due argomenti con la numerazione degli esercizi. Queste ultime, sono tra le implementazioni più utilizzate in assoluto in Java. La documentazione è fondamentale, per essere programmatori migliori bisognerebbe consultarla assiduamente. Essendo una libreria molto estesa, ci sarà sempre un metodo o un'implementazione che fa al caso nostro, che non abbiamo mai utilizzato. La Stream API ora ha ulteriormente ampliato gli orizzonti d'applicazione delle collection.

Esercizio 14.a) *Framework Collections, Vero o Falso:*

1. `Collection`, `Map`, `SortedMap`, `Set`, `List` e `SortedSet` sono interfacce e non possono essere istanziate.
2. Un `Set` è una collezione ordinata di oggetti; una `List` non ammette elementi duplicati ed è ordinata.
3. Le mappe non possono contenere chiavi duplicate ed ogni chiave può essere associata ad un solo valore.
4. Esistono diverse implementazioni astratte da personalizzare nel framework come `AbstractMap`.
5. Una `HashMap` è più performante rispetto ad una `Hashtable` perché non è sincronizzata.
6. Una `HashMap` è più performante rispetto ad un `TreeMap` ma quest'ultima, essendo un'implementazione di `SortedMap`, gestisce l'ordinamento.
7. `HashSet` è più performante rispetto a `TreeSet` ma non gestisce l'ordinamento.
8. `Iterator` ed `Enumeration` hanno lo stesso ruolo ma quest'ultima permette durante le iterazioni di rimuovere anche elementi.
9. `ArrayList` ha prestazioni migliori rispetto a `Vector` perché non è sincronizzato, ma entrambi hanno meccanismi per ottimizzare le prestazioni.
10. La classe `Collections` è una lista di `Collection`.

Esercizio 14.b) *Stream API, Vero o Falso:*

1. `Stream` è una classe che implementa `Collection`.
2. È possibile iterare su uno stream in entrambe le direzioni.
3. Una pipeline è costituita da una sorgente, metodi di aggregazione opzionali e un metodo terminale.
4. Il metodo `map` è da considerarsi un metodo di aggregazione.
5. `Optional` è un'interfaccia che permette di evitare di avere a che fare con le `NullPointerException`.
6. I metodi di riduzione sono operazioni di aggregazione.

7. Il metodo `joining` di `Stream` permette di concatenare stringhe con dei separatori di tipo stringa.
8. La classe `DoubleSummaryStatistics` è un particolare stream.
9. Il metodo `parallelStream` ritorna uno stream capace di usare l'algoritmo Fork/Join per eseguire operazioni sugli elementi di una collection.
10. Uno stream è istanziabile solo a partire da una collezione.

Esercizio 14.c)

Creare una collection che, nel caso si aggiungano due elementi uguali, lanci un'eccezione personalizzata (da creare anch'essa). Creare infine anche una classe di test.

Esercizio 14.d)

Creare una mappa chiamata `MappaIncrementale`, che permette di aggiungere più valori (ordinati in ordine di aggiunta) alla stessa chiave. Creare anche una classe di test.

Esercizio 14.e)

Sfruttando le classi create negli esercizi precedenti creare un'estensione di `MappaIncrementale` (chiamiamola `MappaIncrementaleRobusta`) che, quando si aggiunge un elemento la cui chiave esiste già, lancia un'eccezione come descritto nell'esercizio 14.c. Creare anche una classe di test.



Esercizio 14.f)

Dopo aver svolto l'esercizio precedente, si riempia la classe di test di oggetti `Citta`, la cui classe è stata creata nell'esercizio 13.c e che riportiamo di seguito per comodità:

```
package com.claudiodesio.dat;

public class Citta {

    private String nome;

    private boolean capoluogo;

    private boolean diMare;

    public Citta(String nome, boolean capoluogo, boolean diMare) {
        this.nome = nome;
        this.capoluogo = capoluogo;
        this.diMare = diMare;
    }

    public boolean isDiMare() {
```

```
        return diMare;
    }

    public void setDiMare(boolean diMare) {
        this.diMare = diMare;
    }

    public boolean isCapoluogo() {
        return capoluogo;
    }

    public void setCapoluogo(boolean capoluogo) {
        this.capoluogo = capoluogo;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    @Override
    public String toString() {
        return getNome();
    }
}
```

Aggiungere metodi `equals` e `hashCode`. In una classe `TestStream`, con uno stream stampare tutte le città di mare. Con un secondo stream stampare tutte le città capoluogo. Con un terzo stream stampare tutte le città che finiscono con la lettera 'a'.

Esercizio 14.g)

Riguardo il ciclo `for` migliorato, quali delle seguenti affermazioni sono corrette?

1. Il ciclo `for` migliorato può in ogni caso sostituire un ciclo `for`.
2. Il ciclo `for` migliorato può essere utilizzato con gli array e con le classi che implementano `Iterable`.
3. Il ciclo `for` migliorato sostituisce l'utilizzo di `Iterator`.
4. Il ciclo `for` migliorato non può sfruttare correttamente i metodi di `Iterator`.
5. In un ciclo `for` migliorato non è possibile effettuare cicli all'indietro.

Esercizio 14.h)

Considerato il seguente codice:

```
import java.util.*;

public class Esercizio14H {
    public static void main(String args[]) {
        Collection map = new HashMap(2);
        map.put(1,1);
        System.out.println(map);
    }
}
```

```
}
```

Quale tra le seguenti affermazioni è corretta?

1. Il codice viene compilato con dei warning, ed eseguito stampa {1=1}.
2. Il codice viene compilato con dei warning, ma lancia un'eccezione durante l'esecuzione.
3. Il codice non compila.
4. Il codice viene compilato correttamente, ed eseguito stampa {1=1}.
5. Il codice viene compilato con dei warning, ed eseguito stampa {1=1, null=null}.
6. Il codice viene compilato con dei warning, ed eseguito stampa {1=1, 0=0}.

Esercizio 14.i)

Quali delle seguenti affermazioni sono corrette?

1. L'interfaccia `Iterable` dichiara il metodo `forEach`.
2. `Iterator` estende `Iterable`.
3. `Iterator` definisce il metodo `forEachRemaining`.
4. `Collection` implementa l'interfaccia `Iterable`.

Esercizio 14.l)

Quali delle seguenti affermazioni sono corrette?

1. `Collection` è una superclasse di `List`.
2. Una `Collection` può essere trasformata in un array invocando il metodo `toArray` definito nella classe `Arrays`.
3. Un array può essere trasformato in una `Collection` mediante il metodo `toCollection` della classe `Arrays`.
4. `Collection` definisce il metodo `add`.



Esercizio 14.m)

Consideriamo la seguente classe:

```
import java.util.*;

public class Esercizio14M {
    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<>(3);
        list.add("*");
        list.add("@");
        list.set(1, "$");
        ListIterator listIterator = list.listIterator();
        while(listIterator.hasNext()) {
            System.out.println(listIterator.next());
        }
        while(listIterator.hasPrevious()) {
            System.out.println(listIterator.previous());
        }
    }
}
```

```
}  
}
```

Se eseguiamo questa classe, quale sarà l'output?

Esercizio 14.n)

Si crei un semplice programma che definisca un `ArrayList` di interi, e si trovi un modo affinché venga riempito con i numeri pari che vanno da 2 a 2000000 nella maniera più efficiente.

Esercizio 14.o)

Quali delle seguenti affermazioni sono corrette?

1. Un'implementazione di `Set` non può ordinare i suoi elementi.
2. Un'implementazione di `Set` non ammette più di un elemento `null`.
3. L'interfaccia `Set` è estesa da `SortedSet`.
4. L'implementazione `HashSet` non è thread safe.

Esercizio 14.p)

Quali delle seguenti affermazioni sono corrette?

1. La classe `Vector` è un'implementazione di `List`.
2. Un'implementazione di `List` non ammette elementi `null`.
3. Un reference di tipo `List` può diventare thread safe se gli viene assegnato un oggetto di tipo `List` che viene ritornato dal metodo `synchronizedList`.
4. Per eliminare gli elementi duplicati da una lista, basta riempire un `Set` con gli elementi della `List`.

Esercizio 14.q)

Data la classe:

```
import java.util.*;  
  
public class ClaudioLinkedList extends LinkedList<String> {  
    public ClaudioLinkedList() {  
        add("X");  
        add("L");  
        add("W");  
        add("U");  
        add("D");  
        add("I");  
        add("Z");  
    }  
}
```

aggiungere nella seguente classe `Esercizio14Q`:

```
import java.util.*;  
  
public class Esercizio14Q {
```

```
public static void main(String args[]) {  
  
    ClaudioLinkedList claudioLinkedList = new ClaudioLinkedList();  
    /*INSERISCI CODICE QUI*/  
    System.out.println(claudioLinkedList);  
}  
}
```

il codice al posto del commento INSERISCI CODICE QUI, che permetterà di generare il seguente output:

```
[C, L, A, U, D, I, O]
```

Esercizio 14.r)

Se vogliamo utilizzare una mappa in maniera thread-safe, che opzioni abbiamo? Elencare almeno due opzioni.

Esercizio 14.s)

Se vogliamo utilizzare una lista immutabile, che opzioni abbiamo? Elencare almeno due opzioni. E se vogliamo utilizzare un set immutabile, che opzioni abbiamo? Elencare almeno due opzioni.



Esercizio 14.t)

Considerata la lista di stringhe che ritorna il seguente metodo:

```
public static List<String> getStringList() {  
    String stringa = "I ragazzi che si amano si baciano in piedi "  
        + "Contro le porte della notte "  
        + "E i passanti che passano li segnano a dito "  
        + "Ma i ragazzi che si amano "  
        + "Non ci sono per nessuno "  
        + "Ed è la loro ombra soltanto "  
        + "Che trema nella notte "  
        + "Stimolando la rabbia dei passanti "  
        + "La loro rabbia il loro disprezzo le risa la loro invidia "  
        + "I ragazzi che si amano non ci sono per nessuno "  
        + "Essi sono altrove molto più lontano della notte "  
        + "Molto più in alto del giorno "  
        + "Nell'abbagliante splendore del loro primo amore ";  
    String[] stringhe = stringa.split(" ");  
    return Arrays.asList(stringhe);  
}
```

scrivere una pipeline che consideri solo le parole che non iniziano con “a”, e ne calcoli (e stampi) la lunghezza media.

Esercizio 14.u)

Quali delle seguenti affermazioni sono corrette?

1. `Optional` è un'interfaccia generica.
2. Il metodo `ofNullable` restituisce un oggetto `Optional` che fa da wrapper all'oggetto passato in input.
3. `orElseThrow` è un metodo di `Optional` che restituisce un oggetto `Optional` o lancia un'eccezione che può essere specificata in input, nel caso l'oggetto "wrappato" sia `null`.
4. `findFirst` è un metodo di `Optional` che restituisce un oggetto `Optional` o `null`, nel caso l'oggetto "wrappato" sia `null`.



Esercizio 14.v)

Considerato il seguente codice:

```
import java.util.*;
import java.util.stream.*;

public class Esercizio14V {

    public static void main(String args[]) {
        List<String> stringList = getStringList();
        /*INSERISCI CODICE QUI*/
        System.out.println(map);
    }

    public static List<String> getStringList() {
        String stringa = "I ragazzi che si amano si baciano in piedi "
            + "Contro le porte della notte "
            + "E i passanti che passano li segnano a dito "
            + "Ma i ragazzi che si amano "
            + "Non ci sono per nessuno "
            + "Ed è la loro ombra soltanto "
            + "Che trema nella notte "
            + "Stimolando la rabbia dei passanti "
            + "La loro rabbia il loro disprezzo le risa la loro invidia "
            + "I ragazzi che si amano non ci sono per nessuno "
            + "Essi sono altrove molto più lontano della notte "
            + "Molto più in alto del giorno "
            + "Nell'abbagliante splendore del loro primo amore ";
        String[] stringhe = stringa.split(" ");
        return Arrays.asList(stringhe);
    }
}
```

scrivere una pipeline per creare una mappa che raggruppi le parole del testo che iniziano per la stessa iniziale, ignorando il fatto che l'iniziale sia maiuscola o minuscola. Inserire tale pipeline al posto del commento `INSERISCI CODICE QUI`.

Esercizio 14.z)

Partendo dal risultato dell'esercizio 14.v, sostituire all'istruzione di stampa:

```
System.out.println(map);
```

con una semplice pipeline che stampi riga per riga il contenuto della mappa.

Esercizio 14.aa)

Quali delle seguenti affermazioni sono corrette?

1. La classe `HashMap` estende `Hashtable`.
2. La classe `Hashtable` non è thread-safe.
3. La classe `Hashtable` estende `Properties`.
4. `Enumeration` è stata deprecata da quando sono state introdotte le enumerazioni e la parola chiave `enum` in Java 5.
5. L'interfaccia `Enumeration` ha gli stessi metodi dell'interfaccia `Iterator`.
6. La classe `Vector` implementa `List`.
7. La classe `Vector` non è thread-safe.
8. La classe `Vector` non è generica.
9. In generale la classe `Vector`, essendo sincronizzata, offre prestazioni superiori rispetto ad un `ArrayList`.
10. La classe `Vector` è deprecata.

Esercizi del capitolo 15 e dell'appendice I

Input-Output e Date-Time API

Gli esercizi di questo capitolo saranno inizialmente orientati alla teoria, con tre esercizi di tipo “vero-falso”. Poi riprenderemo l'esercizio della rubrica che avevamo realizzato nei primi capitoli aggiungendo la serializzazione degli oggetti passo dopo passo. Creeremo anche una interfaccia da riga di comando sfruttando il pattern MVC, arrivando a creare un grado di complessità notevole, anche per il programmatore già esperto. Come sempre poi troveremo alcuni esercizi propedeutici alla certificazione da programmatore Oracle. Infine ci sarà anche qualche esercizio per gli altri argomenti come il networking, e quelli riguardanti la Date-Time API descritta nell'appendice I (questi ultimi hanno una numerazione diversa che inizia con la lettera I). Utilizzare la documentazione, anche per svolgere questi esercizi, è fondamentale.

Esercizio 15.a) *Input - Output, Vero o Falso:*

1. Il pattern Decorator permette di implementare una sorta di ereditarietà dinamica. Questo significa che, invece di creare tante classi quante sono le entità da astrarre, al runtime sarà possibile concretizzare uno di questi concetti direttamente con un oggetto.
2. I reader e i writer permettono di leggere e scrivere caratteri. Per tale ragione sono detti “character stream”.
3. All'interno del package `java.io` l'interfaccia `Reader` ha il ruolo di `ConcreteComponent`.
4. All'interno del package `java.io` l'interfaccia `InputStream` ha il ruolo di `ConcreteDecorator`.
5. Un `BufferedWriter` è un `ConcreteDecorator`.
6. Gli stream che possono realizzare una comunicazione direttamente con una fonte o una destinazione vengono detti “node stream”.
7. I node stream di tipo `OutputStream` possono utilizzare il metodo:

```
int write(byte cbuf[])
```

per scrivere su una destinazione.
8. Il seguente oggetto in:

```
BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
```

permette di usufruire di un metodo `readLine` che leggerà frasi scritte con la tastiera delimitate dalla battitura del tasto Invio.
9. Il seguente codice:

```
File outputFile = new File("pippo.txt");
```

crea un file di testo chiamato `pippo.txt` nella cartella corrente.
10. Non è possibile decorare un `FileReader`.

Esercizio 15.b) *Serializzazione, Vero o Falso:*

1. Lo stato di un oggetto è definito dal valore delle sue variabili d'istanza (in un certo istante).
2. L'interfaccia `Serializable` non ha metodi.

3. `transient` è un modificatore applicabile a variabili e classi. Una variabile `transient` non viene serializzata con le altre variabili; una classe `transient` non è serializzabile.
4. `transient` è un modificatore applicabile a metodi e variabili. Una variabile `transient` non viene serializzata con le altre variabili; un metodo `transient` non è serializzabile.
5. Se si provasse a serializzare un oggetto che possiede tra le sue variabili d'istanza una variabile di tipo `Reader` dichiarata `transient`, otterremmo un `NotSerializableException` durante l'esecuzione.
6. Una variabile `static` non sarà serializzata.
7. Una variabile d'istanza di tipo `OutputStream` deve essere dichiarata `transient` affinché sia coinvolta in una serializzazione.
8. Non è possibile usare come parametro di un `try-with-resources` un'implementazione di `Serializable`.
9. È possibile creare un clone di un oggetto `transient` con metodi della libreria input-output.
10. Un oggetto può essere serializzato anche in `Base64`.

Esercizio 15.c) *New Input Output, Vero o Falso:*

1. NIO 2.0 sostituisce del tutto il modello di input output definito con il package `java.io`.
2. L'utilizzo della classe `File` potrebbe essere completamente rimpiazzato dall'utilizzo della classe `Files` e dell'interfaccia `Path`.
3. Il metodo `toPath` della classe `File` restituisce l'oggetto `Path` equivalente.
4. `Path` è un'interfaccia perché la sua implementazione dipende dalla piattaforma.
5. Il metodo `relativize` appartiene alla classe `Files` e restituisce il percorso per arrivare dal `Path` specificato come primo argomento al `Path` specificato come secondo argomento.
6. Il metodo `subPath` dell'interfaccia `Path` non restituisce il nodo radice.
7. Il metodo `delete` dell'interfaccia `Files` solleverà un'eccezione nel caso si tenti di eliminare una directory non vuota.
8. Non ha senso ottenere `Reader` o `Writer` da un oggetto `Files`.
9. Il nome del file temporaneo è sempre stabilito dal sistema operativo.
10. Un file temporaneo viene salvato in una directory che dipende dal sistema operativo.

Esercizio 15.d)

Creare una classe `EditorInterattivo` contenente un metodo `main` che permetta di scrivere in un file quello che si scrive da riga di comando.

Esercizio 15.e)

Creare una classe `More` contenente un metodo `main` che simuli il famoso programma `more` che troviamo nei sei sistemi Linux. L'applicazione, una volta eseguita, aspetterà che l'utente specifichi il file da leggere in input. Dopo aver caricato il file specificato dall'utente, il programma rimarrà in attesa di un ulteriore input da parte dell'utente. Se viene premuto il tasto "n" seguito dal tasto "invio", allora verranno visualizzate le prime 10 righe del file caricato, ed ogni volta che viene ripetuta l'azione il programma dovrà stampare le successive 10 righe, fino a quando termineranno le

righe del file. Specificando però la lettera “q” seguita da “invio” il programma dovrà terminare (anche se non sono state lette tutte le righe).

Esercizio 15.f)

Partendo dalla soluzione dell’esercizio 6.y, continuiamo a fare evolvere la nostra applicazione che simula il funzionamento di una rubrica. In questo e nei prossimi esercizi, realizzeremo quindi nuove classi della nostra applicazione passo dopo passo. Più in avanti andremo ad integrare le classi realizzate nell’esercizio 6.y con quelle che creeremo nei prossimi esercizi.

Utilizzeremo come “strato di persistenza dei dati”, un componente che si occupa di serializzare e deserializzare oggetti sul file system. In altre parole vogliamo creare una classe che si chiama `GestoreFile`, appartenente al package `rubrica.integrazione` e che espone i seguenti metodi:

- `inserisci` che prende in input un oggetto di tipo `Contatto`, e non ritorna nulla. Avrà il compito di serializzare sul nostro file system l’oggetto `Contatto` che prende in input, in un file con il nome del contatto (variabile `nome` della classe `Contatto`) e con suffisso `.con`. Per esempio se il nome del contatto è “Ross”, allora il nome del file dovrà essere `Ross.con`.
- `recupera` che prende in input il nome del contatto da recuperare tramite deserializzazione. Per esempio se viene passato il nome “Phoebe Buffay”, deve essere recuperato il file `Phoebe Buffay.con`.

L’unica classe dell’esercizio 6.y che per ora dobbiamo riusare è la classe `Contatto`, che però estendeva la classe `Entita`, che a sua volta implementava le interfacce `Dato` e `Identificabile`, quindi partiamo da questi file. Al fine di far funzionare la serializzazione della classe `Contatto`, facciamo in modo che la classe `Entita` vada ad implementare anche l’interfaccia `java.io.Serializable`.

Inoltre sostituire il metodo `stampaDettagli` della classe `Contatto` con un metodo `toString`.

Esercizio 15.g)

Si crei una classe di test `Esercizio15G` con la quale si possa verificare che i metodi creati nella classe `GestoreFile` nell’esercizio precedente funzionino correttamente. Questa classe deve creare un array di tre contatti, serializzarli e provare a rileggerli.

Esercizio 15.h)

Partendo dalla soluzione dell’esercizio precedente, creare la classe `Esercizio15H` a partire dalla classe `Esercizio15G` aggiungendo (e chiamando) due metodi di test che testano:

- cosa succede se si prova ad aggiungere un contatto già esistente;
- cosa succede se si prova a recuperare un contatto inesistente.

Riflettete sui risultati dei test e:

- progettare le soluzioni degli eventuali problemi che sono stati riscontrati
- implementare le soluzioni progettate

Esercizio 15.i)

Partendo dalla soluzione dell'esercizio precedente, aggiungiamo nella classe `GestioneFile` i metodi:

- `modifica` che modifica un contatto (non si dovrà poter cambiare il nome del contatto);
- `rimuovi` che cancella il contatto con il nome specificato.

Creare anche i metodi nella classe di test che testano il corretto funzionamento dei metodi `modifica` e `rimuovi`.

Esercizio 15.j)

Partendo dalla soluzione dell'esercizio precedente aggiungiamo un altro requisito, questa volta non funzionale ma architetturale: applicando il principio di inversione della dipendenza (DIP, introdotto nel paragrafo H.2.1 dell'appendice H). Come già fatto in precedenza per la classe `Contatto`, facciamo quindi evolvere la classe `GestoreFile`, in modo tale che rappresenti un'implementazione di uno strumento generico per interagire con un meccanismo di immagazzinamento qualsiasi. Creiamo quindi un'interfaccia `GestoreSerializzazione` da far implementare a `GestoreFile`. Questa modifica ci permetterà di cambiare facilmente il modo in cui serializziamo i nostri oggetti (vedi esercizio 15.l).

Esercizio 15.k)

Partendo dalla soluzione dell'esercizio precedente aggiungiamo un altro requisito. Facciamo un po' di refactoring del codice, eliminando il codice duplicato nelle classi create. In particolare creare la classe `Esercizio15k`, eliminando le duplicazioni di codice che erano presenti nella classe `Esercizio15j`.

Naturalmente il lettore è libero di aggiungere qualsiasi miglioramento al proprio codice in qualsiasi classe.

Nello sviluppo è molto importante ad un certo punto fermarsi a riflettere, e migliorare il proprio codice, tramite “refactoring”, ovvero cambiare la struttura interna del software, senza modificarne il comportamento esterno. Questo a qualcuno può sembrare una perdita di tempo ma in realtà è esattamente il contrario. Il refactoring migliora la qualità del codice a livello non funzionale migliorandone le caratteristiche architetturali. Favorisce alcune di quelle che vengono dette “qualità sistemiche del software”, per esempio promuove la manutenibilità e l'estensibilità del codice, riducendone la complessità. In questo modo avremo meno bachi e più consapevolezza del nostro codice. Il mio consiglio personale è quello di dedicare una percentuale giornaliera del tempo di sviluppo che varia dal 10% al 25% al refactoring, possibilmente nel momento in cui ci si rende conto di essere troppo stanchi. Per esempio in una giornata lavorativa da 8 ore, le ultime due (o almeno l'ultima) potrebbe essere dedicata a migliorare la qualità del software. Per informazioni sul refactoring potete:

- andare sul sito <https://refactoring.com>;

- comprare il libro “Refactoring” di Martin Fowler, come consigliato nella bibliografia online;
- utilizzare i meccanismi di refactoring che offrono gli IDE più famosi.

Esercizio 15.l)

Partendo dalla soluzione dell’esercizio precedente, creiamo anche un’implementazione alternativa della classe `GestoreFile`, che fa uso della libreria NIO2 (classe `GestoreFileNIO2`). Implementare anche la classe di test `Esercizio15L`.



Esercizio 15.m)

Partendo dall’esercizio precedente, usando un oggetto di tipo `Properties` (introdotto nell’appendice G), creare un file di configurazione `config.properties` con contiene un’unica property, che servirà alla nostra applicazione per scegliere se utilizzare per la serializzazione, la classe `GestoreFile` o la classe `GestoreFileNIO2`. Creare una classe `GestoreSerializzazioneFactory`, che implementa il pattern Factory Method definendo il metodo `getGestoreSerializzazione`. Infine creare la classe `Esercizio15M` a partire dalla classe `Esercizio15L`, che sfrutta il meccanismo appena descritto per la serializzazione degli oggetti.



Esercizio 15.n)

Questo e i prossimi esercizi, sono da considerarsi particolarmente difficili. Sono esercizi difficili anche per un professionista. A volte bisognerà andare su internet per studiare librerie e soluzioni, e cercare di essere molto lucidi. Quindi anche se i risultati non saranno brillanti, non dovete preoccuparvi. Se vi bloccate, leggete le soluzioni e andate avanti. Se invece riuscite a risolverli in qualche modo, allora significa che siete arrivati ad un livello di programmazione davvero importante.

A partire da questo esercizio, creeremo una interfaccia utente da riga di comando. È un esercizio complesso che richiede un procedimento passo dopo passo. In questo primo esercizio, usando un oggetto di tipo `Scanner`, definiremo quindi una classe `RubricaCLI`, che rappresenta una interfaccia utente da riga di comando (“CLI” è l’acronimo di “Command Line Interface”, ovvero “interfaccia da riga di comando”). Useremo il pattern architetturale Model View Controller (MVC) che viene schematizzato nella Figura 15.n.1.

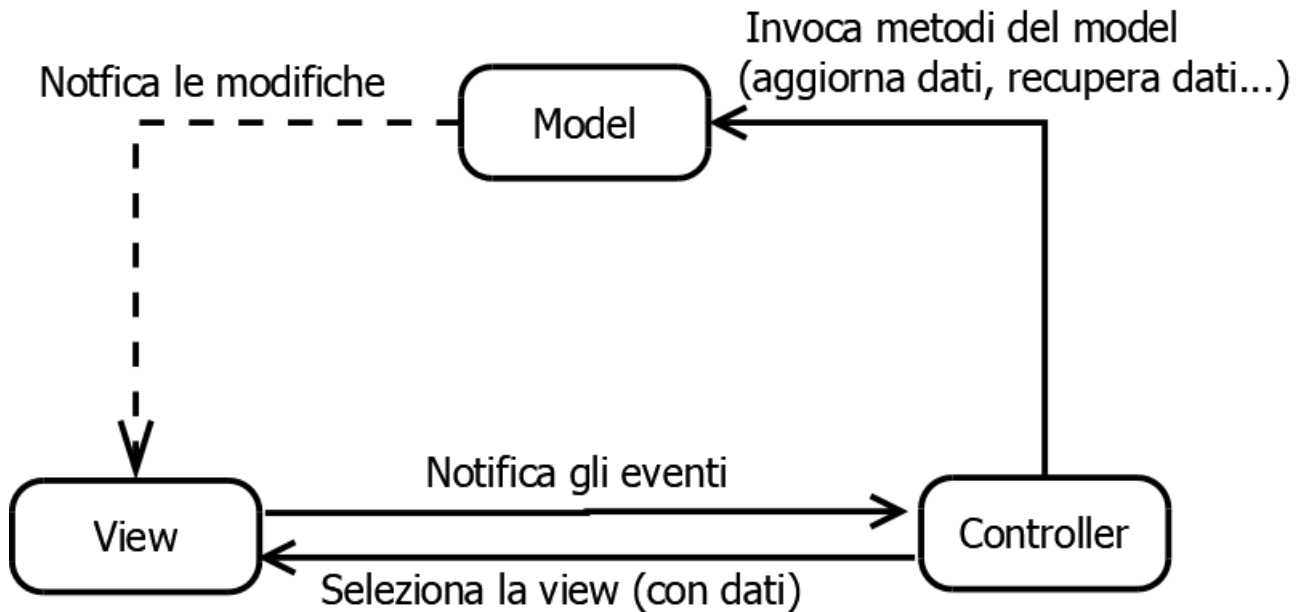


Figura 15.n.1 – Model View Controller.

Notare che la freccia che va dal Model alla View è stata disegnata tratteggiata, perché anche se rappresenta una responsabilità del classico pattern MVC, non sarà implementata nel nostro caso (come spesso accade), per rendere indipendente il componente Model dal componente View. I dati del Model aggiornati, saranno passati alla View direttamente dal Controller.

Il pattern MVC promuove la separazione delle responsabilità in un software in componenti diversi, definendo tre ruoli principali, per l'appunto il Model, la View e il Controller. Questa distinzione ci garantirà una serie di vantaggi dal punto di vista della manutenibilità, la facilità di trovare bug, la riusabilità, e così via. Daremo per scontato che il ruolo di Model sarà interpretato dall'insieme delle classi che abbiamo creato (a parte le classi di test) e che sinora compongono la nostra applicazione. La classe invece, `RubricaCLI` rappresenterà il componente View, ovvero il componente che implementa la cosiddetta *logica di presentazione*. Questo significa che dovrà avere due responsabilità fondamentali:

- definire le “schermate” per l'utente (che nel nostro caso consisteranno in del semplice testo)
- notificare al componente controller eventuali input dell'utente (implementazione che faremo a partire dal prossimo esercizio).

Inoltre nel nostro caso, vogliamo sottolineare che la nostra classe `RubricaCLI`, avrà una responsabilità aggiuntiva:

- gestire le eccezioni (ovvero deve utilizzare blocchi `try-catch` per visualizzare eventuali eccezioni che si sono propagate dal componente Model).

Questo significa che i metodi dei componenti Controller e Model non gestiranno eccezioni ma semplicemente ne permetteranno la propagazione. In altre parole, i metodi dei componenti Controller e Model non utilizzeranno blocchi `try-catch`, ma clausole `throws`, per catturare eccezioni lanciate con la parola chiave `throw`.

La nostra implementazione deve seguire le specifiche rappresentate nel diagramma degli stati rappresentato in figura 15.n.2.

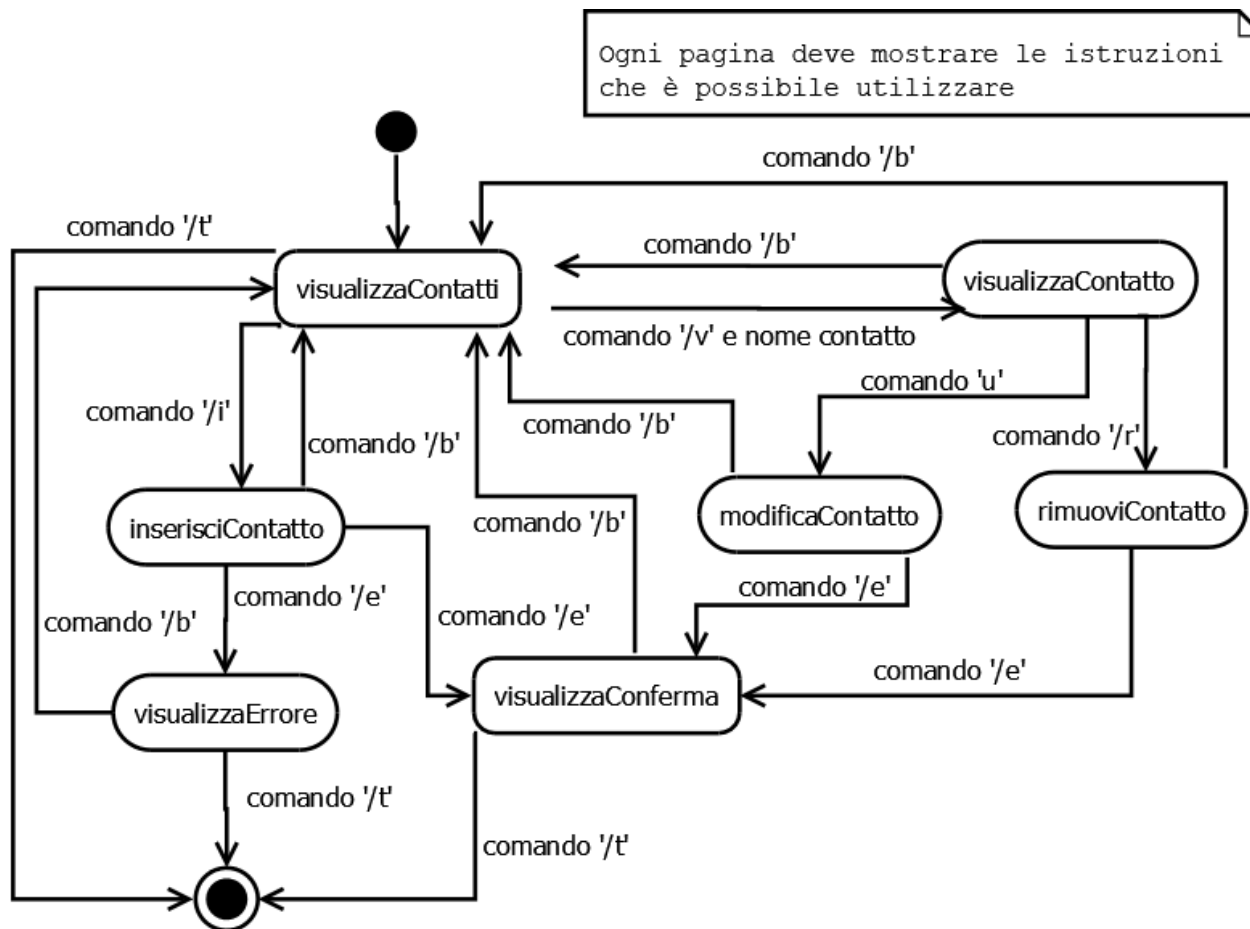


Figura 15.n.2 – Diagramma degli stati.

Il diagramma degli stati solitamente viene utilizzato per descrivere gli stati di un'entità (per esempio di un oggetto), che assume durante il proprio ciclo di vita. Esso definisce come elementi principali gli stati (rappresentati come rettangoli con spigoli arrotondati) e le transizioni tra gli stati. Queste ultime hanno un verso, e possono essere etichettate con una sintassi costituita da tre parti (tutte opzionali): evento-scatenante[condizione]/azione. Inoltre ci sono gli elementi di inizio e fine che abbiamo già incontrato con il diagramma delle attività. Altre informazioni sulla sintassi UML si trovano nell'approfondimento 4.7.

Il diagramma è piuttosto complesso, ma basterà fare un passo alla volta per poterlo interpretare, e alla fine scopriremo che è una specifica fondamentale per il nostro programma. La programmazione di interfacce utente infatti, differisce molto dalla programmazione tradizionale che abbiamo utilizzato sinora. Un diagramma come quello specificato nella figura 15.n.2 ci permetterà di non dover improvvisare una soluzione.

Nel diagramma abbiamo utilizzato gli stati per rappresentare le “schermate testuali” che devono essere visualizzate, e le transizioni etichettate con gli eventi scatenanti che portano da una schermata ad un'altra.

Notare che abbiamo specificato un protocollo ben preciso, dove per ogni schermata esiste un particolare insieme di comandi da poter specificare, per

passare ad un'altra schermata. Tutti i comandi operativi sono costituiti da una sola lettera preceduto dal simbolo “/”,

Per realizzare la classe `RubricaCLI`, iniziamo quindi a creare i metodi rappresentati negli stati del diagramma. Questi dovranno:

- stampare dei messaggi coerenti
- catturare l'input dell'utente con un oggetto `Scanner` (dichiararlo come variabile d'istanza).

In questo esercizio quindi, il nostro compito si deve limitare a:

- creare la classe `RubricaCLI`, che abbia come variabile d'istanza un oggetto `Scanner` inizializzata in un apposito costruttore.
- Definire il metodo `visualizzaContatti` che prende in input un oggetto `List` di contatti e ritorni `void`. Tale metodo dovrà per ora implementare il flusso descritto nel diagramma, ovvero, dopo aver stampato la lista dei contatti che prende in input, e la lista dei comandi possibili da specificare, aspettare l'input dell'utente e con dei costrutti condizionali stampare solo (per ora) la descrizione del comando specificato dall'utente, poi il programma può terminare. Per esempio se l'utente specifica il comando “/i”, allora il programma deve stampare “Chiamato metodo inserisci contatto”. Se invece l'utente specifica il comando “/v” seguito dal nome di un contatto presente nella lista (supponiamo il nome “Giovanni”), il programma deve stampare “Chiamato metodo visualizza contatto per Giovanni”. Infine se l'utente specifica il comando “/t” il programma deve terminare.
- Creare un'eccezione chiamata `ComandoNonValidoException` che deve essere lanciata nel caso in cui il comando specificato dall'utente non sia valido.
- Lanciare l'eccezione `ContattoInesistenteException` (già creata in precedenza) nel caso l'utente inserisca il comando per visualizzare il contatto “/v” seguito da un nome che non si trova nella lista.
- Infine creare la classe `Esercizion15N` che servirà per lanciare l'applicazione (bisogna istanziare l'oggetto `RubricaCLI` e chiamare il metodo `visualizzaContatti` passandogli in input una lista di oggetti `Contatto`).

Nei prossimi esercizi faremo evolvere questa classe, fino a farla diventare pienamente funzionante.

A partire da questo esercizio, per catturare l'input dell'utente, consigliamo di utilizzare sempre il metodo `nextLine` della classe `Scanner`, per non incorrere in strani comportamenti. Infatti, l'utilizzo congiunto di `nextLine`, e di altri metodi come `next`, sullo stesso oggetto `Scanner`, comporta che la lettura debba essere gestita in maniera più complessa. Quindi utilizzare sempre e solamente il metodo `nextLine`.



Esercizio 15.o)

Partendo dalla soluzione dell'esercizio precedente, iniziamo a creare la classe `RubricaController`, che si occuperà di svolgere il ruolo del componente Controller all'interno dell'architettura MVC. In particolare, tale classe ha il compito di definire la cosiddetta logica di controllo, ovvero dovrà implementare le seguenti responsabilità:

- ricevere ed interpretare gli input dell'utente notificati dal componente View;

- invocare metodi di business esposti dal componente Model (sia aggiornando che recuperando dati);
- selezionare la giusta schermata del componente View.

Detto questo, nell'esercizio precedente abbiamo creato il metodo `visualizzaContatti` che si limitava a stampare determinati messaggi. È arrivato il momento di sostituire quei messaggi con notifiche al componente Controller (ovvero la View deve chiamare metodi sulla classe `RubricaController`). Tali metodi, come abbiamo già detto, vengono rappresentati nel diagramma della figura 15.n.1 come le transizioni che portano da una schermata ad un'altra. I metodi di `RubricaController` cominceranno sempre con la parola "gestisci".

Con l'esercizio precedente abbiamo invocato dalla classe `Esercizio15N` il metodo `visualizzaContatti` della classe `RubricaCLI`. La classe `Esercizio15N` si occupava di passare la metodo `visualizzaContatti` anche la lista dei contatti. In questo esercizio invece, la classe `Esercizio150` invocherà un nuovo metodo della classe `RubricaCLI` che chiameremo `start`. Questo metodo a sua volta invocherà il metodo `start` della classe `RubricaController`. Qui, per rispettare le responsabilità del componente Controller, verrà invocato un metodo chiamato `getContatti` (da creare) del Model che restituirà tutti i contatti esistenti. Una volta che i contatti sono stati restituiti dal metodo `getContatti`, il metodo `start` della classe `RubricaController` invocherà il metodo `visualizzaContatti` della classe `RubricaCLI`. Tutto questo è riassunto nel sequence diagram riportato in figura 15.o.1:

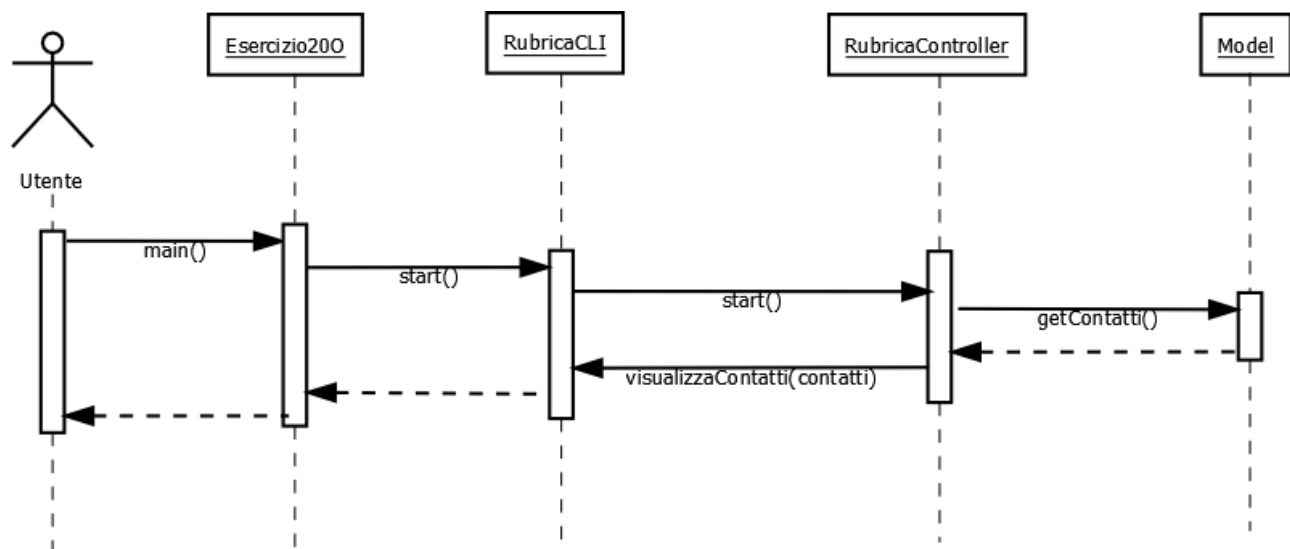


Figura 15.o.1 – Diagramma dei sequenza.

Quindi in questo esercizio dovremo:

- Creare il metodo `getContatti` nel componente Model, che restituisce i contatti già creati. Notare che inizialmente non avremo contatti da recuperare, quindi per testare il metodo `getContatti`, senza dover aspettare il completamento del prossimo esercizio, è possibile utilizzare i file .con creati negli esercizi precedenti (copiandoli nella vostra cartella di lavoro attuale), oppure creare un client di test che inserisca qualche contatto come fatto nei precedenti esercizi.
- Creare la classe `RubricaController` come classe innestata nella classe `RubricaCLI` (in questo modo risparmieremo diverse righe di codice).
- Creeremo il metodo `start` nella classe `RubricaController`. Questo deve chiamare il metodo `getContatti` del componente Model, e una volta ottenuti i contatti, deve passarli al

metodo `visualizzaContatti` di `RubricaCLI`. Nell'esercizio precedente avevamo chiamato dalla classe `Esercizio15N` il metodo `visualizzaContatti` della classe `RubricaCLI`. In quel caso la classe `Esercizio15N` si occupava anche di creare la lista dei contatti da passare al metodo `visualizzaContatti`. Ora invece sarà il metodo `start` nella classe `RubricaController` a recuperare i contatti e a passarli al metodo `visualizzaContatti` della classe `RubricaCLI`.

- Creare il metodo `start` nella classe `RubricaCLI` che invocherà il metodo `start` della classe `RubricaController`.
- Creare la classe `Esercizio15O` che invoca il metodo `start` della classe `RubricaCLI` per far partire l'applicazione.



Esercizio 15.p)

Partendo dalla soluzione dell'esercizio precedente, implementare il flusso di inserimento di un contatto come descritto nel diagramma mostrato in figura 15.n.2. In particolare:

- Creare il metodo `gestisciVisualizzaContatti` nella classe `RubricaController`, che deve prendere in input il comando inserito dall'utente, e la lista dei contatti. Tale metodo deve essere invocato nel metodo `visualizzaContatti` (che va cambiato completamente) quando l'utente ha inserito un comando, e deve avere la responsabilità di interpretare questo comando e invocare i giusti metodi. In particolare, deve eseguire il flusso condizionale che avevamo definito nel metodo `visualizzaContatti` nell'esercizio precedente, con l'eccezione del caso dell'inserimento. Infatti se è stato inserito il comando `"i"` allora deve essere invocare il metodo `inserisciContatto` della classe `RubricaCLI`. Per il resto il codice per ora non deve cambiare nulla. In pratica bisogna spostare parte del codice che avevamo nel metodo `visualizzaContatti` della classe `RubricaCLI`, nel metodo `gestisciVisualizzaContatti` della classe `RubricaController`, modificando la condizione relativo all'inserimento.
- Modificare il metodo `visualizzaContatti` in modo tale che dopo aver catturato il comando dell'utente, ne deleghi l'interpretazione la metodo `gestisciVisualizzaContatti` della classe `RubricaController`, ovvero deve invocare questo metodo passandogli il comando inserito e la lista dei contatti.
- Creare il metodo `inserisciContatto` nella classe `RubricaCLI`. Tale metodo deve richiedere all'utente prima l'inserimento del nome, poi del numero di telefono, e infine dell'indirizzo. Dopo aver raccolto le informazioni del contatto, deve offrire l'opportunità all'utente di inserire il comando `"e"` per confermare l'inserimento, o il comando `"b"` per tornare alla visualizzazione dei contatti (vedi figura 15.n.2). Una volta recuperato il comando inserito dall'utente, deve invocare il metodo `gestisciInserisciContatto` nella classe `RubricaController`, che deve avere la responsabilità di interpretare tale comando.
- Creare il metodo `gestisciInserisciContatto` nella classe `RubricaController`. Tale metodo deve prendere in input il comando inserito dall'utente, il nome, il numero di telefono e l'indirizzo del contatto. Questo metodo deve avere la responsabilità di interpretare il comando inserito e deve essere invocato nel metodo `inserisciContatto` che aveva la responsabilità di recuperare l'input dell'utente. In particolare, nel caso l'utente abbia inserito il comando `"b"` bisogna invocare nuovamente il metodo `visualizzaContatti` della

classe `RubricaCLI`. Nel caso l'utente abbia inserito il comando `“/e”` bisogna invocare il metodo `inserisci` del componente `Model`, senza gestire eventuali eccezioni, che invece devono essere gestite direttamente dai metodi di `RubricaCLI`, come già detto in precedenza. Infine deve invocare il metodo `visualizzaConferma` della classe `RubricaCLI`.

- Non deve essere possibile creare un contatto senza nome. Creare quindi un'eccezione chiamata `NomeVuotoException`, da lanciare nel metodo più opportuno.
- Creare il metodo `visualizzaConferma` della classe `RubricaCLI`. Questo metodo deve essere inteso come generico e riutilizzabile, deve stampare la frase `“Operazione confermata!”`, e deve prendere in input il messaggio che deve mostrare subito dopo (per esempio potremmo passare l'istruzione `"Inserito contatto:\n"+ contatto.toString()`). Infine deve permettere all'utente di inserire i comandi descritti nel diagramma dalla figura 15.n.2, ovvero i comandi `“/b”` per tornare alla visualizzazione dei contatti e `“/t”` per terminare il programma. Dopo aver catturato il comando dell'utente deve esserne delegata l'interpretazione al metodo `gestisciVisualizzaMessaggio` della classe `RubricaController`. Stesso discorso per un metodo generico che si chiama `visualizzaErrore`, che può essere chiamato in tutte le clausole `catch` dei metodi di `RubricaCLI`, per ottenere una gestione centralizzata delle eccezioni.
- Il metodo `gestisciVisualizzaMessaggio` della classe `RubricaController`, deve prendere in input il comando recuperato dal metodo `visualizzaConferma` della classe `RubricaCLI`. Seguendo il diagramma in figura 15.n.2, questo metodo deve invocare il metodo `visualizzaContatti` se il comando inoltrato dall'utente è `“/b”`, oppure terminare il programma se il comando inoltrato dall'utente è `“/t”`.
- Creare la classe `Esercizio15P` a partire dalla classe `Esercizio15Q` per testare l'applicazione.



Esercizio 15.q)

Partendo dalla soluzione dell'esercizio precedente, implementare i flussi di modifica e cancellazione di un contatto come descritto nel diagramma mostrato in figura 15.n.2. In particolare:

- Modificare il metodo `gestisciVisualizzaContatti` nella classe `RubricaController`, che prende in input il comando inserito dall'utente, e la lista dei contatti. Se l'utente specifica il comando `“/v”` seguito dal nome di un contatto esistente, sostituire l'attuale metodo di stampa, con la chiamata al metodo di un nuovo metodo: `visualizzaContatto` della classe `RubricaCLI`, che prende in input il contatto specificato.
- Creare il metodo `visualizzaContatto` della classe `RubricaCLI`, che deve quindi mostrare i dettagli del contatto selezionato e proporre all'utente la possibilità di tornare indietro a visualizzare tutti i contatti specificando il comando `“/b”`, oppure specificare il comando `“/u”` per modificare il contatto selezionato, o specificare il comando `“/r”` per rimuovere il contatto selezionato. Poi deve catturare il comando e delegarne l'interpretazione al metodo `gestisciVisualizzaContatto` della classe `RubricaController` che prende in input il comando e il contatto.
- Creare il metodo `gestisciVisualizzaContatto` che deve avere la responsabilità di interpretare il comando inserito. In particolare, nel caso l'utente abbia inserito il comando

“/b” bisogna invocare nuovamente il metodo `visualizzaContatti` della classe `RubricaCLI`. Nel caso l’utente abbia inserito il comando “/u” bisogna invocare il metodo `modificaContatto` della classe `RubricaCLI`. Se invece viene l’utente inserisce il comando “/r”, allora deve esser invocato il metodo `rimuoviContatto` della classe `RubricaCLI`.

- Creare il metodo `modificaContatto` della classe `RubricaCLI` che deve prendere in input il contatto selezionato. Esso deve chiedere all’utente di specificare un nuovo numero di telefono, e un nuovo indirizzo. Poi indicare all’utente la possibilità all’utente di scegliere tra il comando “/b” per ritornare a vedere tutti contatti, o il comando “/e” per eseguire l’aggiornamento. Infine deve catturare il comando dell’utente e delegarne l’interpretazione al componente `Controller`, invocando il metodo `gestisciModificaContatto` passando ad esso il comando, e il nome, il numero di telefono e l’indirizzo del contatto.
- Creare il metodo `gestisciModificaContatto` della classe `RubricaController` per interpretare il comando da eseguire. In particolare, esso, oltre a gestire come al solito il comando “/b” per tornare a vedere tutti i contatti della rubrica, nel caso l’utente abbia specificato il comando “/e” deve invocare il metodo `modifica` sul componente `Model`, per poi invocare il metodo `visualizzaConferma` della classe `RubricaCLI`.
- Creare il metodo `rimuoviContatto` della classe `RubricaCLI` che deve prendere in input il nome del contatto selezionato. Esso deve indicare all’utente la possibilità di scegliere tra il comando “/b” per ritornare a vedere tutti contatti, o il comando “/e” per eseguire la rimozione del contatto. Infine deve catturare il comando dell’utente e delegarne l’interpretazione alla classe `RubricaController`, invocando il metodo `gestisciRimuoviContatto` passando ad esso il comando e il nome del contatto.
- Creare il metodo `gestisciRimuoviContatto` della classe `RubricaController` per interpretare il comando da eseguire. In particolare, esso, oltre a gestire come al solito il comando “/b” per tornare a vedere tutti i contatti della rubrica, nel caso l’utente abbia specificato il comando “/e” deve invocare il metodo `rimuovi` sul componente `Model`, per poi invocare il metodo `visualizzaConferma` della classe `RubricaCLI`.
- Creare la classe `Esercizio15Q` a partire dalla classe `Esercizio15P` per testare l’applicazione.

Esercizio 15.r)

Quali delle seguenti affermazioni sono corrette?

1. La classe `File` definisce un metodo `getPath` che restituisce una stringa.
2. La classe `File` definisce un metodo `getParent` che restituisce un oggetto `File` che rappresenta una directory.
3. `pathSeparator` è una costante statica della classe `File`, che rappresenta il separatore di path dipendente dal sistema operativo.
4. La classe `File` definisce un metodo `delete` che restituisce un booleano `true` se il file sarà davvero eliminato, oppure `false` in caso contrario.

Esercizio 15.s)

Quali delle seguenti affermazioni sono corrette?

1. Le implementazioni di `InputStream` sono ottimizzate per leggere byte, le implementazioni di `OutputStream` sono ottimizzate per scrivere byte. Infatti vengono dette “byte stream”.
2. Le classi alla base della gerarchia dei “character stream” sono `Reader` e `Writer`.
3. Il metodo `readLine` è definito nella classe `Reader`.
4. Il costruttore di un `ObjectInputStream` deve prendere in input un oggetto di tipo `FileInputStream`.

Esercizio 15.t)

Quali delle seguenti affermazioni sono corrette?

1. Se istanziamo un oggetto `File` specificando come parametro del costruttore il nome di un file che non esiste, esso sarà creato.
2. Per leggere il contenuto di un file di testo, basta utilizzare la classe `FileReader`.
3. Una directory può essere creata grazie alla classe `File`.
4. La classe `File` definisce un metodo che restituisce i nomi dei file all'interno di una directory.

Esercizio 15.u)

Data la seguente classe:

```
import java.io.*;

public class Esercizio15R {
    public static void main(String args[]) throws IOException {
        try (FileOutputStream fos = new FileOutputStream("nuovo file.txt");
            DataOutputStream dos = new DataOutputStream(fos);) {
            dos.writeInt(8);
            dos.writeDouble(0.1176);
        }
    }
}
```

Una volta creato il file nuovo file.txt, quanto spazio occuperà sull'hard disk?

1. Non verrà creato nessun file perché il file non viene chiuso correttamente.
2. 12 byte.
3. Non verrà creato nessun file perché il file non compila.
4. 64 byte.
5. Non verrà creato nessun file perché durante l'esecuzione sarà lanciata un'eccezione.
6. 128 byte.
7. La dimensione del file dipende dalla piattaforma su cui viene eseguito il programma.
8. 96 byte.

Esercizio 15.v)

Data la seguente classe:

```
import java.io.*;

public class Esercizio15S {
```

```
public static void main(String args[]) throws Exception {
    try (FileOutputStream fos = new FileOutputStream("nuovo file.txt");
        DataOutputStream dos = new DataOutputStream(fos);) {
        for (int i = 0; i < 50; i++) {
            dos.writeInt(i);
        }
    }
}
```

Scrivere la classe che legge il file creato.

Esercizio 15.w)

Quali delle seguenti affermazioni sono corrette?

1. Per istanziare un `Reader` dobbiamo gestire una `IOException`.
2. Per scrivere da un `OutputStream` dobbiamo gestire una `IOException`.
3. Per leggere da un oggetto `InputStream` dobbiamo gestire una `IOException`.
4. Per istanziare un oggetto `Writer` dobbiamo gestire una `FileNotFoundException`.

Esercizio 15.x)

Quali delle seguenti affermazioni sono corrette?

1. È importante chiudere uno stream, altrimenti la memoria che occupa non sarà deallocata sino al termine del programma.
2. L'unica classe che mette a disposizione Java per leggere in maniera interattiva istruzioni da riga di comando, è la classe `Scanner`.
3. La classe `BufferedReader` dichiara un metodo chiamato `readLine` che legge una riga di testo intera dalla fonte passata in input al `BufferedReader`.
4. La classe `BufferedReader` dichiara un metodo chiamato `lines` che restituisce un oggetto `Stream<String>`.

Esercizio 15.y)

Quali delle seguenti affermazioni sono corrette?

1. `Path` definisce un metodo che restituisce i nomi dei file all'interno di una directory.
2. `Path` dichiara un metodo per recuperare la directory in cui è contenuto.
3. `Path` dichiara un metodo per recuperare l'autore (l'owner) del file.
4. `Path` è un'interfaccia.

Esercizio 15.z)

Quali delle seguenti affermazioni sono corrette?

1. La classe `Files` definisce un metodo che restituisce i nomi dei file all'interno di una directory.
2. Il metodo `find` della classe `Files` restituisce uno `Stream` di `Path`.

3. `Files` e `Path` appartengono al package `java.nio2.file`.
4. `Files` è un'interfaccia.
5. È possibile copiare un file in un altro con il metodo `copy`.

Esercizio 15.aa) Networking, Vero o Falso:

1. In una comunicazione di rete devono esistere almeno due socket.
2. Un client, per connettersi ad un server, deve conoscere almeno il suo indirizzo IP e la porta su cui si è posto in ascolto.
3. Un server si può mettere in ascolto anche sulla porta 80, la porta di default dell'HTTP, senza per forza utilizzare quel protocollo. È infatti possibile anche che si comunichi con il protocollo HTTP su una porta diversa dalla 80.
4. Il metodo `accept` blocca il server in uno stato di “attesa di connessioni”. Quando un client si connette, il metodo `accept` viene eseguito per raccogliere tutte le informazioni del client in un oggetto di tipo `Socket`.
5. Un `ServerSocket` non ha bisogno di dichiarare l'indirizzo IP, ma deve solo dichiarare la porta su cui si aspetterà connessioni.

Esercizio I.a) Date-Time API, Vero o Falso:

1. I metodi di tipo “from” permettono di recuperare un certo tipo temporale, a partire da un tipo temporale con più informazioni.
2. L'unico modo per fare il parsing di una stringa per ottenere un oggetto `Instant`, è passare tramite la classe `DateTimeFormatter`.
3. La classe `Duration` calcola la distanza tra due istanti, quindi è definita sulla timeline.
4. Non è possibile memorizzare informazioni sull'orario in un oggetto di tipo `LocalDate`.
5. È possibile memorizzare informazioni sulla data in un oggetto di tipo `LocalTime`.
6. È possibile memorizzare informazioni sulla data in un oggetto di tipo `ZonedDateTime`.
7. Il metodo `between` di `ChronoUnit` restituisce un oggetto `Duration`.
8. I temporal adjusters possono essere passati a metodi di tipo “with” per compiere operazioni su date e orari.
9. I temporal queries possono essere passati a metodi di tipo “with” per recuperare informazioni su date e orari.
10. In generale è possibile sostituire la classe `Date` con la classe `Instant`.

Esercizio I.b)

Creare una classe `DateUtils` che dichiari solo metodi statici (e che quindi è inutile istanziare). Questa classe, dati due istanti in input, deve essere capace di restituire il numero di:

- secondi
- minuti
- ore
- giorni
- settimane

- mesi

che intercorrono tra i due istanti.

Esercizio I.c)

Modificare la classe `DateUtils` per dotarla di un metodo che, dato un istante in input, deve essere capace di restituire il numero di:

- secondi
- minuti
- ore
- giorni
- settimane
- mesi

che intercorrono tra l'istante specificato e l'istante attuale.

Esercizio I.d)

Nella classe `DateUtils` creare un metodo che restituisca l'ora esatta nel formato “HH:mm ss”.

Esercizio I.e)

Nella classe `DateUtils` creare un metodo che formatti la data specificata secondo il pattern specificato in input.



Esercizio I.f)

Nella classe `DateUtils` creare un metodo che analizzi la data specificata secondo il pattern specificato in input, e restituisca una `LocalDate`.



Esercizio I.g)

Creare una classe `TestDateUtils` per testare la correttezza dei metodi di `DateUtils`.

Esercizio I.h)

Quali tra le seguenti affermazioni sono corrette riguardo la standardizzazione dei metodi della libreria Date And Time API?

1. I metodi `from` hanno un parametro sempre meno completo rispetto al tipo che devono ritornare.
2. I metodi `of` ritornano istanze di oggetti sulla classe su cui sono invocati.
3. I metodi `plus` e `minus` restituiscono copie dei parametri in input.
4. `with`, `is`, `to` e `at` sono usati solo come prefissi, mai come nomi completi.

Esercizio I.i)

Quali tra le seguenti affermazioni sono corrette riguardo la geolocalizzazione della libreria `Date And Time API`?

1. La classe `ZoneId` astrae il concetto di area geografica che condivide lo stesso orario, ed è solitamente individuata da una coppia del tipo “regione/città”.
2. La classe `ZoneOffset` astrae il concetto di fuso orario.
3. La classe `ZoneDateTime` rappresenta la versione localizzata della classe `LocalDate`.
4. Il metodo `atZone` della classe `LocalDateTime` restituisce una `ZoneDateTime`.

Esercizio I.j)

Quali tra le seguenti affermazioni sono corrette riguardo la gestione del codice legacy della libreria `Date And Time API`?

1. Il metodo `from` è dichiarato sia nella classe `GregorianCalendar` sia nella classe `Date`.
2. Il metodo `toInstant` per ottenere un'istanza della classe `Instant`, è dichiarato sia nella classe `Calendar` sia nella classe `Date`.
3. È possibile sostituire la classe `Date` con la classe `Instant`.
4. La classe `GregorianCalendar` può essere sostituita a seconda dei casi con i tipi `ZonedDateTime`, `LocalTime` o `LocalDate`.
5. La classe `TimeZone` può essere sostituita a seconda dei casi con i tipi `ZoneId` o `ZoneOffset`.

Esercizi del capitolo 16

Moduli

Non sarà facile iniziare ad utilizzare i moduli per i programmatori. Si tratta di un concetto che abbraccia una branca dell'informatica diversa: l'architettura software. Gli esercizi presentati di seguito puntano prima a chiarire tutti i concetti teorici, per poi creare moduli in maniera progressiva sino a creare un'architettura sostenibile, per l'applicazione che simula una rubrica creata con gli esercizi del modulo precedente (e di quello successivo).

Gli esercizi sono stati definiti senza utilizzare IDE, se state utilizzando un IDE, dovrete essere consapevoli di come vengono organizzati automaticamente i moduli.

Esercizio 16.a)

Quali delle seguenti affermazioni sono corrette:

1. Con il sistema modulare possiamo evitare eccezioni al runtime come `ClassNotFoundException`.
2. Il forte incapsulamento permette di rendere accessibile un certo package solo ai package specificati.
3. Nel JDK 9 il file `rt.jar` è stato eliminato.
4. La JVM della versione 9 quando esegue un programma che usa i moduli deve gestirli, e quindi risulterà meno performante rispetto a quando eseguirà un programma che non utilizza i moduli.
5. Possiamo evitare di utilizzare i moduli perché esiste il concetto di modulo anonimo.

Esercizio 16.b)

Quali delle seguenti affermazioni sono vere?

1. Un file JAR modulare ha estensione `.jmod`.
2. Il JDK 9 ha definito nuove applicazioni come `jlink` e molte altre applicazioni come `jdeps` del JDK sono state modificate per supportare l'introduzione dei moduli.
3. Un modulo è costituito da un unico file.
4. Le classi di alcuni package come `sun.*`, e `*.internal.*` sono state eliminate.
5. Un modulo automatico è un file JAR puntato dal `modulepath`.

Esercizio 16.c)

Quali delle seguenti affermazioni sono vere?

1. Le parole che definiscono le direttive del modulo (`module`, `open`, `opens`, `provides`, `requires`, `to`, `transitive`, `uses` e `with`) sono dette "restricted word".

2. Sia i package sia i moduli seguono la stessa convenzione per i nomi.
3. È possibile annotare il descrittore di un modulo `module-info.java` con alcune annotazioni.
4. Il descrittore di un modulo `module-info.class` deve trovarsi nella directory radice del modulo.

Esercizio 16.d)

Spiegare cosa significa eseguire il seguente comando:

```
javac -d mods/com.domain.mymodule src/com.domain.mymodule/com/domain/*  
src/com.domain.mymodule/module-info.java
```

Esercizio 16.e)

Spiegare cosa significa eseguire il seguente comando:

```
java --module-path mods -m com.domain/com.domain.HelloModularWorld
```

Esercizio 16.f)

Definire i seguenti concetti riguardante l'architettura software:

1. Componente software
2. Partizionamento verticale
3. Framework (partizionamento orizzontale)
4. Coesione
5. Accoppiamento
6. Sottosistema
7. Principio di inversione della dipendenza

Esercizio 16.g)

Immaginiamo di avere creato un software gestionale per un negozio di computer. Immaginiamo anche di avere un modulo che contiene il codice che gestisce la fatturazione del negozio, e chiamiamolo `negozio.fatturazione`. Questo modulo contiene i package `negozio.fatturazione.articoli`, `negozio.fatturazione.algoritmiinterni` e `negozio.fatturazione.funzionidisponibili`. Poi consideriamo un altro modulo che contiene il codice che rappresenta l'interfaccia grafica dell'applicazione, e chiamiamolo `negozio.interfacciagrafica`. Dichiarare i relativi descrittori che esplicitano la dipendenza che esiste tra questi due moduli.

Esercizio 16.h)

Partendo dalla soluzione dell'esercizio precedente dichiarare il descrittore di un modulo che contiene il codice che permetta la vendita di articoli chiamato `negozio.vendita`, ed eventualmente modificare i descrittori già definiti. Tenere presente che:

- Il modulo `negozio.vendita` contiene i package chiamati `negozio.vendita.funzionidisponibili`, `negozio.vendita.articoli` e `negozio.vendita.algoritmiinterni`.
- Dall'interfaccia grafica sarà possibile vendere articoli del negozio.
- Contestualmente alla vendita deve essere possibile fatturare l'articolo venduto.

Esercizio 16.i)

Partendo dalla soluzione dell'esercizio precedente, cosa possiamo fare se volessimo che il modulo `negozio.fatturazione` legga `negozio.interfacciagrafica`?

Esercizio 16.l)

Partendo dalla soluzione dell'esercizio 16.h, cosa possiamo fare per permettere al modulo `negozio.interfacciagrafica` di accedere tramite reflection ad un metodo privato definito nel package `negozio.fatturazione.algoritmiinterni`?

Esercizio 16.m)

Consideriamo la soluzione dell'esercizio precedente. Se non è possibile modificare più il nostro codice, ma ci siamo accorti che una classe del package `negozio.vendita` deve utilizzare un metodo privato del package `negozio.fatturazione.algoritmiinterni` tramite reflection, cosa possiamo fare?

Esercizio 16.n)

Quali delle seguenti affermazioni è corretta:

1. La classe `ServiceLoader` è stata introdotta già nella versione 6 di Java.
2. Il componente service provider interface (SPI) dipende dalle sue implementazioni.
3. Con la classe `ServiceLoader` possiamo eliminare completamente la dipendenza tra moduli.
4. Per implementare un servizio con `ServiceLoader`, le implementazioni di una service provider interface devono essere esportate dai rispettivi moduli.
5. Un metodo provider è una sorta di metodo factory.

Esercizio 16.o)

Partendo dalla soluzione dell'esercizio 15.n, definire dei package per le varie classi. Poi creare e compilare un modulo che esporta il package che contiene tutte le classi che rappresentano dati.

Esercizio 16.p)

Partendo dalla soluzione dell'esercizio 15.o, creare un modulo che esporti il package che contiene le eccezioni.

Per compilare con un unico comando più moduli solitamente si utilizza l'attributo `--module-source-path`. Per esempio per compilare l'esercizio del paragrafo 16.3 sul `ServiceLoader`, abbiamo usato il seguente comando:

```
javac -d mods --module-source-path
  src src/com.claudiodesio.spi/module-info.java
  src/com.claudiodesio.spi/com/claudiodesio/spi/*
  src/com.claudiodesio.invs/module-info.java
  src/com.claudiodesio.invs/com/claudiodesio/invs/*
  src/com.claudiodesio.certs/module-info.java
  src/com.claudiodesio.certs/com/claudiodesio/certs/*
  src/com.claudiodesio.factory/module-info.java
  src/com.claudiodesio.factory/com/claudiodesio/factory/*
  src/com.claudiodesio.handlers/module-info.java
  src/com.claudiodesio.handlers/com/claudiodesio/handlers/*
  src/com.claudiodesio.client/module-info.java
  src/com.claudiodesio.client/com/claudiodesio/client/*
```

(scritto tutto su una riga senza andare a capo).

Esercizio 16.q)

Partendo dalla soluzione dell'esercizio 16.p, si crei un modulo che esporti il package contenente le classi di utilità.

Esercizio 16.r)

Partendo dalla soluzione dell'esercizio 16.q, creare un modulo che esporti il package contenente la classe che gioca il ruolo di service provider interface. Il nostro obiettivo è quello di trasformare il modo in cui serializziamo i contatti con servizi di `ServiceLoader`.



Esercizio 16.s)

Riportiamo di seguito la classe `Esercizio16N` che rappresenta il client della rubrica creata negli esercizi del capitolo 15:

```
import java.util.function.*;

public class Esercizio15N {

    private GestoreSerializzazione<Contatto> gestoreFile;

    private Contatto[] contatti;

    Esercizio15N() {
```

```
        contatti = getContatti();
        gestoreFile = new GestoreFileNIO2();
    }

    private void eseguiTest() {
        System.out.println("TESTIAMO LA CREAZIONE DEI TRE CONTATTI");
        creaContatti();
        System.out.println("RECUPERIAMO I TRE CONTATTI");
        recuperaContatti();
        System.out.println(
            "TESTIAMO LA CREAZIONE DI UN CONTATTO GIÀ ESISTENTE");
        creaContattoEsistente();
        System.out.println("PROVIAMO A RECUPERARE UN CONTATTO NON ESISTENTE");
        recuperaContattoNonEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO ESISTENTE");
        modificaContattoEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO ESISTENTE");
        rimuoviContattoEsistente();
        System.out.println("MODIFICHIAMO UN CONTATTO NON ESISTENTE");
        modificaContattoNonEsistente();
        System.out.println("RIMUOVIAMO UN CONTATTO NON ESISTENTE");
        rimuoviContattoNonEsistente();
    }

    public void creaContattoEsistente() {
        esegui()->gestoreFile.inserisci(contatti[0]);
    }

    public void modificaContattoEsistente() {
        esegui()->gestoreFile.modifica(new Contatto(
            "Daniele", "Via dei microfoni 1", "07890"));
    }

    public void rimuoviContattoEsistente() {
        esegui()->gestoreFile.rimuovi(contatti[2].getNome());
    }

    public void modificaContattoNonEsistente() {
        esegui()->gestoreFile.modifica(
            new Contatto("Pluto", "Via dei microfoni 1", "07890"));
    }

    public void rimuoviContattoNonEsistente() {
        esegui()->gestoreFile.rimuovi("Ligeia");
    }

    public void recuperaContattoNonEsistente() {
        esegui()->gestoreFile.recupera("Pippo");
    }

    public void creaContatti() {
        for (Contatto contatto : contatti) {
            System.out.println("Creazione contatto:\n"+ contatto);
            creaContatto(contatto);
        }
    }

    public void recuperaContatti() {
        for (Contatto contatto : contatti) {
            System.out.println("Recupero contatto: "+ contatto.getNome());
        }
    }
}
```

```
        recuperaContatto(contatto.getNome());
    }
}

public void recuperaContatto(String nomeContatto) {
    esegui()->gestoreFile.recupera(nomeContatto);
}

private void creaContatto(Contacto contatto) {
    esegui()->gestoreFile.inserisci(contatto);
}

private Contacto[] getContatti() {
    Contacto contattol = new Contacto(
        "Daniele", "Via delle chitarre 1", "01234560");
    Contacto contatto2 = new Contacto(
        "Giovanni", "Via delle scienze 2", "0565432190");
    Contacto contatto3 = new Contacto(
        "Ligeia", "Via dei segreti 3", "07899921");
    Contacto[] contatti = {
        contattol, contatto2, contatto3
    };
    return contatti;
}

public <O> O esegui(Retriever<O> retriever) {
    O output = null;
    try {
        output = retriever.esegui();
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
    return output;
}

public void esegui(Executor executor) {
    try {
        executor.esegui();
    } catch (Exception exc) {
        System.out.println(exc.getMessage());
    }
}

public static void main(String args[]) {
    Esercizio15N esercizio15N = new Esercizio15N();
    esercizio15N.eseguiTest();
}
}
```

Abbiamo riportato in grassetto le istruzioni su cui concentrare la nostra attenzione. Si noti come nel costruttore viene creato esplicitamente un oggetto della classe `GestoreNIO2`, ed assegnato ad un reference di tipo `GestoreSerializzazione`. Il nostro obiettivo per questo esercizio sarà quello di creare due moduli diversi per trasformare in servizi i due modi in cui abbiamo serializzato gli oggetti `Contacto` nel capitolo 15 `GestoreFile` e `GestoreNIO2`. Poi nel prossimo esercizio li utilizzeremo.

**Esercizio 16.t)**

Partendo dalla soluzione dell'esercizio 16.s, rinominare la classe `Esercizio15N` in `Esercizio16T`, e sfruttare il `ServiceLoader` per caricare i servizi. Fare in modo che sia possibile specificare da riga di comando quale gestore di serializzazione si deve usare. Poi creare un modulo che contenga tale classe ed eseguire l'applicazione.

**Esercizio 16.u)**

Partendo dalla soluzione dell'esercizio 16.t, creare una classe `factory` e spostare al suo interno il metodo `getGestoreSerializzazione` presente nella classe `Esercizio16T` e il relativo modulo. Modificare di conseguenza anche il descrittore del modulo `com.claudiodesio.rubrica.test`.

Esercizio 16.v)

Impacchettare i moduli creati nei rispettivi JAR modulari ed eseguire l'applicazione.

Esercizio 16.z)

Con `jlink` creare un ambiente personalizzato con il solo modulo `java.base`. Poi utilizzando la cartella `lib` dell'esercizio precedente eseguire il nostro JAR modulare eseguibile tramite il runtime appena creato.

Esercizi del capitolo 17 e dell'appendice G

Java Database Connectivity e package java.util

Gli esercizi pratici di questo capitolo utilizzeranno come database Apache Derby, ma ovviamente è possibile utilizzare anche altri database se il lettore ha le conoscenze giuste. Potete trovare nell'appendice J, un'introduzione all'installazione e l'uso di questo RDBMS. L'integrazione delle nostre applicazioni con Derby, ovviamente aggiungerà complessità. Inoltre troverete anche esercizi che riguardano gli argomenti trattati nella appendice G. Questi esercizi hanno una numerazione diversa che inizia con la lettera G.

Esercizio 17.a) JDBC, Vero o Falso:

1. `Connection` è solo un'interfaccia.
2. Un'applicazione JDBC è indipendente dal database solo se si parametrizzano le stringhe relative al driver, lo URL di connessione, lo username e la password.
3. Se si inoltra ad un particolare database un comando non standard SQL 2, questo comando funzionerà solo su quel database. In questo modo si perde l'indipendenza dal database, a meno di controlli o parametrizzazioni.
4. Per eliminare un record bisogna utilizzare il metodo `executeQuery`.
5. Per aggiornare un record bisogna utilizzare il metodo `executeUpdate`.
6. `CallableStatement` è una sottointerfaccia di `PreparedStatement`.
`PreparedStatement` è una sottointerfaccia di `Statement`.
7. Per eseguire una stored procedure bisogna utilizzare il metodo `execute`.
8. L'auto-commit è impostato a `true` per default.
9. In ambienti enterprise dove si eseguono applicazioni in contesti Java EE, `DataSource` va utilizzato in luogo di `Driver`.
10. Il tipo `CHAR` di SQL si mappa al tipo `char` di Java.



Esercizio 17.b)

Creare una classe chiamata `DBGenerator` che crea con istruzioni DDL, un database eseguendo le istruzioni contenute nei file `DBConfig.txt` e `DBScript.txt` forniti nella cartella 17.b del file del codice del libro. In particolare le istruzioni del file `DBConfig.txt` sono le seguenti:

```
db.url=jdbc:derby:Rubrica;create=true  
db.user=  
db.pwd=
```

Dovremo leggeremo questo file utilizzando un oggetto di tipo `Properties`, visto che contiene coppie di tipo chiave-valore. Notiamo che non sono stati definiti valori per username e password, quindi il programma non dovrà leggerle (sono state riportate solo per rendere l'idea del file di configurazione). Notare anche che la "url" per connettersi al database è leggermente diversa da

quella che ci si potrebbe aspettare rispetto agli altri database. Infatti, questa sintassi, una volta letta con JDBC, creerà uno schema chiamato *Rubrica*, come visto già nel paragrafo 17.2.4.

Poi il programma dovrà eseguire le seguenti istruzioni che sono contenute nel file *DBScript.txt*:

```
CREATE TABLE CONTATTO (NOME VARCHAR(50), TELEFONO VARCHAR(20), INDIRIZZO
VARCHAR(255), PRIMARY KEY (NOME))
INSERT INTO CONTATTO (NOME, TELEFONO, INDIRIZZO) VALUES ('Claudio De Sio
Cesari', '091011121314151617', '17, Java Street')
INSERT INTO CONTATTO (NOME, TELEFONO, INDIRIZZO) VALUES ('Stevie Wonder',
'1010101010', '10, Music Avenue')
INSERT INTO CONTATTO (NOME, TELEFONO, INDIRIZZO) VALUES ('Gennaro Capuozzo',
'1111111111', '1, Four Days of Naples Square')
```

Questo file potrebbe essere letto ottenendo un oggetto *Stream* chiamando il metodo *list* della classe *Files* (leggere la documentazione).

Una volta creato il database, dovrebbe essere creata la cartella *Rubrica* all'interno della cartella di dove è stato lanciato il file. Per controllare il contenuto del database, è possibile utilizzare il programma *IJ* (vedi Appendice K), oppure procedere con il prossimo esercizio.

È fondamentale settare la variabile *classpath* in modo tale che punti al file *derby.jar* (vedi Appendice E)

Esercizio 17.c)

Partendo dalla soluzione dell'esercizio precedente, creare una classe chiamata *DBExplorer*, che definisce un metodo chiamato *printContacts* che stampa il contenuto della tabella *CONTATTO*.



Esercizio 17.d)

Partendo dall'esercizio 15.q e dal database creato nell'esercizio 17.b e 17.c, creare una classe *GestoreDB* che implementa l'interfaccia *GestoreSerializzazione*. In particolare:

1. Implementare il metodo *getContatti* per recuperare i contatti dal database.
2. Dal momento che tale metodo (ed anche gli altri che implementeremo nei prossimi esercizi) lancerà l'eccezione *SQLException*, bisogna modificare l'interfaccia *GestoreSerializzazione* in modo tale che rimanga compatibile con la dichiarazione dei metodi anche delle altre implementazioni (*GestoreFile* e *GestoreFileNIO2*).
3. Di conseguenza bisognerà apportare anche delle modifiche alla classe *RubricaCLI* affinché compili.
4. Modificare il file *config.properties* in modo tale che venga caricata la classe *GestoreDB* per gestire la serializzazione.
5. Testare il corretto funzionamento dell'applicazione (per ora deve funzionare solo il caricamento dei contatti dal database *Rubrica*).

Esercizio 17.e)

Partendo dall'esercizio 17.d, implementare il metodo `recupera` e testare il corretto funzionamento dell'applicazione.

Esercizio 17.f)

Partendo dall'esercizio 17.e, implementare il metodo `inserisci` e testare il corretto funzionamento dell'applicazione.

Esercizio 17.g)

Partendo dall'esercizio 17.f, implementare il metodo `modifica` e testare il corretto funzionamento dell'applicazione.

Esercizio 17.h)

Partendo dall'esercizio 17.g, implementare il metodo `rimuovi` e testare il corretto funzionamento dell'applicazione.

Esercizio G.a) *Package java.util, Vero o Falso:*

La classe `Properties` estende `Hashtable` ma consente di salvare su un file le coppie chiave-valore rendendole persistenti.

La classe `Locale` astrae il concetto di “zona”.

La classe `ResourceBundle` rappresenta un file di properties che permette di gestire l'internazionalizzazione. Il rapporto tra nome del file e `Locale` specificato per individuare tale file, permetterà di gestire la configurazione della lingua delle nostre applicazioni.

L'output del seguente codice:

```
StringTokenizer st = new StringTokenizer(
    "Il linguaggio object oriented Java", "t", false);
while (st.hasMoreTokens()) {
    System.out.println(st.nextToken());
}
```

sarà:

```
Il linguaggio objec
t
orien
t
ed Java
```

Il seguente codice non è valido:

```
Pattern p = Pattern.compile("\\bb");
Matcher m = p.matcher("blablabla...");
boolean b = m.find();
System.out.println(b);
```

La classe `Preferences` permette di gestire file di configurazione con un file XML.

La classe `Formatter` definisce il metodo `printf`.

L'espressione regolare `[a]` è un quantificatore (greedy quantifier).

Il seguente codice:

```
Date d = new Date();  
d.now();
```

crea un oggetto `Date` con la data attuale.

Un `SimpleDateFormat` può formattare e analizzare qualsiasi valuta grazie alla specifica di un `Locale`.



Esercizio G.b)

Creare una classe `Traduttore` che espone un metodo `traduci` per tradurre un numero limitato di parole dall'inglese all'italiano e viceversa, utilizzando un resource bundle.

Esercizio G.c)

Creare una classe `TestTraduttore` per testare la correttezza delle traduzioni.



Esercizio G.d)

Creare una classe `StringUtils` che dichiara solo metodi statici (e che quindi è inutile istanziare). Essa deve esporre un metodo `ricerca`, che tramite espressioni regolari deve ricercare tutte le parole in un testo (specificato in input) che iniziano con un certo carattere, e restituirle all'interno di una lista.

Esercizio G.e)

Creare una classe `TestStringUtils` per testare la correttezza del metodo `ricerca`.

Esercizio G.f)

Creare un semplice programma che simuli il lancio di un dado. Eseguendo il programma sarà stampato un numero casuale da 1 a 6. Si utilizzi la documentazione ufficiale per trovare un modo per generare numeri casuali.

Suggerimento: basta un'unica classe con un metodo `main` contenente un unico statement.



Esercizio G.g)

Riprendiamo il discorso iniziato nell'esercizio 5.w, e implementato nell'esercizio 14.q, introdotto nell'approfondimento 5.7.

Nel caso d'uso dell'autenticazione, un utente inseriva uno username e una password per autenticarsi nel sistema. Ma chi ha inserito nel sistema le credenziali per permettere all'utente di autenticarsi con username e password? La risposta la troviamo nel diagramma dei casi d'uso, riportato nuovamente nella figura G.g.1.

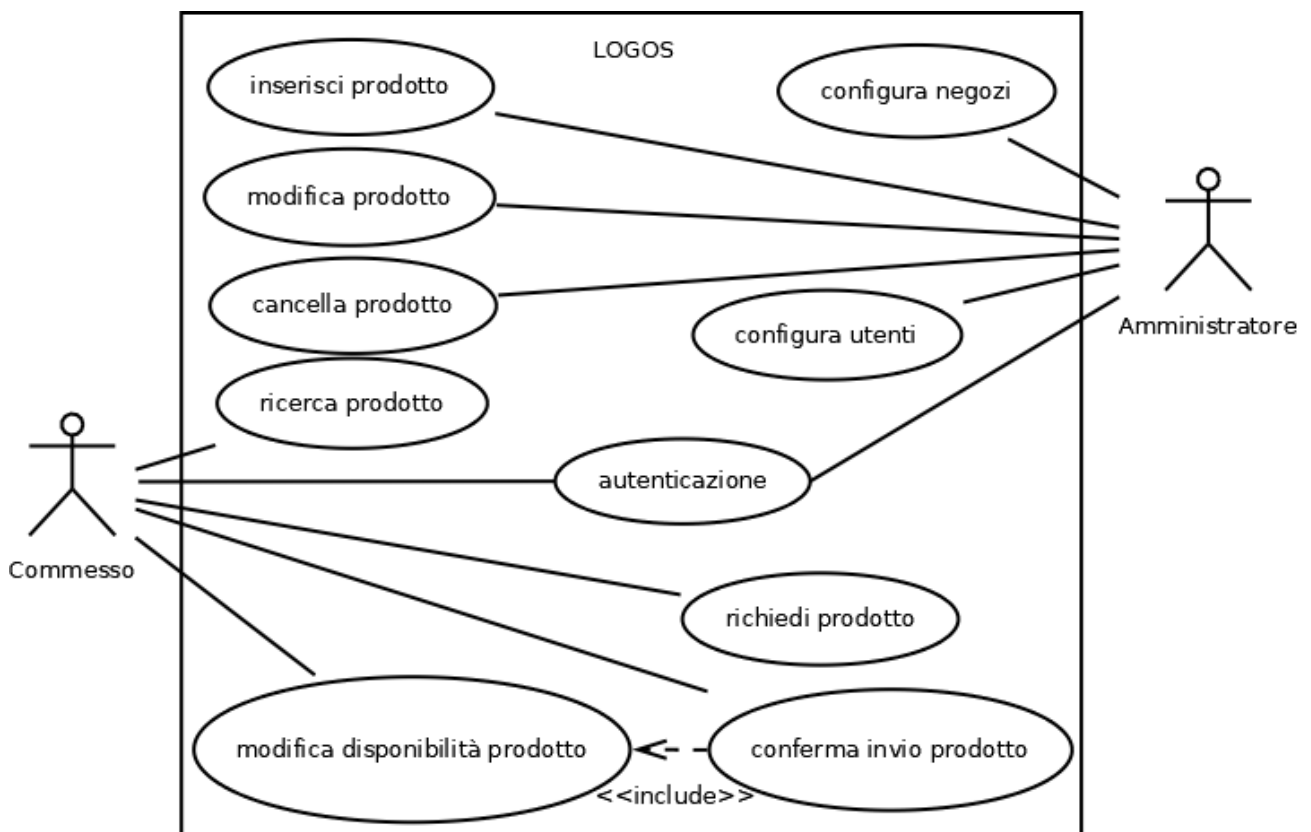


Figura G.g.1 - Use case diagram aggiornato di Logos.

Il caso d'uso *configura utenti*, è il caso d'uso che questo esercizio richiede di implementare. Anche questo caso d'uso deve essere inteso come un piccolo programma funzionante, indipendentemente da Logos, in modo tale da poterlo eventualmente riutilizzare in altri programmi. Per poter immagazzinare le coppie username e password, utilizzare un file di properties. Tenere presente che oltre a username e password, la classe `Utente` deve avere anche altre proprietà da configurare, come il nome e il ruolo.

Riutilizzare il codice degli esercizi 14.q e degli altri esercizi correlati.

Esercizio G.h)

Tenendo presente la soluzione dell'esercizio precedente, modificare il codice dell'esercizio 6.z ed in particolare la classe `ProfiliUtenti`, in modo tale da caricare gli oggetti `Utente` a partire dal file `di.properties`.

Esercizio G.i)

Quali tra le seguenti affermazioni sono corrette?

1. Un oggetto `Formatter` permette di immagazzinare coppie chiave-valore come un oggetto `Properties`, senza il bisogno di dover specificare un file da utilizzare per l'immagazzinamento dei dati.
2. Un oggetto `System.out` è di tipo `java.io.PrintStream`.
3. La classe `java.io.PrintStream` definisce il metodo `printf` che sfrutta il metodo `format` della classe `Formatter`.
4. La classe `Formatter` definisce un overload del metodo `format`.
5. La classe `Formatter` permette di formattare anche in base a oggetti `Locale`.

Esercizio G.j)

Quali tra le seguenti affermazioni sono corrette riguardo la classe `Preferences`?

1. Un oggetto `Preferences` permette di immagazzinare coppie chiave-valore come un oggetto `Properties`, senza il bisogno di dover specificare un file da utilizzare per l'immagazzinamento dei dati.
2. Un oggetto `Preferences` non usa il metodo `setProperty`.
3. La classe `Preferences` estende `Hashtable`.
4. La classe `Preferences` si istanzia senza poter utilizzare costruttori, ma metodi che restituiscono istanze. Infatti è addirittura una classe astratta.
5. La classe `Preferences` definisce metodi per inserire specifici tipi di dati come valore.

Esercizio G.k)

Quali tra le seguenti affermazioni sono corrette riguardo le regular expression?

1. La classe `Matcher` definisce il metodo `find` che ritorna la stringa che coincide con l'espressione booleana di `Matcher`.
2. La classe `Matcher` si istanzia senza poter utilizzare costruttori, ma metodi che restituiscono istanze.
3. Un oggetto `Pattern` definisce i metodi `start` e `end`.
4. All'interno di stringhe Java, il simbolo `\` deve essere ripetuto, per evitare che il compilatore lo consideri il prefisso di un carattere di escape.
5. Un "greedy quantifier" è un simbolo che simboleggia la molteplicità di occorrenze coincidenti con un certo pattern.

Esercizi del capitolo 18 e dell'appendice M

Java e il mondo XML e JAXB

Qui troverete sia esercizi sul capitolo 18 che sull'appendice L. Questi ultimi hanno una numerazione diversa che inizia con la lettera L e verranno aggiunti a breve.

Esercizio 18.a) JAXP, Vero o Falso:

1. Per le specifiche DOM, ogni nodo è equivalente ad un altro e un commento viene visto come un oggetto di tipo `Node`.
2. Infatti l'interfaccia `Node` implementa `Text`.
3. Per poter analizzare un documento nella sua interezza con DOM, bisogna utilizzare un metodo ricorsivo.
4. Con il seguente codice:
`Node n = node.getParentNode().getFirstChild();`
si raggiunge il primo nodo "figlio" di `node`.
5. Con il seguente codice:
`Node n = node.getParentNode().getPreviousSibling();`
si raggiunge il nodo "fratello" precedente di `node`.
6. Con il seguente codice:
`NodeList list = node.getChildNodes();`
`Node n = list.item(0);`
si raggiunge il primo nodo "figlio" di `node`.
7. Con il seguente codice:
`Element element = doc.createElement("nuovo");`
`doc.appendChild(element);`
`Text text = doc.createTextNode("prova testo");`
`doc.insertBefore(text, element);`
viene creato un nodo chiamato `nuovo`, in cui viene aggiunto `testo`.
8. La rimozione di un nodo provoca la rimozione di tutti i suoi nodi "figli".
9. Per analizzare un documento tramite l'interfaccia SAX bisogna estendere la classe `DefaultHandler` e effettuare override dei suoi metodi.
10. Per trasformare un file XML e serializzarlo in un altro file dopo una trasformazione mediante un file XSL, è possibile utilizzare il seguente codice:

```
try {  
    TransformerFactory factory = TransformerFactory.newInstance();  
    Source source = new StreamSource(new File("input.xml"));  
    Result result = new StreamResult(new File("output.xml"));  
    Templates template = factory.newTemplates(  
        new StreamSource(new FileInputStream("transformer.xsl")));  
    Transformer transformer = template.newTransformer();  
    transformer.transform(source, result);  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

Esercizi del capitolo 19

Introduzione alle Interfacce grafiche (GUI)

Qui troverete esercizi sul capitolo 19 (altri verranno aggiunti a breve).

Esercizio 19.a) GUI, AWT e Layout Manager, Vero o Falso:

1. Nella progettazione di una GUI è preferibile scegliere soluzioni considerate standard per facilitare l'utilizzo all'utente.
2. Nell'MVC il Model rappresenta i dati, il Controller le operazioni e la View l'interfaccia grafica.
3. Le GUI AWT sono invisibili di default.
4. Per ridefinire l'aspetto grafico di un componente AWT è possibile estenderlo e ridefinire il metodo `paint`.
5. AWT è basato sul pattern Decorator.
6. In un'applicazione basata su AWT è necessario sempre avere un top level container.
7. È impossibile creare GUI senza layout manager, otterremmo solo eccezioni al runtime.
8. Il `FlowLayout` cambierà la posizione dei suoi componenti in base al ridimensionamento.
9. Il `BorderLayout` cambierà la posizione dei suoi componenti in base al ridimensionamento.
10. Il `GridLayout` cambierà la posizione dei suoi componenti in base al ridimensionamento.

Esercizio 19.b) Gestione degli eventi, Applet e Swing, Vero o Falso:

1. Il modello a delega è basato sul pattern Observer.
2. Senza la registrazione tra la sorgente dell'evento e il gestore dell'evento, l'evento non sarà gestito.
3. Le classi innestate e le classi anonime non sono adatte per implementare gestori di eventi.
4. Una classe innestata può gestire eventi se e solo se è statica.
5. Una classe anonima per essere definita si deve per forza istanziare.
6. Un `ActionListener` può gestire eventi di tipo `MouseListener`.
7. Un pulsante può chiudere una finestra.
8. È possibile (ma non consigliabile) per un gestore di eventi estendere tanti adapter per evitare di scrivere troppo codice.
9. La classe `Applet`, estendendo `Panel`, potrebbe anche essere aggiunta direttamente ad un `Frame`. In tal caso però, i metodi sottoposti a override non verranno chiamati automaticamente.
10. I componenti di Swing (`JComponent`) estendono la classe `Container` di AWT.

Esercizi del capitolo 20

Introduzione a JavaFX

Qui troverete esercizi sul capitolo 20 (altri verranno aggiunti a breve).

Esercizio 20.a) *JavaFX, Vero o Falso:*

1. È possibile creare interfacce miste che usano librerie di Swing e di JavaFX.
2. Per visualizzare un'interfaccia grafica bisogna utilizzare un oggetto `Stage` posizionato su un oggetto `Scene`.
3. Le interfacce create con JavaFX si creano usando `layout pane`, o quali sono pannelli che definiscono come i componenti aggiunti su di essi devono essere posizionati.
4. Il `GridPane` permette di posizionare i suoi componenti in una griglia le cui celle devono essere della stessa dimensione.
5. Il linguaggio FXML permette di usare gli stessi componenti grafici che utilizza JavaFX in maniera dichiarativa.
6. È possibile usare un file CSS solo con interfacce create con FXML.
7. JavaFX diversamente da Swing/AWT, non gestisce gli eventi con un modello a delega. Infatti si usano le espressioni `lambda` e non i gestori degli eventi.
8. Il `binding` permette a due oggetti che definiscono delle `property` JavaFX di legarsi e aggiornare il loro stato in base all'aggiornamento dell'altro oggetto.
9. Gli effetti speciali definiti JavaFX che permettono il movimento, terminano con il suffisso `Motion`, infatti estendono la classe astratta `Motion`.
10. È possibile eseguire un'applet con le librerie JavaFX.